The Ledger

Specification, Version 16.1

R. Delloye

July 2026

Contents

Abstract

Reconciliation exists because many systems store the same facts, and independently maintained copies eventually disagree. The Ledger removes the cause: every fact is recorded exactly once, in an immutable log of atomic transactions, and every other number — balances, valuations, profit and loss, reports — is a deterministic projection of that single record, reproducible by any party and reconstructible at any past date. This specification defines the system in full, rated one-way against the Ledger Framework Constitution v1.4: the objects the system manipulates and the signed coordinate vector every balance generalises to; the three machines that evaluate the pipeline — the Event Monitor, the Events Executor, and the Transaction Executor, three roles and one writer; smart contracts as pure, stateless transcriptions of legal terms; the three homes of state; valuation and profit and loss; ingestion, corporate actions, and the market data operator; collateral, margin, and lending under the declared legal regime of the governing agreement; virtual ledgers and the total return swap; the settlement interface; presentation and reporting projections; alignment with the Common Domain Model; and the invariant catalogue and executable-check regime that make every guarantee a property a machine can run. Six instruments with fixed terms and fixed quantities recur through every chapter, so that each claim is exercised on the same worked facts it governs.

Contents

1 The Objective and the Commitments

Governed by C-1.1–1.5, C-2.1–2.8, C-Auth.1, C-Auth.4 (§16.5).

1.1 One record, or reconciliation

The objective, the diagnosis, and the remedy are the constitution's, and this specification cites them rather than re-arguing them. The objective is one clean post-trade and risk-management platform: a single system of record from which positions, valuations, profit and loss, collateral, and reports are all derived, exactly and reproducibly (C-1.1). The diagnosis is architectural — reconciliation is the cost of storing one fact in more than one place, and two independently maintained stores of a fact eventually disagree (C-1.2–C-1.3). The remedy is to keep exactly one canonical record — an ordered, append-only stream of atomic transactions — of which every reported number is a projection, so that two projections of one record cannot disagree about any fact it contains (C-1.4–C-1.5).

That last sentence is the design's entire ambition; the rest of this specification is what it forces. Its scope — the recording, valuation, and lifecycle of trading-book positions held at fair value, and the projection of committed activity into settlement — is fixed component by component in Chapter 17, together with the external authorities the ledger reconciles against at its boundary and never replaces.

1.2 The eight commitments

Eight commitments are non-negotiable, and every design decision in this specification is tested against them. Each is stated here with the one consequence it forces on the rest of the document.

Full auditability. Every number is traceable, mechanically and completely, to the recorded events that produced it. History is never rewritten: a mistake is repaired by a recorded correction that names what it supersedes, so the error and its repair both survive. This commitment forces the immutable, hash-chained log of Chapter 2 and Chapter 3, and the repair discipline of Chapter 15: a wrong booking is corrected forward by a compensating transaction through the same admission door — agreed, authorised, and in the open — never by amendment of the past.

Full reproducibility. Given the same record, any party — the firm, a counterparty, an auditor, a regulator — recomputes the same state, the same views, the same contract outputs, to the last minor unit. This commitment forces every arrow of Chapter 2 to be a deterministic function of recorded inputs and every quantity in this document to be an exact integer in minor units; the single input not on the record, the clock, is confined to one machine, whose emissions are themselves recorded (Chapter 4).

Time travel embedded in the design. The exact state of the system at any past moment is reconstructible from the record alone, at any time, forever — independently of whether the software that produced the events still exists. This commitment forces replay to re-read admitted transactions and never to re-execute smart contracts; Chapter 14 states time travel as a theorem of the fold, not a feature bolted onto it.

Correctness. Illegal states are made unrepresentable wherever possible: a check can be forgotten or bypassed; a type cannot. Where wrongness cannot be prevented — a contract can always compute the wrong economics — it is made mechanically detectable and deterministically repairable. Defaults fail closed: the system stops rather than improvises. This commitment forces the single admission door of Chapter 4, the typed lifecycle graphs of Chapter 3, and the two layers of correctness of Chapter 15 — structural correctness guaranteed at admission, economic correctness made checkable by recomputation.

Testability. The ledger is correct because it is built to be tested. Every step is a pure, versioned function with explicit inputs, explicit outputs, and strong types — typing alone eliminates whole classes of defects before a single test runs — and every core invariant is

expressed as an executable check over generated products, events, and histories, never defended only in prose. Correctness then composes: because the steps are pure and their composition is fixed — the map, then the fold — verifying each part verifies the whole. This commitment forces the invariant catalogue of Chapter 14 and the executable-check regime of Chapter 15, and it forces a rule of method: a property that cannot be tested mechanically is redesigned until it can be.

Clarity. One name per component and one component per name; behaviour carried as declared data rather than hidden in code; every claim stated so that it can be checked by a reader with no prior exposure. This commitment fixes the vocabulary of this document — unit, wallet, balance, move, transaction, watch, the immutable log, projection, the Event Monitor, the Events Executor, the Transaction Executor, smart contract, the market data operator, the three homes, virtual wallet, virtual ledger — and forces the declared-data discipline that recurs in every chapter: terms, watches, market data operators, and collateral eligibility are data the record carries, never behaviour a reader must infer from code.

Order. Events do not commute: each event's transaction is computed against the ledger all earlier events have built, so the same events in a different order are a different history — buy shares then meet the dividend's record date and you are entitled; meet the record date then buy and you are not. Each event bears three times — *execution*, when it happened in the world; *monitor*, when the boundary observed it; *door*, when the single writer admitted it (Chapter 4) — and meaning lives in execution order, so the fold advances in execution order and a late arrival whose execution time precedes events already folded takes its place among them and the tail refolds. The log's total order is execution time, then door time, then the event's hash: part of the meaning of the record, never bookkeeping. This commitment forces the single writer and the single total order of Chapter 4, the one-event-at-a-time step of Chapter 2 — map one event, fold its transaction, only then map the next, so the transaction list is an output of the fold, never an input to it — and the refusal to reorder anything unless harmlessness is proved (Chapters 4 and 14).

Simulability. The current state is the initial state plus the fold of all transactions; the events that produced it are one path among the paths that could have occurred. The machinery must therefore run unchanged on hypothetical event streams: simulated paths branched from any recorded state, driven by generated events, folded by the same contracts and the same door, the one non-record input — the seed — recorded so that every path replays exactly. This commitment forces the purity of every contract (Chapter 5), the confinement of the clock to the Event Monitor (Chapter 4), and the wall that keeps effects — settlement, models — outside the fold (Chapters 8 and 11); the simulated ledger of Chapter 10 and the generator regime of Chapter 15 are its two readings.

The commitments are jointly non-negotiable, and a design is admitted only if it satisfies all eight: a design that is auditable and reproducible but untestable is redesigned, and a design that is elegant but cannot be replayed is rejected — not because one commitment outranks another, but because each is required and none is waived. No design decision in this document rests on custom or typical practice; each is derived from the commitments and then, where an external standard binds, shown to satisfy it. One boundary bounds all eight: they secure one consistent internal record, not its correspondence with the world outside — internal unbreakability is not external correctness, and whether a projection matches the market, the custodian, or the counterparty is reconciled at the boundary (Chapter 8), never assumed.

1.3 The discipline of the document

The direction of authority is one-way. This specification is rated against the Ledger Framework Constitution v1.4 and against nothing else; each chapter opens by naming the constitution sections it discharges, and a genuine conflict with the constitution is never drafted around — it is recorded in Chapter 17's open-problems index with the exact amendment it would require.

Six instruments run through the document as threads, introduced as a cast in Chapter 2: a cash-settled future, a one-touch option, a cash dividend, a stock split, an OTC variance swap, and a total return swap. Their terms, parties, dates, and quantities are fixed once and never vary, so a number met in one chapter can be checked against the same number in another. Each visit is an *episode* in a fixed five-part shape: the trigger; the contract that catches it; the transactions it produces, as numbered rows with exact quantities; what the door checks; and the design point it substantiates. All quantities are exact integers in minor units; amounts stated in whole currency are exact in cents.

Two conventions govern notation. No knowledge of category theory is assumed: every concept is explained in plain terms with a concrete example, and any categorical restatement is a marked *second telling* on which nothing later depends. Code appears only as short typed signatures and data declarations; the reference implementation lies outside this document entirely.

2 The Picture: Map, Then Fold

Governed by C-3.1–3.11 (§16.5).

Everything this framework does is a *map* followed by a *fold*. The constitution defines both words and draws the whole picture (C-3.1); this chapter does not re-derive them — it instantiates the picture on the six threads. In one line: the world presents a list of events, $E = [e]$, and the ledger is the fold of that list — one event at a time, each step mapping the event to its transaction tx against the ledger built so far, then folding that tx in, so the transactions $T = [tx]$ are output the fold produces and never a list handed to it; and every number the ledger reports — a balance, a valuation, a profit and loss, a report — is a further fold of that same record. Four terms recur, each defined in Chapter 3 and used here only at a gloss: a **unit** is anything that can be held or owed; a **wallet** is where units are held; a **transaction** is the atomic bundle of changes the ledger records; and a **projection** is any view computed from the record.

2.1 The cast: six threads

Six instruments recur through this specification as threads. This section is their cast: their terms, parties, and quantities are fixed here once and never vary, so a number met in one chapter is the same number met in another. Every later episode opens with a pointer back to this table and states only what that episode adds; a standing term is referred to here, never re-established. This chapter follows one of them — the one-touch OT-1 — through the whole pipeline; the rest are worked where they bite.

	Instrument, kind, parties	Fixed terms and headline numbers
T1	FUT-IDX — cash-settled future on IDX; W-ALPHA / the CCP	multiplier 10, USD, settled-to-market; buy 1 at 995.00; settlement prints 1,000.00 / 990.00 / 1,005.00; cumulative VM USD 100 = PnL
T2	OT-1 — one-touch on stock OMEGA; W-BANK writes, W-ALPHA holds	barrier breached when OMEGA's official close is at or below 80.00 — a discretely-monitored (closing-price) one-touch; payout USD 1,000,000; spot 95.00 at inception; marked USD 400,000 while live; knock at close 79.00
T3	ACME — cash dividend; issuer / W-ALPHA	1.50 per share; W-ALPHA holds 10,000 shares; entitlement USD 15,000
T4	ACME — 2-for-1 split; issuer / W-ALPHA	10,000 → 20,000 shares; price frame 100.00 → 50.00; declared dividend figure 1.50 → 0.75
T5	VS-1 — OTC variance swap on IDX; bilateral	one year, 252 fixings, strike 400 variance points, USD 1,000 per point; realised variance 441; settlement USD 41,000
T6	TRS-1 — total return swap; reference W-QIS in virtual ledger V-1 (strategy S-QIS)	notional USD 10,000,000; financing 4% p.a., quarterly; first reset NAV 10,300,000; net move USD 200,000

2.2 The world presents events

The world presents the system with a list of events. Some arrive from outside: trades executed in the market, corporate-action notices, fixings and other observations, captured and recorded at the boundary. Others are not received but noticed. Every unit declares, from its terms, the conditions its life requires watching — its **watches** (Chapter 3) — and the Event Monitor evaluates those watches against the clock and the recorded observations, emitting an event onto the record the moment a condition is met. The future FUT-IDX declares such watches at registration: one for each daily settlement print, one for its expiry; the one-touch OT-1 declares one, its underlying's recorded close against the barrier. A coupon date arriving, an expiry falling due, a close printing below a barrier — each becomes an event the same way a trade does, and once emitted the two kinds are indistinguishable: both are on the record.

2.3 The map: each event becomes a transaction

Each event is mapped to a transaction, and the responsibility for the map divides in two. The **smart contracts** are the mapping functions: each is a stateless piece of code that, given one event and the current state of the ledger, computes the transaction the event requires (Chapter 5). The **Events Executor** fires the map one event at a time as the fold advances: for each event, in the total order, it fires the responsible contract, hands it the event together with the current projection of the ledger, and collects the proposed transaction tx — so $T = [tx]$ is built by the fold, not consumed by it; *responsible* is a defined word, the routing the registry names (Chapters 4 and 8). Many contracts, one Events Executor; the contracts carry all the product knowledge, the Events Executor carries none (Chapter 4).

The map is not blind to state. To turn a dividend event into cash moves, the contract must read who holds the stock at that moment — the fold of everything admitted so far. Map and fold therefore advance together, one event at a time, in a single total order: the transaction for event e is computed against the ledger that the transactions for all earlier events have already built.

2.4 The fold: the transactions become the ledger

There is no list of transactions waiting to be folded — the fold builds it. As each proposed transaction arrives in the total order, the **Transaction Executor** applies it to the state of the system: balances, and the three homes of unit and position state (Chapter 6, *State: The Three Homes*). It serialises, validates, and applies one transaction at a time, $(tx, \text{Ledger}) \to \text{Ledger}$, and the immutable log is precisely the list the fold has so far built — the fold's output, never its input. Nothing else writes. And because the log is one list, every other view is a further fold of it: a balance sums the moves that touch a wallet, a valuation folds the composition against a price vector (Chapter 7), a report folds it into a presentation (Chapter 12). One record, and many folds of it — never many records.

2.5 Two axes of time: what was known, and what is in force

A correction arrives late. IDX's official close for day 2 is 990.00, but the print reaches the boundary only on day 4 — after day 3 has already settled against the prints then on the record. The system now holds two honest questions with different answers. *As the book stood at the close of day 3*, day 2's variation margin was computed without the 990.00: that is what was reported, and no later arrival may rewrite it. *As the book stands once the correction is in*, day 2's margin is recomputed with the 990.00 in force and the tail from day 2 forward re-derived. Both answers are correct. They answer different questions.

Two coordinates, not one, therefore fix a historical view. The first is the *log prefix*: how far down the immutable log the reconstruction reads — what had been admitted by a chosen point.

Its axis is *door time*, the position the single writer assigns each event at admission (Chapter 4), and it is the axis of what was known. The second is the *knowledge horizon*: which of the recorded observations are treated as in force. Its axis is *execution time*, the time each event happened in the world, asserted by its source; it is the axis of what a recorded observation says about the world, apart from when it was recorded. A back-dated observation — day 2's close, arriving on day 4 — sits at execution-time index $k = 2$ on the second axis while it sits late on the first. The projection reads both:

```
project :: LogPrefix -> KnowledgeHorizon -> View
-- LogPrefix        : how far down the immutable log to read   (door time)
-- KnowledgeHorizon : which recorded observations are in force (execution time)
```

A third time, *monitor time* — when the Event Monitor observed the event at the boundary (Chapter 4) — is recorded on every event but indexes no view. It, the other two times, and the total order the fold obeys are fixed in Chapter 4; this section fixes only the two axes a historical view rides on.

Fix both coordinates and the view is a deterministic fold, computed the same by any party. Two facts follow, and they are the whole of bitemporality. First, the *as-known-at* view — the projection over a fixed door-time prefix — is never rewritten by a later correction: a correction is a new recorded observation admitted further down the log, so it lies beyond the fixed prefix and cannot reach into it. Second, the *as-of* view — the projection over the prefix that already includes the correction — recomputes bit-exactly from that prefix, because the fold reads only recorded observations, obeys execution-time order, and is deterministic. The late close does not edit day 2: it is recorded at its own execution-time index $k = 2$, the fold from that index forward is re-derived, and the earlier as-known-at view survives beside it. Theorem 14.5 states this over the pair of axes and proves both facts.

The constitution names both answers and delegates their indexing here. C-12.1 (v1.3) fixes the standing duty as computing both honest answers — what the book said then, and the numbers with a later correction in force — each a deterministic replay of the one log, and leaves *how the two views are indexed* to this specification. The log prefix and the knowledge horizon are that indexing: the prefix fixes what was known, the horizon which recorded observations are in force. The single-axis reading once parked as entry PARK-2 is discharged by the owner's §12 ruling (Chapter 17); the machinery this section states is the indexing the constitution now delegates.

2.6 Data, functions, and machines

Three kinds of thing appear in this picture, and they are never confused: data, functions, and machines. The data kinds are Event, Transaction, and Ledger. The functions are the arrows between the data. The machines are what evaluate the arrows — one machine per arrow, and they are the subject of Chapter 4. Typed:

```
watch    : (Record, Clock)       -> [Event]      -- the Event Monitor
contract : (Event, Ledger)       -> Transaction  -- the Events Executor
apply    : (Transaction, Ledger) -> Ledger       -- the Transaction Executor

step (ledger, e) = apply (contract (e, ledger), ledger)  -- map, then fold
final ledger     = fold step over E = [e]
```

The three arrows differ in what they read. `watch` reads the Record and the clock and yields events; `contract` and `apply` consume the Ledger — the folded state — and never the clock. One `step` is a map (fire the contract) immediately followed by a fold (apply the result), and the whole run is that step folded over $E = [e]$. Map, then fold: the phrase is the architecture, not a metaphor for it.

An observation is not outside this picture. It enters as a moveless `Transaction` through

`apply` — a transaction with an empty move list that folds the observation into a home (Chapter 8) — so the Record the `watch` arrow reads and the Ledger `contract` and `apply` consume are two views of one log, never two stores. The observations a `contract` reads are therefore home facts already folded into the Ledger, and the single writer of `apply` is the only writer of any observation.

2.7 The clock, and proposal versus authority

Two properties of the picture carry most of its weight.

The clock is the single input that is not on the record, and the Event Monitor is the single place it is consulted. The event the Monitor emits is itself recorded, so everything downstream of the emission is a function of the record alone — which is exactly why a late firing produces the identical transaction, and why, given the record and the clock, any party can recompute which events should have been emitted, and when (Chapter 4).

The map is proposed; the fold is authoritative. A contract's output is a proposal, not a fact: a transaction joins the list only when the Transaction Executor admits it. The smart contracts, the Event Monitor, and the Events Executor sit outside the trust boundary and may only propose; the one Transaction Executor writes. Either arrow may refuse — a contract may find nothing due; the door rejects a malformed transaction and writes nothing — and a refusal is a returned value, never a half-applied state. Because both the events and the transactions are on the record, economic correctness is checkable after the fact: re-run the map on the recorded event and compare the result against the recorded transaction (Chapter 15, *Testability and the Executable-Check Regime*). Verification lives with the events; authority lives with the transactions.

2.8 One instrument walks the pipeline

OT-1 (Cast, §2.1) walks the pipeline whole. Its standing terms are fixed above; this section adds only the walk across the three arrows. At registration its terms declared one watch: OMEGA's recorded official closes against the barrier.

Trigger (watch). OMEGA's official close of 79.00 is recorded. The declared condition is met, and the Event Monitor emits the touch event onto the record. No ledger state has changed: the Monitor emits events, and an event is an input to the map, not a write.

Contract (map). The Events Executor fires OT-1's contract, handing it the touch event and the current projection of the ledger. The contract reads the terms, the holder (W-ALPHA), and the unit's state (*live*), and returns one transaction:

	Kind	Content
1	unit-state change	OT-1: UnitStatus *live* → *triggered*
2	move	owned(USD) 1,000,000: W-BANK → W-ALPHA

Apply (fold). The Transaction Executor admits it — the move's paired legs leave conservation undisturbed, the cause-derived identifier names the touch event, and the writer is OT-1's own contract — and the fold advances: the ledger now shows the cash with W-ALPHA and the option *triggered*. Where OT-1 stands encumbered under a collateral agreement the payment leg routes differently, and Chapter 9 works that same knock start to finish; the pipeline is the one drawn here either way.

One list of events, one map, one fold: the option's whole knock is a single event carried across the three arrows, and nothing off the record ever touches it.

2.9 Not event sourcing

The Ledger is event-driven because finance is: trades execute, prices print, coupons mature, and barriers are touched whether or not the software cares, so events are recorded because reality presents them, not because a pattern was chosen. The difference is where authority sits. In event sourcing the event stream is the canonical history, state is whatever the current code derives from it, and reinterpreting history through new code is a feature. Here the admitted transaction log alone is the canonical history, and two properties set it apart. First, one global order, not a set of per-aggregate streams stitched back together: the hash chain records arrivals in door order and the fold obeys one total order in execution time (Chapter 4), both global over the one log, so every view is a projection of it and two projections cannot disagree. Second, replay re-reads transactions and never re-executes contracts (Chapter 4), so history cannot be silently reinterpreted. Events are inputs to the map, never the accounting record: contracts transform them into proposed transactions, and the Transaction Executor admits or refuses. Correcting the record is neither reinterpretation nor an edit, and the two axes of Section 2.5 say why: a wrong interpretation is repaired forward by an authorised compensating transaction in the open (Chapter 15), and a corrected observation — a close that arrives late or is later restated — is a new recorded observation at its own execution-time index, admitted further down the log rather than written over the old one. The as-known-at view over any earlier door-time prefix is therefore preserved, and the as-of view recomputes from the prefix that carries the correction; neither overwrites the other. The events stay on the record for one purpose — so the map can be re-run and verified.

3 The Objects

Governed by C-4.1–4.11 (§16.5).

At the highest level a ledger is three things and a record: units — what can be held; wallets — where it is held; balances — how much of each unit each wallet holds; and the immutable log that records every change to them. Each object is defined here once; nothing later in the specification introduces another. The chapter ends where a unit's terms begin: with the product graph, the declared structure from which a unit's watches, its state schema, and its contract's actions are all derivable.

3.1 Units

A **unit** is anything that can be held or owed inside the ledger: a currency (USD in cents), an equity security identified by its ISIN, a bond, a listed or OTC derivative, a securities loan, a managed-account mandate, a margin obligation under a collateral agreement, or any other contractual instrument. Units are first-class and extensible: a new instrument type is a new element of the universe of units, never a modification of the machinery. Because every unit — asset, liability, or contractual obligation — is represented uniformly, one move machinery and one conservation discipline cover them all, without special cases.

Units divide into **valued** and **non-valued**. A unit is valued exactly when it represents an economic claim whose value exists independently of the governance or continued operation of the instrument that created it: a stock, a bond, a client's redemption claim on a managed account, a total return swap. A unit is non-valued when it is the governance itself: a mandate's rulebook, a portfolio-swap reset mechanism, a quantitative strategy's governance shell. The price of a non-valued unit is undefined — not zero — because pricing the governance structure beside the claim it governs would count the same economic reality twice. Valuation (Chapter 7) sums over the valued units only.

Every unit carries three things: its **terms** — the contractual text, versioned and append-only; its **state** — where it stands in its lifecycle; and its **smart contracts** — the executable form of

those terms. From the terms follow its **watches**: the dates, observations, and conditions the Event Monitor must observe on its behalf. Declaring them is the unit's duty — at registration, and again whenever a later transaction creates a new obligation — and the declaration is exhaustive: the Monitor does exactly what the units have asked of it, nothing more and nothing less. Section 3.8 shows that this duty is not a discipline to remember but a consequence of the form the terms take.

Episode — the future (T1): terms declare the watches. *Trigger.* The exchange lists FUT-IDX, and on day 0 W-ALPHA's purchase of one contract at 995.00 executes; both facts are captured and recorded at the boundary. *Contract.* Each recorded event is mapped to its transaction by the responsible contract (Chapter 2): the listing to a registration, the execution to a trade booking. *Transactions.* Two, in order:

1. *Registration* (moveless). ProductTerms FUT-IDX version 1: cash-settled future on index IDX, multiplier 10, USD, settled-to-market — the regime a declared term of the CCP agreement unit (Chapter 9). Unit-state change: watches declared — one per daily settlement-price publication, and one for the expiry.
2. *Trade* (day 0). One move: 1 FUT-IDX, owned, from the CCP's virtual wallet to W-ALPHA. One position-state change: W-ALPHA's traded price 995.00 recorded.

The door. Both transactions carry cause-derived identifiers, so a duplicated capture commits once; the move's paired legs make conservation automatic; and no cash moves — a settled-to-market future exchanges nothing at trade, the first variation-margin move being day 1's (Chapters 5 and 9). *Design point. The watches follow from the terms and are declared at registration: the future's whole life — every print, the expiry — is asked for before the first event arrives.*

Episode — the variance swap (T5): the declaration is exhaustive. *Trigger.* VS-1 is executed and its confirmation captured at the boundary: a one-year OTC variance swap on IDX, variance strike 400 variance points, variance notional USD 1,000 per point, no cap, 252 scheduled fixings against the recorded IDX official closes — the same observation source FUT-IDX settles against. *Contract.* The confirmation maps to the registration transaction that installs the unit. *Transactions.* One, moveless:

1. ProductTerms VS-1 version 1: the terms above, the fixing schedule carried as declared data.
2. Unit-state change: 252 fixing watches declared — one per scheduled fixing date, each a watch on the recorded IDX official close; the 252nd is the final fixing, on whose firing settlement is computed (Chapter 7).

The door. A moveless transaction is admitted like any other: idempotent under its cause-derived identifier; nothing to conserve, everything to record. *Design point. Exhaustive means exhaustive: all 252 conditions the swap's life requires are on the record before the first fixing prints, and no watch is discovered mid-life.*

Episode — the TRS (T6): a non-valued unit beside an ordinary one. *Trigger.* Two registrations are captured at the boundary: the strategy S-QIS, and the swap TRS-1 written on the strategy's wallet. *Contract.* Each confirmation maps to its registration transaction. *Transactions.* Two, both moveless:

1. ProductTerms S-QIS version 1: the strategy's rulebook, the complete terms of a *non-valued* unit whose contract manages wallet W-QIS in virtual ledger V-1 (Chapter 10). S-QIS has no price: undefined, not zero.
2. ProductTerms TRS-1 version 1: notional USD 10,000,000; reference leg the NAV of W-QIS; financing leg fixed 4% per annum, quarterly. Unit-state change: watches declared — each quarterly reset date, at which the contract reads the stamped NAV observation then on the record.

The door. Consistency of reference: TRS-1's terms name a wallet and an observation source that exist on the record; the registrations commit once under their cause-derived identifiers.

Design point. The strategy's governance shell is non-valued because pricing it beside the wallet it governs would count the same economics twice; the TRS that references it is an ordinary valued unit of the true ledger — referencing a virtual ledger makes nothing special.

3.2 Wallets

A **wallet** is a named logical partition of position space — a container, and deliberately nothing more. Its state at any moment is, for each unit it holds, a signed quantity: its **balance**. A wallet names two parties: a **manager**, authorised to perform transactions on it, and a **beneficial owner**, legally accountable for the book. Both are declared at creation, and the distinction is load-bearing: authorisation is checked against the manager, accountability and reporting attach to the beneficial owner. Wallets carry no smart contract and no logic of their own; where a wallet appears to need behaviour — the rules of a managed account — that behaviour is a unit (the mandate), and the wallet stays a container. W-QIS above is exactly this shape: the rulebook lives in the non-valued unit S-QIS, and the wallet only holds.

3.3 The Atomic Move

An **atomic move** is the sole operation that changes a balance: it transfers a positive quantity of exactly one unit from a source wallet to a destination wallet in a single indivisible step — the same quantity debited at the source and credited at the destination. Because a move only transfers, it can never create or destroy quantity; issuance and retirement are themselves paired-leg moves against the issuer's wallet (Chapter 14 states the conservation law this yields and proves it by induction on the log).

Quantities are exact integers in minor units: a USD amount is an integer count of cents, a holding an integer count of shares or contracts, a recorded statistic an integer at its declared scale. No quantity in the ledger is a floating-point number. Where an event divides unevenly — a per-share figure across a holding it does not divide, a pro-rata allocation with an odd cent — the rounding convention is part of the declared terms, and the remainder is booked explicitly as its own move: the convention says where the odd cent goes, and the odd cent's journey is on the record. Nothing is ever silently dropped, which is what makes conservation exact rather than approximate.

3.4 The Transaction

A **transaction** is an atomic, all-or-nothing bundle of four kinds of change: moves, unit-state changes, position-state changes, and terms changes. The four align one-for-one with the four components of the ledger's state: balances, and the three homes of Chapter 6 — UnitStatus, PositionState, ProductTerms. A transaction applies in full or not at all, and some transactions carry no moves: the registration of a new unit and the recording of a barrier breach are complete transactions with empty move lists. The episodes above have already shown three of the four kinds; the split shows the fourth.

Episode — the split (T4): a terms change is a transaction component. *Trigger.* 2026-06-01: ACME's 2-for-1 split becomes effective. The recorded split notice had declared the effective-date watch; the condition is met and the Event Monitor emits the event. *Contract.* ACME's contract fires with the event and the current projection of the ledger: W-ALPHA holds 10,000 shares. *Transactions.* One, atomic:

1. Move: 10,000 ACME, owned, from the issuer's virtual wallet to W-ALPHA — the holding becomes 20,000 by a paired-leg move, one such move per holder, never by an edit.
2. Terms change: ProductTerms ACME version 2, carrying the 2-for-1 event. How the recorded price frame 100.00 reads as 50.00 after the event — and the declared per-share dividend figure 1.50 as 0.75 — is the market data operator's work, Chapter 8.

The door. Atomicity: the move and the new terms version apply in full or not at all — no instant exists at which 20,000 shares stand beside pre-split terms (Chapter 14's consistency of reference begins here). Conservation: the issuer wallet's balance falls by 10,000 as W-ALPHA's rises. *Design point. A terms change is one of the four components of a transaction, atomic with the moves it accompanies; terms are versioned and append-only, never edited.*

3.5 The Immutable Log

The **immutable log** is the ordered, append-only, hash-chained record of every admitted transaction — hash-chained so that any alteration of history is detectable. It is the sole canonical source of truth. Three words are used precisely throughout this specification. The **log** is the recorded sequence itself. The **ledger** is the state obtained by folding the log: balances and the three homes. The **record** is everything the log carries. Every observation whose reproduction the framework guarantees enters the record the same way every fact does — as a moveless **observation-recording transaction**, a transaction with an empty move list that folds the observation into a home (Chapter 6), exactly as the barrier-breach recording above. There is no second store beside the log: the record *is* the log, and the single-writer discipline reaches every observation, not only every move. "On the record" means present there, and a **projection** is any view computed from the record — which is why, in Chapter 2's typed picture, the watch arrow reads the Record while contract and apply consume the Ledger, the observations a contract reads being home facts already folded into it.

3.6 Closure by Virtual Wallets

Every counterparty outside the system is represented by a **virtual wallet** inside it: the CCP a future faces, the issuer of a stock, the writer of an option each hold a wallet, and every move has both legs on the book. The system is therefore closed, and conservation holds by construction, not by check. The split episode shows the closure working: the 10,000 new shares credited to W-ALPHA are debited from the issuer's virtual wallet in the same move — quantity appears nowhere from nothing, and the sum over all wallets, virtual ones included, stays at zero.

3.7 The Coordinate Vector

When a balance must carry more information than a single number, it generalises to a signed vector — but the coordinate alone is too coarse, and one concrete failure fixes the shape. Suppose wallet W posts USD 100 as collateral under agreement G_1 and, in the same unit, receives USD 100 as collateral under a *different* agreement G_2. Both are real encumbrances: USD 100 is pledged away under G_1, and USD 100 sits held-but-not-owned under G_2. A single signed *posted* axis records the first as $+100$ and the second as -100, and their sum is 0 — so the balance reads as if nothing were encumbered, and Chapter 12's encumbrance projection reports zero for a fully encumbered wallet. The net across two agreements is not the encumbrance; it hides it.

So the coordinate is not the whole index. **The balance is a signed vector indexed by (coordinate, agreement)**: write $\mathrm{bal}_{c,G}(w, u)$ for the signed quantity of unit u that wallet w holds on coordinate c under agreement G. The constitution no longer legislates the schema; it fixes what any schema must satisfy — ownership first and alone value-bearing, every further coordinate mass-only under a named agreement, and three admission tests (C-4.10) — and delegates the schema itself to this specification, on the specialists' advice. The schema declared here is three coordinates — **owned, lent, posted** — with the rays *lent out* and *borrowed* the two signs of lent, *posted as collateral* and *received as collateral* the two signs of posted. Ownership is unqualified — owned is the coordinate the move machinery acts on directly and carries no agreement (Chapter 9) — so G ranges over agreements only on the non-owned coordinates,

where every quantity sits under exactly one collateral or margin agreement and reaches the plane only through a reference to it. In the failure above, $\mathrm{bal}_{\mathrm{posted},G_1}(W, \mathrm{USD}) = +100$ and $\mathrm{bal}_{\mathrm{posted},G_2}(W, \mathrm{USD}) = -100$ are two distinct coordinates of the vector, and neither vanishes. The first test admits a coordinate only where a distinct real-world action can change it independently of ownership: lending changes possession without changing ownership, pledging encumbers without transferring, and nothing else in scope passes the test, so there is no fourth coordinate. The second test demands a grain fine enough that two distinct real-world facts can never net to nothing, and the G_1/G_2 failure above is its witness: two postings under two agreements are two facts, valued, discharged, defaulted, and returned separately, so the agreement is part of what the coordinate *is*, not a label on it — an encumbrance is never hidden by an offsetting one.

A move writes exactly one (c, G) coordinate of one unit, paired legs as always, so conservation holds per (unit, coordinate, agreement) — the finest declared grain, and the primitive (Chapter 14, Theorem 14.1); that is the third test, discharged as a theorem. The three-coordinate vector **(owned, lent, posted)** is the **aggregate projection** of this finer object, summing over agreements:

$$\mathrm{bal}_c(w, u) \;=\; \sum_G \mathrm{bal}_{c,G}(w, u).$$

It is a projection, not the primitive: it is exactly the sum the failure above shows loses information — a fully encumbered wallet and a wholly unencumbered one share the image 0, which is what the second admission test exists to forbid at the primitive grain. The (coordinate, agreement) schema therefore stands as this specification's declared schema, compliant with all three tests of C-4.10: the distinct-action test and the grain test demonstrated above, and conservation at the finest declared grain held as Theorem 14.1.

Only the owned coordinate carries economic value and drives profit and loss. The remaining coordinates carry mass, never value: they record possession and encumbrance under the named agreement that indexes them, no quantity reaches them without reference to the agreement unit that governs it, and a lifecycle event extinguishes value, never mass — a unit leaves the ledger only from the zero vector, every (c, G) coordinate zero. Anything computable from the coordinates — availability, encumbrance, possession — is a projection, computed when needed and never stored; and the encumbrance projection reads the per-agreement coordinates gross, never their aggregate net (Chapter 12), for the reason the failure above makes plain.

This section fixes the definition and nothing else. Which coordinate a given inflow writes, the agreement semantics of the non-owned planes, the guards checked at the door, and the regime key that decides among them are Chapter 9's.

3.8 Term Sheets as Graphs

A unit's terms are not free text with a contract bolted on; they have a shape, and the shape is a graph. Take the one-touch OT-1 of the thread map: payout USD 1,000,000, its barrier breached when OMEGA's official close is at or below 80.00 — a discretely-monitored (closing-price) one-touch. Its life has three states — live, triggered, expired. From live, exactly two things can happen: a recorded OMEGA official close at or below 80.00 (the unit becomes triggered and the payout falls due), or the expiry date arrives untouched (the unit becomes expired, worthless). From triggered and from expired, nothing further happens. Drawn, that is a graph: three nodes, two edges leaving one of them.

Definition 3.1 (Product graph)**.** *The terms of a unit define its* product graph*: the nodes are the unit's lifecycle states, a set closed and known at registration; each node carries a payload — the sufficiency facts of constitution §7, held in the three homes of Chapter 6 (the variance swap's accrued fixing statistic, the future's last settlement applied); each edge carries a* guard *— an event kind the Event Monitor can watch, together with a predicate — and an* action *— the transaction template the traversal emits.*

The structure is declared data: nodes, edges, guards, and payload schemas are recorded with the terms and readable without running anything. The arithmetic inside an action is not data and does not pretend to be: it is a small, named, versioned pure function — the variance swap's realised-variance accumulator, the future's margin difference — referenced from the edge that uses it. Declared structure carries behaviour, per the constitution's Clarity commitment (§2); named pure functions carry computation; and nothing is hidden in code the graph does not point to.

The graph is identical with the watch discipline of this chapter. The watch list of a unit is exactly the out-edge set of its current node, so declaring the graph declares the watches — the exhaustiveness demanded of units above is not a rule to remember but a projection of the graph. Arming is edge registration; firing is traversal; and a traversal expires its sibling edges: when OT-1's touch edge fires, the expiry edge dies with the live node that carried it. The declared → armed → fired/expired lifecycle of Chapter 4's handshake is a graph fact, not a bookkeeping convention. The variance swap reads the same way: 252 fixing edges, and the 252 declared watches of its episode are their guards, exhaustive at registration because the graph is closed at registration. A scheduled watch is dated at or after the unit's inception: a node's out-edge cannot fire before the node exists, so no traversal the graph schedules lands before the unit that carries it *(product-graph well-formedness).* The declared *arms a further watch* relation — an edge whose firing registers a watch on another unit — is moreover required acyclic: a product whose declared arming graph contains a cycle is refused at registration, a decidable static check on the declared edges. This bounds the cross-unit arming cascade and discharges hypothesis H-WF of the firing closure (Chapter 4).

One class of edge the three-node picture omitted, and constitution §7 requires it. OT-1's barrier names OMEGA's official close, so OT-1's terms *reference another unit.* Split OMEGA two-for-one and every price OMEGA quotes halves: a barrier of 80.00 struck against pre-split OMEGA now stands against a stock trading near half its former level, and the next recorded close — near 47.50 — would read $47.50 \leq 80.00$ and knock, paying USD 1,000,000 on a stock that never fell. The barrier is a declared term, not market data, so no market data operator adjusts it (Chapter 8); unless the graph carries an edge that adjusts the term, the split cannot be recorded and the knock is a phantom. Hence the rule: *every unit whose terms reference another unit declares, on each of its lifecycle nodes, a* ***corporate-action out-edge*** *— its guard a recorded corporate-action event on a referenced underlying, its action the appending of a new ProductTerms version that applies the declared term-adjustment operator (Chapter 8) to every referencing term.* The edge is a self-loop: it lands the unit on the node it left — `live` stays `live` — and changes nothing but the terms version. For OT-1 the appended version reads barrier 40.00 where the old read 80.00, so the post-split close $47.50 > 40.00$ no longer knocks. Because the edge is an out-edge, it is a watch by Invariant 3.2(i), so the adjustment is armed before any split arrives; and because it fires as a transaction like any other, it is exactly constitution §7's guarantee that "when the underlying stock splits, the contract's strike and multiplier are adjusted by appending a new terms version." A unit whose terms reference nothing — a plain equity — declares no such edge; its own split moves its share count by a move (Chapter 8), a different mechanism entirely.

Because the node set is closed at registration, each state can be a distinct type and each edge a function between those types — and the edges are the only constructors those types have:

```
data Live      = Live      { accrued  :: Payload }
data Triggered = Triggered { touchRef :: EventId }
data Expired   = Expired

touch  :: Fixing -> Live -> Triggered   -- the only constructor of Triggered
expire :: Expiry -> Live -> Expired
adjust :: CorpAction -> Live -> Live    -- corporate-action self-loop: the node is
                                        -- unchanged; a new ProductTerms version
```

```
    is the change
-- Triggered -> Live: no such function exists, so no program un-knocks a
-- note; a case analysis that forgets a node does not compile.
```

An illegal transition is not refused; it is unrepresentable — there is no program that writes it. Totality is compiler-checked: exhaustive case analysis over a closed node set is exactly what these types force, so a contract that forgets the expired state is a compile error, not a production incident.

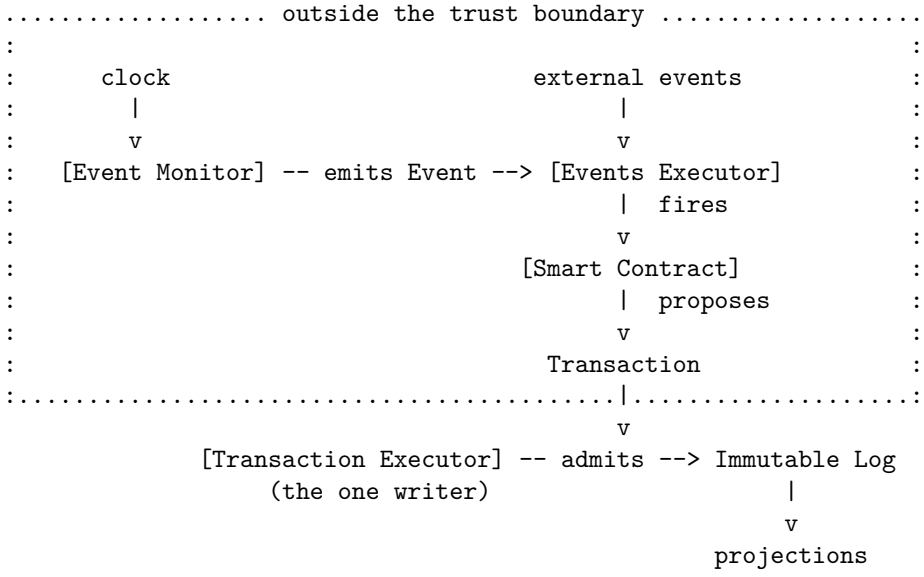
Invariant 3.2 (Graph consistency)**.** *For every unit there exists a product graph from which its watches, its state schema, and its contract's actions are all derivable, and their mutual consistency holds: (i) at every commit, the unit's watch list equals the out-edge set of its current node; (ii) every fired event matches exactly one out-edge and lands the unit on that edge's target node; (iii) every transaction the contract emits equals the edge's declared action applied to the recorded inputs; (iv) a traversal expires exactly the declared sibling edges, and no others.*

The four properties are executable, not defended in prose: their executable forms join the invariant catalogue of Chapter 14 and the test regime of Chapter 15, where they are property-tested over generated products, events, and histories.

4 The Machines

Governed by C-5.1–5.8, C-2.7, C-4.12, C-12.6 (§16.5).

The framework has three machines, one for each arrow of the typed picture of Chapter 2 (*The Picture: Map, Then Fold*): the Event Monitor evaluates `watch`, the Events Executor evaluates `contract`, and the Transaction Executor evaluates `apply`. All three are generic machinery. No machine holds a line of product knowledge: what a dividend, a touch, or an expiry *means* lives in the smart contracts (Chapter 5); the machines only notice, deliver, and admit. And only the last of the three ever writes. The Event Monitor emits events; the Events Executor collects proposals; the Transaction Executor alone changes the state of the ledger. Three roles, one writer. In one picture — machines in brackets, data on the arrows, the dotted line the trust boundary:



4.1 The Event Monitor

The Event Monitor owns time and attention. It holds every declared watch — the dates, observations, and conditions each unit's terms require watching (Chapter 3) — and evaluates

them against the clock and the recorded observations, emitting an event onto the record the moment a condition is met. It holds no private list: the watches it evaluates are on the record, declared by the units, so the Monitor's workload is itself a projection. And it emits events, never transactions: nothing the Monitor does can place so much as one move before the door.

The clock is the single input to the whole system that is not on the record, and the Event Monitor is the single place it is consulted. The event the Monitor emits *is* recorded, so everything downstream of the emission is a function of the record alone. Two consequences follow, and both are load-bearing. A late firing produces the identical transaction: the contract reads the recorded event and the recorded state, never the wall clock, so delay costs timeliness and nothing else. And emission itself is verifiable: because watches are declared data and the Monitor is deterministic, any party holding the record and the clock recomputes which events should have been emitted, and when — the Monitor is auditable against its own inputs.

The relationship between the units and the Monitor is a recorded handshake. Each declared watch is acknowledged by the Monitor, and the acknowledgement — an event like any other, flowing through the same pipeline — is written into the unit's state. A watch is thus visibly *declared*, then *armed*, then *fired* or *expired*, and each step is on the record. A declared watch that is never armed is therefore a visible gap, never a silent one: it stands in the unit's state as declared-but-unarmed, an overdue item any reader can query. The full chain can be walked in either direction — register a unit, the conditions it declares, the acknowledgements, the events, the transactions, the state updates — which is the auditability commitment (§2) applied to attention itself.

The Monitor can affect timeliness and liveness; it cannot affect the correctness of ledger state, because it only emits events, and an event changes nothing until a contract's proposal passes the door.

4.2 The Events Executor

The Events Executor owns delivery. It does two things and nothing else. First, it captures the events arriving from outside — trades executed in the market, corporate-action notices, fixings and other observations — and *proposes* their recording: an observation crosses as a moveless observation-recording transaction (Chapter 8), which the Transaction Executor admits or refuses like any other. Second, for each event, arrived or emitted, it fires the contracts the registry's routing names for that event's kind (Chapter 8), handing each the event together with the current projection of the ledger, and collects the proposed transaction. The Executor never decides routing; it reads it from the registry, and *responsible* is a defined word — the contract or contracts the registration names. Many contracts, one Events Executor: the contracts carry all the product knowledge; the Events Executor carries none.

Capture of every arrival is unconditional — the guarantee runs from arrival, the perimeter being TA-ARRIVAL's matter (Chapter 16). The Executor stamps every arrival with its **capture envelope**, carried even when the payload is malformed, and proposes its recording *before* payload validation. The envelope records the delivering source and three times, and no other mechanism carries them.

Execution time is when the event happened in the world — the trade's execution timestamp, the exercise notice's exercise time, the print's observation time — asserted by the source and read from the payload; for an event the Event Monitor emits from the record rather than receives, it is the record-derived world fact: the record date of a record-date firing, the scheduled instant of an expiry, the breaching close's observation time of a barrier touch. It is the time a court would enforce, contestable only in the world and corrected only by a later event, never edited at the door (TA-EXECUTION-TIME, Chapter 16); it alone orders the fold.

Monitor time is when the Event Monitor observed the arrival at the boundary — the Monitor's single clock read above — recorded as provenance. It orders nothing and gates nothing; with execution time it splits an event's lateness into the world's segment, execution to monitor,

and ours, monitor to door. It is *null* exactly for an emitted event, born inside the record with no arrival to observe — a watch firing or its acknowledgement — whose world segment is therefore zero.

Door time is when the single writer admits the event, assigned at the door and strictly monotonic (§4.3); it subsumes the arrival's log position, and says when the book learned, never what is true.

An arrival whose event kind is registered then routes as above. One whose kind is undeclared, whose reference the record cannot resolve, or whose payload fails validation — provenance incomplete included — is still recorded, under the envelope it did carry, as a **quarantined event**: the record-then-refuse-processing discipline of failure mode W4 (Chapter 8). Never routed, never lost; "carrying its provenance" means at least this envelope — source, execution time present or record-derived, monitor time present or null, door time assigned at admission — and the payload's own provenance where it has one.

A trigger carries timing, not data: the firing says *when*, and everything the event means is read from the record. The Events Executor runs the contracts; it never writes the ledger — not even an observation — and its output is only proposals. A proposal is not a fact — it becomes one only if the one Transaction Executor admits it.

4.3 The Transaction Executor

The Transaction Executor owns the record. It is the gatekeeper of the ledger door: the single component through which every transaction must pass, and the only component permitted to change the ledger's state. On each proposal it validates structure. Conservation is automatic — a move is paired-leg by construction, so the per-(unit, coordinate) sums cannot be disturbed (Chapter 3); authorisation, idempotence, consistency of reference, and writer discipline are enforced as checks at the door. If the proposal is admissible, the Executor appends it to the immutable log; if not, it refuses, and the refusal is a returned value, never a half-applied state. It creates nothing and runs nothing. There is no other way to change the ledger's state.

Idempotence is the door's own defence, owed to no one upstream, and its key is the transaction's own identifier. Start with the case the identifier must get right. One corporate action — a single cash dividend declared on stock ACME — is one cause, yet it touches every unit that references ACME, and each is adjusted by its own transaction. The Monitor emits one event; the contract fires once per affected unit and proposes one transaction per unit. These are *different* transactions of the *same* cause, and every one of them must commit. An identifier that keyed on the cause alone would collapse the whole set to one and silently drop every unit but the first. So the cause alone cannot be the key.

The derivation rule. Every proposed transaction carries the identifier

$$txid \;=\; H(causeEventId,\; contractId,\; unitId,\; seq),$$

where *causeEventId* is the recorded event that fired the contract, *contractId* and *unitId* name the contract and the unit the transaction acts on, *seq* distinguishes the several legs one firing may propose on a single unit, and H is a fixed collision-resistant hash. The door keys idempotence on *txid*: a proposal whose *txid* already stands on the log is committed zero further times; a proposal whose *txid* is new is admitted on its merits. Two properties make this rule exactly right, and each is proved, not asserted.

Constant under retry. Re-delivering the same firing recomputes the identical tuple. All four components are read from the record: *causeEventId* is the recorded event, *contractId* and *unitId* are fixed by which contract fired on which unit, and *seq* is assigned by the contract as a pure function of that recorded input (Chapter 5). A retry, a duplicate, or a late re-firing therefore presents the same four components, so H returns the same *txid*; the door finds it already on the log and commits it once. Delay and duplication cost timeliness and nothing else.

Injective over a cascade. Fix one cause, so *causeEventId* is held constant across the whole cascade. The remaining three components (*contractId*, *unitId*, *seq*) identify a leg uniquely: two legs of the cascade differ in the unit they act on, or — for two legs on one unit — in *seq*; no two distinct legs agree on all three. So the map (*contractId*, *unitId*, *seq*) $\mapsto$ *txid* is injective on the cascade's legs, *provided H* is injective on the tuple domain. That is the standing assumption on H, stated rather than left implicit: over the finite domain of well-formed cause tuples H admits no collision — exactly the guarantee a collision-resistant hash is chosen to give. Under it, a cascade of n legs carries n distinct identifiers, the door sees n new *txid*s, and all n commit. The two properties are jointly exact: a retry collapses because its tuple is unchanged, and a genuine second leg commits because its tuple differs.

```
txid :: EventId -> ContractId -> UnitId -> Seq -> Txid  -- txid = H(cause,
    contract, unit, seq)
-- retry:   same four arguments        => same Txid     => committed once (
    constant under retry)
-- cascade: one cause, distinct (contract, unit, seq)   => distinct Txids  => all
    n commit (injective)
```

The single-writer discipline is a correctness requirement, not an implementation preference. The Transaction Executor assigns each admitted event its *door time* and presents to the log a strictly monotonic sequence of committed transactions in door order — the hash chain, the incorruptible record of arrival (Chapter 14). Meaning rides a second order, the *total order* of §4.5, which every projection folds — so two projections of one record cannot disagree — and which door time makes total by resolving two events of one execution instant by admission sequence. Where two simultaneous events on one instrument propose transactions that do not commute and whose terms declare no precedence, the Executor refuses admission rather than guess: an execution-simultaneous pair whose result depends on their order is one no door-time tiebreak may silently decide, and defaults fail closed (§2). This refusal is constitutional, not a design preference: C-2.7 forbids reordering anything unless harmlessness is proved, and two non-commuting transactions of one instant with no declared precedence are exactly the case where it cannot be; the door-time tiebreak orders only pairs harmless to reorder or grounded in a declared precedence. The refusal is visible, and resolution — a declared precedence, a corrected proposal — arrives through the same door.

4.4 One event, three machines, one write

The pipeline walk of §2 ran OT-1 (Cast, §2.1) across the three arrows; here the same knock crosses the three machines, and this section adds what that walk did not — the door's checks and what happens when the emission fires twice. OMEGA's recorded close of 79.00 meets the declared condition; the Monitor emits the touch, the Events Executor fires OT-1's contract on the current projection, and the contract returns one transaction — OT-1's UnitStatus *live*→*triggered* and the USD 1,000,000 move W-BANK→W-ALPHA (§2). Where the unit stands encumbered the payment leg routes through the agreement's conditional obligation (Chapter 9); the machines are identical in both tellings.

The door. The Transaction Executor checks the paired legs (the per-(unit, coordinate) sums are undisturbed), the cause-derived identifier (the touch event, not yet processed), and writer discipline (OT-1's state is written by OT-1's contract), and admits. One append; the fold advances; the ledger now shows the cash with W-ALPHA and the option *triggered*.

Duplicate emission. Emission is at-least-once (Chapter 16), so the Monitor may emit the touch again. The Events Executor fires the contract again; the contract finds UnitStatus already *triggered* and proposes nothing. Had a duplicate proposal been produced anyway, its identifier recomputes to a *txid* already on the log and the door commits it zero further times. Two independent defences, either one sufficient.

One event, three machines, one write — and the recorded state is exactly what makes a duplicate harmless.

4.5 The total order and the refold

The fold advances in one order, and it is not the order of arrival. The **total order** on admitted events is lexicographic on the triple (execution time, door time, event hash):

$$e_1 < e_2 \iff (\text{exec}\, e_1,\ \text{door}\, e_1,\ \text{hash}\, e_1) <_{\text{lex}} (\text{exec}\, e_2,\ \text{door}\, e_2,\ \text{hash}\, e_2).$$

It is deterministic, total, and computable by any party from the record alone — every component is on the record — and it is the fold order: map-then-fold advances along it, and it is the meaning of the record (Chapter 2). Duplicates never enter it: an arrival identical under the cause-derived identifier is absorbed once, before ordering, so the relation ranges over distinct events and a duplicate never takes a position.

The relation is defined over the events of a *single lineage* — the authoritative one, of which Invariant 14.7 guarantees exactly one at each position. Absorption, and hence the order, is intra-lineage: an abandoned-lineage event and its rebuilt twin on a new lineage share execution time *and* hash (both lineage-invariant), are separated only by the reissued door time, and are never both carried in one fold nor ordered against each other. Within a lineage, distinct events have distinct content and hence distinct hashes (the collision-resistance below), so key injectivity, trichotomy, and antisymmetry hold as proved.

Absorption is by identifier alone, whatever execution time the arrival asserts: the cause-derived identifier fixes the event, and its first admitted execution time is the recorded fact. A source that later asserts a different execution time does not correct by re-arrival — a genuine execution-time contest is a distinct correction event that names the wrong one (C-12.4, TA-EXECUTION-TIME), carrying its own cause-derived identifier, so it is never absorbed but re-inserted at its true execution position and refolds the tail. Absorption therefore drops a duplicate, never a contest. The duplicate check ranges over the full retained log, superseded and voided firings included (retain-not-delete, C-12.5): a duplicate of a voided firing absorbs against the retained identifier rather than resurrecting the firing — had the void deleted it, the duplicate would be admitted as new.

Each level earns its place. *Execution time* carries the meaning: buy-then-record-date and record-date-then-buy differ in it, and the fold must honour it. *Door time*, strictly monotonic per lineage, makes the order total for two events of one execution instant, resolving them by admission sequence; within a lineage (exec, door) is already strict. *The event hash* does two things door time cannot: it keeps the order total when door-time monotonicity fails — that monotonicity is a property-tested obligation, not a typed guarantee (Chapter 15) — and, because execution time and the hash are lineage-invariant where door time is not, a party can recompute the *new* lineage's order from execution time and hash alone, without trusting the reissued door times (Chapter 14). Across lineages the hash does not discriminate — an abandoned event and its rebuilt twin share it — which is why a fork's two lineages are never ordered against each other. Under door-time monotonicity the hash is never reached for distinct events: redundant in practice, load-bearing in principle. Totality is proved under one named assumption — the hash admits no collision on the finite domain of well-formed events, the same collision-resistance the cause-derived identifier rests on (§4.3).

Two orderings therefore coexist over the one log, and conflating them is an error. The **hash chain** records admission in door order; append-only, position-dependent, never rewritten (Chapter 14) — the incorruptible record of *what arrived, and when we learned it.* The **total order** is the execution-time order the fold obeys — *what the events mean.* One record, two orders of it: the arrival record and the meaning. This is the bitemporal split of Section 2.5 lifted from a unit's fixings to the global fold.

The reordering step. A *late arrival* is an admitted event whose execution time precedes events already folded. Its handling is a fixed procedure. Let e be newly admitted, $x = \text{exec}\, e$, and X the execution time of the fold's head.

(a) **Detection.** At admission the Transaction Executor compares x with X. If $x \geq X$, e extends the fold at its head — the timely path, no refold. If $x < X$, e is a late arrival. Detection compares two recorded execution times and reads no clock; the $x \geq X$ fast path's classification correctness rests on door-time strict monotonicity (`prop_doorTimeMonotonic`), and an absorbed duplicate is never detected here.

(b) **Insertion.** e takes its total-order position by $(x, \text{door}\, e, \text{hash}\, e)$; the events strictly after it are the **tail**. Nothing moves in the hash chain — e is appended at its head in door order — only its position in the fold order is interior.

(c) **Refold.** Every tail event refolds, its transaction recomputed against the ledger the reordered prefix now builds. The fold is a pure function of the ordered log (Chapter 2), so re-running map-then-fold from the insertion point forward is deterministic and consults only the record. The superseded transactions are not erased: they are the as-known-at history, preserved beside the recomputed tail (Section 2.5). The refold *appends* the recomputed tail further down the log and edits no byte: a re-fold, never a rewrite, append-only across the correction as within it (Chapter 14, C-12.5). The refold also re-derives every *Monitor emission* the reordered prefix implies — the arming acknowledgement and the firing alike, a watch declaration being contract-written and so already recomputed by the fold; a watch the new order re-arms at a relocated edge has its arming acknowledgement synthesised before its firing, the declared, armed, fired handshake intact. An emission the new order newly implies — a watch it newly declares-and-arms, or a data-predicate or scheduled watch it newly satisfies — is **synthesised** at its record-derived execution position: an emitted event carrying its watch's registered event kind (declared with a router when the watch armed, C-4.12, Chapter 8), routable by construction and never quarantined; its monitor time null and its door time the canonical top value $\top$ — the null of an event that never crossed the arrival door, record-derived and identical for every party and every rerun — sorting after every real door time of its execution instant, so a synthesised firing never precedes the observation that triggers it; its cause-derived identifier keying on the correction *and the watch it fires* — $H(\text{correction}, \text{watch}, \text{occasion})$ — the *occasion*, the record position (observation point or scheduled date) at which the watch's edge is evaluated, distinct edges giving distinct occasions — the watch discriminating distinct firings as $(\text{contract}, \text{unit}, \text{seq})$ discriminates a cascade's legs (§4.3) — so one correction newly satisfying several watches, even several on one unit, synthesises firings that stay injective, none absorbed as a duplicate. An emission the new order no longer implies — a superseded firing or arming — keeps its recorded event, its consequence voided (C-12.6). The refold is one forward pass from the insertion point: at each execution position it folds the event there, evaluates the armed predicates against the running state, and folds any newly synthesised emission in the same pass. A synthesised emission's execution time is the instant its condition became true — the tipping observation just folded, or a minted unit's scheduled date, never before its inception (Chapter 3, product-graph well-formedness) — so it never falls behind the cursor and no position is revisited; each event, arrival or synthesised emission, folds exactly once. This derivation reads the record, not the clock (Chapter 4): the ordered prefix determines its own firings.

(d) **Flagging.** A refold is never silent (C-12.6), and each flag is a recorded, queryable mark, not prose. The event e carries a *reordered* mark naming its execution time and insertion position; every state whose recomputed value differs from its as-known-at value carries a *restated* mark naming e; a state the refold leaves identical carries none. An auditor recovers which states moved, and why, from the record alone.

(e) **Explain.** The difference between what the book said and what it now says is published as a named profit-and-loss explain item, attributed three ways: to the reordering, to the causing event e, and to the segment of its lateness — the world's, execution to monitor, and ours, monitor to door.

(f) **Money.** Views recompute; money does not. Where the refold changes a quantity already settled, the delta stands as a visible open item and moves only as an authorised compensating transaction under C-12.4 (Chapter 15); nothing already remitted is clawed back by the fold. The end-to-end trace is §4.6.

Analysis. *Determinism.* Same log, same refolded state: the order is computed from the three-time envelope and step = apply ∘ contract is a pure function of the ledger (clock-confined, Chapter 4), so any party re-deriving the order and refolding lands on identical state. *Idempotence.* Refolding twice equals refolding once (`prop_refoldIdempotent`, Chapter 15) — the second pass finds e at its position, folds the same $\text{sort}_<$ (memo P4-iii), and re-derives the same closure: the closure is a fixpoint (Lemma 4.2), so no emission is added or suppressed, and a re-derived emission already present is absorbed under its cause-derived identifier (C-12.3), never duplicated. A duplicate late arrival, absorbed before ordering, occupies no position and triggers no refold.

Theorem 4.1 (Refold equals timely)**.** *Fix the external arrivals of the authoritative lineage, and let e be a late arrival among them. The ledger state after the reordering step — balances and the three homes — equals the state that folding those same external arrivals in execution order from the start would reach, each order's implied watch firings re-derived by step (c).*

Why it holds. The firing-derivation is a *fixpoint* of interleaved fold-and-derive: a synthesised emission takes a past execution position and can imply further emissions. Lemma 4.2 makes this fixpoint unique and record-derived, so the refold and the timely fold close the *same* arrivals to the *same* fixpoint; both are then fold step over one $\text{sort}_<$ sequence, and fold step is deterministic: equal input, equal fold state. Only provenance and the timing of external effects differ.

Lemma 4.2 (Firing closure)**.** *Let A be the external arrivals (finite). For finite $S \supseteq A$ let $D(S)$ be the Monitor emissions implied by folding $\text{sort}_<(S)$: at each total-order position p, an emission whose watch's edge-predicate the fold strictly below p newly satisfies. A* closure *of A is a finite $F \supseteq A$ with $F = A \cup D(F)$.*

1. Existence, uniqueness. *An emission at $p = (\text{exec}, \text{door}$ or $\top, \text{hash})$ depends only on the fold strictly below p; its execution time is at or above its trigger's — a data emission at the tipping observation just folded, a scheduled emission at or after its unit's inception* (H-FWD, Chapter 3) *— so the dependency strictly decreases in the well-order $<$, and F is fixed by well-founded recursion along $<$ and is unique. The operator is* not *monotone — a late arrival can suppress an emission (the covered call) — so uniqueness is by stratification on execution time, never a monotone least fixpoint. At one execution instant the order is completed by declared precedence for a non-commuting pair (an autocall's knock-out before its coupon) or the pair is refused (C-2.7, DL-01); the door-then-hash tiebreak orders only pairs that commute, so the recursion is well-defined within an instant as across instants. Among synthesised emissions of one instant the order is the* derivation order*: each emission sits strictly above every event, real or synthesised, whose writes its predicate reads* and *every event with which it write-conflicts — both setting one non-additive home field. This relation is a function of the record — what each emission reads and writes is recorded — so the order is identical for every party; the forward pass* computes *it and does not define it. A pair unordered by this relation — disjoint in reads and in non-additive writes, balance-moves excepted as additive — genuinely* commutes*, reaches one fold state in either order, and falls wholly to the event hash, definite because the hash admits no collision on*

the finite domain of well-formed events (H-CR, §above). A pair that write-conflicts with no reads-dependency to force its order is non-commuting *— two firings setting one unit's status — and is ordered by declared precedence or refused (C-2.7, DL-01), never by the hash.*

2. Termination. *Under* (H-FIN) *the candidate emission occasions (recorded observation points, scheduled dates) within the bounded horizon are finite and each watch fires at most once (`prop_firedMatchesOneEdge`), and under* (H-WF) *the "arms a further watch" relation is well-founded; then the closure adds finitely many emissions and the forward pass terminates.*

3. Confluence. fold step *is deterministic (replay determinism, P2), so the closure is independent of the order emissions are discovered and inserted.*

A retained emission whose predicate no longer holds re-runs to a fold-state no-op (H-INERT)*: its contract, re-executed against the refolded state, proposes the empty transaction, so a retained superseded emission leaves the fold state the timely side — which never held it — also reaches. H-FIN's at-most-once clause, H-FWD, H-INERT, and H-WF are property-tested (Chapter 15) — H-WF by the acyclicity condition on the cross-unit* arms a further watch *relation, checked at registration and refusing a cyclic product at the door (`prop_armingWellFounded`, Chapter 3). Only H-FIN's finiteness clause is structural, the per-unit closed node set (Chapter 3); a violated finiteness hangs the closure, it does not fail a test. None is an axiom.*

The theorem is over fold state alone and claims nothing of settled money, emitted external effects, or already-fired notifications: these are not rewound but discharged as C-12.6's residue — the restated flag, the explain item, and the open item that moves only under authorised compensation (f). Its hypothesis is per-event purity of step (clock confinement) with execution-time stability (TA-EXECUTION-TIME), which fixes e's position; without per-event purity the theorem fails, which is why the specification confines the clock.

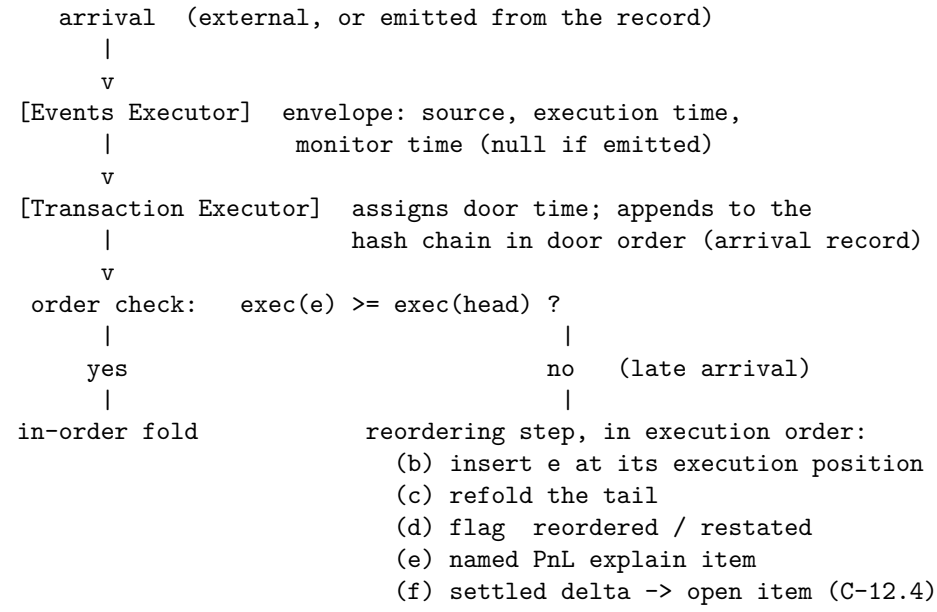
Cost, and snapshots. The refold re-runs fold step over the tail once: $O(|\text{tail}'|)$, where $|\text{tail}'|$ counts the external arrivals *and* the emissions the reordered prefix synthesises after the insertion point. The single forward pass folds each exactly once — no cascade multiplier — because no synthesised emission lands behind the cursor. A years-late correction has the whole subsequent history as its tail; unbounded lateness gives an unbounded tail, intrinsic to honouring execution order and never capped. Snapshots — materialised fold states — are keyed not by an ordinal position, which an interior insertion renumbers, but by the stable $(\text{exec}, \text{door}, \text{hash})$ triple T_{last} of the snapshot's last folded event — its door slot $\top$, ordered after the real door times of its instant, when that event is an emitted firing — immutable and record-derived. Inserting e with triple T_e invalidates every snapshot whose $T_{\text{last}} \geq T_e$; the nearest survivor is the greatest $T_{\text{last}} < T_e$, and the refold folds the events whose triple lies in $(T_{\text{last}}, \text{head}]$. No ordinal index, no renumbering: the conforming mitigation restores that survivor and refolds forward, a pure acceleration of the same fold, bit-identical to the full refold (Theorem 4.1) and decidable from the log alone.

Mitigation	Conforms?	Why
Snapshot + incremental refold from the nearest surviving snapshot ($T_{\text{last}} < T_e$)	yes	acceleration of the total-order fold; result bit-identical (Thm 4.1)
Refold only the invalidated tail, hash chain untouched	yes	what step (c) specifies; the prefix below T_e is unchanged
Refuse late arrivals (tolerance window or finality cutoff)	no	drops a court-enforceable execution time; the book stays knowingly wrong — the covered-call failure
Fold in door order, execution time as metadata	no	fold order $\neq$ execution order; contradicts C-2.7
Snapshot keyed by any ordinal position (door or total-order)	no	any positional key is stale under interior insertion; the conforming key is the stable last-event triple

Interactions. Watch firings themselves refold by step (c): a predicate the new order falsifies (a record-date firing that now finds zero shares) keeps its recorded event, its consequence voided; one the new order newly satisfies (a barrier a corrected close shows breached) is synthesised at its execution position, recorded firings retained as provenance (C-12.6). A settled leg of a split settlement obligation is terminal (Chapter 11): the refold restates the *view* of the split, the settled leg stands, any delta is a visible open item, and the failed-residual leg refolds freely. A late arrival that is also unroutable is held as a quarantined event carrying its execution time (Chapter 8); when its kind is registered or its reference resolved, the processing transaction's cause is that quarantined event, so it takes its execution-time place and the tail refolds — quarantine defers routing, execution-time insertion applies when routing resolves.

4.6 The event's journey, end to end

One pipeline carries an event from the world to the fold, and the covered call traces every stage. On Monday W-ALPHA's book holds 100 shares against a short American call it wrote; the stock's dividend has record date Wednesday, D per share. On Tuesday the holder exercises early — rational precisely because exercising before the record date captures the dividend — so at Tuesday's execution time the shares are called away. The assignment notice, travelling through the clearing house, reaches the door only Thursday.



Trace the assignment through the pipeline. Its envelope records execution time Tuesday (the notice's exercise time) and monitor time Thursday (the boundary observation); the door assigns

Thursday's door time and appends to the hash chain, the arrival record complete and never rewritten. At the order check exec = Tuesday $<$ X = Wednesday's record-date firing, so the reordering step runs: the assignment inserts at Tuesday, the tail refolds, the shares leave Tuesday, and *Wednesday's record-date firing now finds zero shares* — the entitlement $100 \times D$ was never W-ALPHA's. Three things are born here (C-12.6). The flags: the assignment *reordered*; the share balance, position home, and vanished entitlement *restated*. The explain item: "dividend entitlement $-(100 \times D)$, reordering, cause = Tuesday's assignment, lateness world's = Tuesday to the clearing-house observation, ours = observation to Thursday's door." The open item, where a dividend was already received: it moves back only as an authorised compensating transaction under C-12.4. The book that honestly showed 100 shares on Wednesday survives as the as-known-at view; the vanished dividend is named, not silent.

4.7 The orchestration substrate

The three times and the refold are ledger facts; the orchestration substrate that schedules the work supplies none of them and stores none of them. The immutable log is the sole book of record (Chapter 14). The substrate holds only schedules in flight and orchestration position, and survives no wipe. Its own clocks order nothing: none is any of the three times, which live on the log; the substrate reads them back as data and never writes them. Detection, the refold, and the past-dated firings it synthesises (step (c)) are the single writer's work (§4.5); the substrate only re-fires *forward* on the refolded state, never a past-dated firing.

The single writer serialises admission and refold: each triggered refold runs to completion over its full tail, its termination resting on H-FIN and H-WF (Lemma 4.2), while the overdue-watch sweep (Chapter 16) backstops a *lost* refold, not a non-terminating one. Only the quiescent closure is the specified, committed, observable state. Because the fold state is a function of the arrival set (Theorem 4.1) — door time decides only commuting ties, a non-commuting pair never falling to it — and the refold is idempotent (§4.5), any interleaving of late arrivals reaches one quiescent closure; the sole residual is a same-instant non-commuting pair, including two conflicting execution-time corrections of one instant, ordered by declared precedence or refused (C-2.7, DL-01), never by an arrival-order door tiebreak.

When the refold changes a unit's state, that unit's orchestration is *signalled to re-read*: it discards its in-memory mirror, rehydrates its node and armed-watch set from the refolded projection — the rehydration it already runs on start and on any doubt, and a refold is exactly "any doubt" — and re-fires forward under fresh cause-derived identifiers, never rewinding its own history. A record-derived backstop (the overdue-watch sweep, Chapter 16) makes a lost signal cost latency, never correctness. The forbidden alternative replays an orchestration's recorded history against the new fold, or compensates its past actions substrate-side: both treat the substrate as the book and smuggle ledger state into it. A timer that fired under the old order stays fired — arming and firing are recorded acknowledgements on the log — and the re-read orchestration reconciles by the C-12.6 flags, settled quantity moving back only as authorised compensation under C-12.4. The acceptance test is wipe-and-rebuild, evaluated at quiescence — after the re-read's forward acknowledgements are deposited: the state rebuilt from the refolded log equals the *timely* state, the fold of the same external arrivals with firings re-derived (Theorem 4.1, `prop_refoldEqualsTimely`), so an absent derived firing fails it — not merely what the in-place re-read reached. If the two differ, ledger state was smuggled into the substrate. The substrate-specific mapping is carried in the orchestration companion, not here.

4.8 The watch handshake

Trigger. On 2026-05-04 the issuer of stock ACME announces a cash dividend of 1.50 per share: record date 2026-05-15, payment date 2026-05-22. The announcement is an external event; the Events Executor captures and records it at the boundary.

Contract. The announcement fires ACME's smart contract once — not to pay, but to register the watches the dividend creates. Nothing is owed yet and nothing moves; what the dividend has created is two future obligations of attention, and the unit declares them exhaustively, exactly as it declared its watches at registration.

Transaction. One moveless transaction:

	Kind	Content
1	unit-state change	ACME: record-date watch declared, 2026-05-15
2	unit-state change	ACME: payment-date watch declared, 2026-05-22

The door. A moveless transaction is well-formed: a transaction is a bundle of four kinds of change and may carry no moves (Chapter 3). The identifier derives from the recorded announcement; conservation is undisturbed because nothing moved; the declaring writer is ACME's own contract.

The handshake. The Monitor acknowledges each declared watch; each acknowledgement is an event like any other, flows through the same pipeline, and lands as a unit-state update: both watches now visibly *armed*. On 2026-05-15 the Monitor emits the record-date event and that watch is *fired*; on 2026-05-22 the payment-date watch fires likewise. What the two firings do — record each holder's entitlement movelessly, then pay against the recorded entitlements — is Chapter 5's episode, and the entitlement's home is Chapter 6's; the machines' contribution is that neither date could have passed silently. Had either watch remained unacknowledged beyond its arming window, the unit's state would show a declared, unarmed watch: an overdue item, never an absence.

A watch is visibly declared, then armed, then fired or expired; a date that matters cannot pass silently, because attention itself is on the record.

4.9 The trust boundary

The three roles sit at different levels of trust, and the dotted line of the opening picture is exact: everything above it is untrusted. The Transaction Executor is the trusted component — the ledger's integrity rests on it and on nothing else. The smart contracts, the Event Monitor, and the Events Executor sit outside the boundary, untrusted in three different ways.

A contract can be economically wrong. Its proposal can be structurally impeccable and still book the wrong economics, and the door cannot know that. The defence is not prevention but recomputation: the events and the transactions are both on the record, so verification re-runs the map on the recorded event and compares (Chapter 15); a discrepancy is a defect of the contract, not of the ledger, and it is repaired forward by an authorised compensating transaction through the same door (Chapter 15).

The Event Monitor and the Events Executor cannot corrupt the ledger at all. A late emission or a late firing produces the identical transaction, because everything downstream reads the record and not the clock. A duplicate is committed once, under the cause-derived identifier. A spurious firing finds nothing due — the one-touch's second touch above — and proposes nothing, or proposes something the door refuses. Every failure these two machines can commit maps to delay or noise, never to wrong state.

What the untrusted machines do carry is liveness. Integrity owes them nothing; timeliness owes them everything: watches must actually arm, events must actually be emitted, contracts must actually be fired, obligations must actually be discharged by their deadlines. Those duties are stated as minimum requirements — durable watches and timers, at-least-once emission, acknowledged watches, triggers carrying timing not data, replayable orchestration, no write privilege, obligation liveness — in Chapter 16, and each degrades, when unmet, to a visible overdue item rather than to silence.

5 Smart Contracts

Governed by C-6.1–6.5 (§16.5).

The ledger admits well-formed transactions and records them; it performs no product-specific logic of its own. What a dividend, an expiry, a barrier knock, or a settlement print means for a particular instrument lives in that instrument's *smart contract*. An event arrives from the outside or is emitted by the Event Monitor; the ledger can do nothing with it until it is a transaction; the smart contract is the one function that performs the conversion.

One signature states the entire interface.

```
contract :: Event -> Ledger -> Transaction

-- what a contract is NOT (each of these would be a defect):
--   contract :: Event -> State Memory Transaction   -- it may not remember
--   contract :: Event -> IO Transaction             -- it may not reach out
--   contract :: Event -> Clock -> Transaction        -- it may not read time
```

Read the signature as a claim about what a contract may touch. Its only inputs are the event and the ledger as it currently stands: the declared terms of the instrument, the recorded observations, and the visible unit and position state at the moment of firing. There is no clock among the inputs — the clock is consulted once, by the Event Monitor of Chapter 4, and never again downstream — no private store, and no channel to the world. A smart contract is therefore a pure, deterministic function of its inputs, and, because none of those inputs is private to it, a stateless one. Each contract is exactly this: a stateless, machine-readable transcription of the legal terms and conditions of the instrument it governs. Purity is what makes economic verification possible at all: to check a recorded transaction, re-run the same contract on the same recorded event and compare (Chapter 15). A function that read a clock, a store, or the network could not be re-run to the same answer, and the guarantee would be lost.

Two principles govern every contract.

Principle 5.1 (Everything a contract consumes is on the record)**.** *Every input a smart contract reads — the instrument's terms, the observations it depends on, the positions it acts upon — is present on the record at the moment of firing. A contract reads nothing that is not recorded, so its output is a function of the record alone.*

Principle 5.2 (A contract never waits and never remembers)**.** *The waiting is done by the watch, which sits in the record, and by the Event Monitor, which notices when the watch's condition is met; the contract is invoked only after its event has fired, runs once, and returns. Whatever a firing must carry forward to a later firing it writes into the record — into one of the three homes of Chapter 6 — and the later firing reads it back. A contract that had to remember would need private state, and private state is a second store of the very facts the ledger already holds: the one thing this architecture exists to abolish.*

5.1 The dividend: three firings, one contract

The dividend shows both principles in one instrument fired three times. Wallet W-ALPHA holds 10,000 shares of the stock ACME.

The trigger. On 2026-05-04 the issuer announces a cash dividend of USD 1.50 per share on ACME. The announcement is captured at the boundary and recorded — an external event, like a trade.

The contract that catches it. The stock's smart contract fires on the announcement, and fires again on each date the announcement creates. The three firings are one contract, invoked three times on three different events; it carries nothing between them.

The transactions it produces.

Firing	Date	Event	Transaction emitted
1	2026-05-04	announcement	arm the record-date and payment-date watches; *no move*
2	2026-05-15	record date	PositionState: W-ALPHA entitlement = 10,000 × USD 1.50 = USD 15,000.00; *no move*
3	2026-05-22	payment date	move USD 15,000.00, issuer → W-ALPHA; PositionState: entitlement discharged

The first firing pays nothing; it registers the two watches the dividend creates and writes them into the unit's state. The second reads the owned positions as the ledger then stands and records each holder's entitlement — a moveless transaction: no cash changes hands, but the position state now carries what each holder is owed. The third reads those recorded entitlements and emits the cash moves against them. Nothing links the firings but the record: firing 2 does not remember firing 1, it reads the armed watch; firing 3 does not remember firing 2, it reads the recorded entitlement.

What the door checks. On firing 3 the Transaction Executor checks conservation — the USD 15,000.00 debited from the issuer wallet equals the amount credited to W-ALPHA, a paired-leg move that can neither create nor destroy quantity — and idempotence: the payment proposal carries an identifier derived from its cause, so a retried or late firing commits once (Chapter 4).

Two dividends in flight. A second dividend can be declared before the first has paid. ACME declares again on 2026-06-09, payable 2026-06-30, while the 2026-05-22 payment is still a week off. The record-date firing of the second dividend records a second entitlement for W-ALPHA, and the position state now carries both at once. They do not collide: each entitlement is a separate entry in PositionState, keyed by the cause-derived identifier of the firing that wrote it (Chapter 6), so the payment-date firing of the first discharges the first and retires its entry, leaving the second live until its own payment date. Two entitlements in flight is a routine Tuesday, and the position state holds one live fact for each.

Where an agreement encumbers the holder — the shares pledged, the cash trapped — the payment routes through a conditional obligation unit instead of paying directly; that refinement is Chapter 9's, and it changes what firing 3 emits, never that the contract is stateless and fired three times.

Three firings, three transactions, one stateless contract; between them the contract remembers nothing — the armed watches, the recorded entitlement, and the positions are all on the record.

5.2 The trigger classes differ in what they read, never how they run

The three firings of the dividend were reached three different ways. The announcement arrived from outside. The record date and the payment date were calendar conditions the Event Monitor noticed against the clock. A fourth way remains: a predicate on a recorded observation — a settlement price printing, a close falling below a barrier. These are the trigger classes, and they classify what a contract reads, never how it runs.

```
data Trigger
  = External  Observation  -- captured at the boundary (a dividend announcement)
  | Calendar  Date         -- a date arriving (a record, payment, or expiry date)
  | Condition Predicate    -- a predicate on a recorded observation (a print, a
    touch)
-- every trigger, whatever its class, fires the same  contract  above.
```

The class decides which watch the Monitor evaluates and which inputs the firing reads; the signature above is unchanged across all of them. For an `External` observation, which contract

fires is not improvised: it is the routing the data-kind registry declares for that event's kind (Chapter 8). A future exercises two of the classes on one instrument.

5.3 The future: settlement prints and the expiry unwind

FUT-IDX is a cash-settled future on the index IDX, multiplier 10, settled to market. W-ALPHA holds one contract, entered at 995 against the clearing house wallet. Its terms declare a watch on each daily settlement print and a watch on its expiry.

The trigger and the contract. On each recorded settlement print the Monitor emits and the future's contract fires. It reads two facts, each already on the record: the day's settlement level, a UnitStatus fact identical for every holder, and W-ALPHA's last applied level, a PositionState fact. From the two it computes the day's variation margin and emits it. On the third print, which is also the expiry, the same contract additionally unwinds the position — in one atomic transaction.

Day	Print	Variation margin	Move	PositionState
1	1,000.00	$(1000 - 995) \times 10 = +50$	USD 50, CCP → W-ALPHA	level ← 1,000.00
2	990.00	$(990 - 1000) \times 10 = -100$	USD 100, W-ALPHA → CCP	level ← 990.00
3	1,005.00	$(1005 - 990) \times 10 = +150$	USD 150, CCP → W-ALPHA	position unwound

The contract never remembers yesterday's level; it reads it from PositionState, computes against today's print, and writes today's level back for tomorrow's firing. Cumulative variation margin is $+50 - 100 + 150 = +100 = (1005 - 995) \times 10$ — exactly the position's profit, arrived at one recorded firing at a time.

More than one lot. The single-contract reading above stores one level, but a holder can carry several lots entered on different days, and a new lot can trade between two prints. So PositionState does not store a bare level; it stores the *aggregate settled value* $S = m \sum_i q_i \cdot \text{level}_i$ as one integer, for multiplier m — the value the position's lots have already been marked to (Chapter 6). The day's variation margin is then $m\,\text{settle}_t\,Q - S$, where $Q = \sum_i q_i$ is the total quantity. For one lot at level 995 this is $S = 10 \times 995 = 9{,}950$ and day 1 gives $10 \times 1{,}000 - 9{,}950 = +50$, the table above. When a new lot of q' trades at price p before the print, the firing folds it in at its trade price, $S \leftarrow S + m\,q'\,p$: if W-ALPHA trades a second contract at 992 before day 2's print, $S = 10{,}000 + 10 \times 992 = 19{,}920$ and $Q = 2$, so day 2's margin is $10 \times 990 \times 2 - 19{,}920 = -120$ — the old lot marked from yesterday's 1,000 (-100) and the new lot from its own trade price 992 (-20). The whole position needs a single stored integer, not one entry per lot. This is what a clearing house keeps, and it is why the futures fact rides inside the position-state collection (Chapter 6) as one aggregate entry rather than accumulating a fact per lot.

What the door checks. The margin move is conserved — what W-ALPHA receives the clearing house pays, and conversely — and each firing multiplies the multiplier by *the recorded settlement level*, never a stale one, so no firing books a quantity against a price from a different state (Chapter 14).

The daily print and the expiry unwind are the same contract fired on different events; the settlement print and the dividend announcement differ in what the contract reads, never in how it runs.

5.4 The one-touch: one firing, one transaction, and idempotence

OT-1 (Cast, §2.1) declares two watches on OMEGA. The first is on OMEGA's recorded official close against the barrier — and because the barrier is observed at that close, the watch is exactly the barrier edge's declared guard, so the term is transcribed faithfully (Chapter 3's product-graph consistency). The second is OT-1's corporate-action out-edge: because OT-1's barrier references OMEGA, its terms watch OMEGA for a corporate action, and should OMEGA split two-for-one

this edge fires, appends a new ProductTerms version carrying the barrier 80.00 → 40.00 under the term-adjustment operator (Chapter 8), and leaves OT-1 on *live* — a self-loop that bumps the terms version, not the state (Chapter 3, Section 3.8). Without it the first post-split close near 47.50 would knock a barrier struck at 80.00 against a halved stock, a phantom the adjusted barrier 40.00 refuses.

The trigger and the contract. OMEGA closes at 79.00; the close is recorded, the condition is met, and the Monitor emits the touch. The contract fires and reads exactly two things from the record: the declared payout and who holds the option. It returns one transaction — a single move of USD 1,000,000 from W-BANK to W-ALPHA, together with one unit-state change, OT-1 → *triggered*.

What the door checks. Conservation on the payout, and idempotence. Idempotence is not something the contract defends: should the Monitor emit the touch a second time — barriers are often crossed on many later days — the next firing reads the unit and finds it already *triggered*, so it proposes nothing, and even a duplicate of the original proposal carries the same cause-derived identifier and commits once at the door.

Idempotence is guaranteed not by the contract's vigilance but by the recorded state and the cause-derived identifier; the touch contract is a pure function of the record — terms and holder in, one transaction out.

5.5 The variance swap: a contract never remembers

VS-1 is a one-year variance swap on IDX with 252 scheduled fixings against the same recorded official closes the future settles on. Each scheduled fixing is a declared watch; on fixing k the Monitor emits and the contract fires. To extend the realised variance the firing needs the previous close and the running accrued statistic — the fixings $1 \dots k-1$. It does not hold them: the previous close is a recorded observation the firing reads from the record — a home fact folded from its observation-recording transaction, never a live feed — and the accrued statistic is a UnitStatus fact that firing $k-1$ wrote for exactly this purpose (Chapter 6). Firing k reads both, forms the return, adds its square, and writes the updated accrued statistic back for firing $k+1$. A firing 200 days into the swap's life reconstructs its entire history from the record and holds nothing of its own between firings.

Ordering. The fold obeys execution order, not arrival order — the per-unit case of the total order (Chapter 4). A fixing carries two coordinates: its execution-time index k — the fixing-sequence position it belongs to — and its door-time position, where its print was appended to the log (Chapter 2, Section 2.5). The accrual folds in execution-time order, so replay is deterministic regardless of arrival order. A duplicate print is inert, absorbed once under its cause-derived identifier; a late or out-of-order print — one whose door-time position trails its execution-time index — is not a no-op: it is folded at its observation position k, and the accrued-statistic tail $A_k, A_{k+1}, \dots$ is recomputed from k forward. The recomputed result is identical to timely arrival because the fold is over the execution-time sequence, while the as-known-at accrual reported before the print arrived is preserved beside it; any figure already *posted* against a superseded tail is corrected by an authorised compensating transaction, never left standing (Chapter 15, forward repair).

Firing k finds fixings $1 \dots k-1$ on the record; a contract never remembers.

5.6 Faithfulness: every right and obligation is a unit

Purity and statelessness say how a contract runs. One further demand says what it must produce: the alignment with the legal contract runs deep enough that every right and obligation the legal contract creates is represented in the ledger as a unit. A subscription is booked as an exchange — cash against the client's redemption claim — never as a transfer for nothing. A dividend not yet paid is a recorded entitlement, seen above. A payout trapped by a collateral agreement is a

conditional obligation unit (Chapter 9). Because each such right and obligation is a unit or a declared term of one, the contract that transcribes the instrument has, at every firing, a home on the record for everything it must carry — which is why it can afford to remember nothing. Statelessness and faithfulness are the same discipline seen from two sides: the contract keeps no private state precisely because the record already holds, as units and their declared terms, every fact its next firing will need.

6 State: The Three Homes

Governed by C-7.1–7.5 (§16.5).

A unit's state is not one thing. Some of it is a property of the unit alone — the settlement level a future has printed, the fixings a variance swap has accrued, whether a note's barrier has knocked — the same fact for every holder. Some of it is a joint property of the unit and a particular wallet — the variation margin this holder applied yesterday, the dividend that holder is owed. And some of it is the contractual text itself — a multiplier, a strike, a fixing schedule — versioned and append-only. Three kinds of fact, and the state model gives each its own home: **UnitStatus**, **PositionState**, and **ProductTerms**. With the balances of Chapter 3 these are the four components of the ledger's state, and the four kinds of change a transaction bundles (Chapter 3) write them one for one.

6.1 The three homes

ProductTerms holds the versioned, append-only contractual terms of an instrument: the future's multiplier, the option's barrier and payout, the variance swap's strike and 252-fixing schedule. A new version is appended, never an edit; the current terms are the latest version, and any past state is read against the version in force then. Its facts are properties of the instrument's terms, and a corporate action that changes those terms writes a new version (the split below).

UnitStatus holds the facts that are properties of the unit as a whole — the same for every holder: the last recorded settlement level, the running accrued sum of squared returns, the live / triggered / expired lifecycle node — including, for a settlement-obligation unit (Chapter 11), its instructed / settled / failed node, so that an instructed-but-unsettled trade keeps its settlement level in a home like any other unit and needs no fourth home invented for it. It is a materialised projection of the unit-state changes on the log, written by the instrument's contract as its life unfolds.

PositionState holds the facts that are joint properties of a unit and a wallet: this holder's last applied settlement level, this holder's recorded dividend entitlement. A fact lands here exactly when it depends on which wallet holds the unit — when one unit-level event yields a different value to each holder.

None of the three holds a balance — that is the coordinate vector of Chapter 3, the transaction's fourth kind of change — and none holds a view. The keys distinguish them: ProductTerms and UnitStatus are keyed by the unit, PositionState by the wallet-and-unit pair.

```
data Homes = Homes
  { productTerms  :: Unit           -> [TermsVersion]  -- append-only; latest is
    current
  , unitStatus    :: Unit           -> UnitFacts       -- properties of the unit
    alone
  , positionState :: (Wallet, Unit) -> PosFacts        -- joint facts, a
    collection (below)
  }
-- each field is a fold of the log; discard and replay reconstructs it.
```

PositionState holds a collection, not a slot. One `PosFacts` value per (wallet, unit) pair would be too few, because two ordinary situations need more than one live fact on the pair at once.

Two dividends in flight. ACME declares a dividend on 2026-06-02, payable 2026-06-23, and a second on 2026-06-09, payable 2026-06-30 — before the first has paid. On the two record dates W-ALPHA's holding earns two entitlements, say USD 15,000 and USD 12,000, and both are owed at the same time, each waiting for its own payment date. A holder carrying several unpaid entitlements on one stock is a routine Tuesday, not an exotic case. The pair (W-ALPHA, ACME) must therefore hold two live facts, and each must retire on its own payment date, not the other's.

A futures lot traded today. W-ALPHA holds a futures position marked to yesterday's settlement level and today trades one more lot at a new price, before today's print. Today's variation margin owes the old lots their margin from yesterday's level and the new lot its margin from today's trade price. The pair must carry both references at once.

So the slot is not one fact. `PosFacts` is a *collection* of a pair's live facts, each keyed by the cause-derived identifier of Chapter 4 — the identifier of the firing that wrote the fact. Two firings on the same pair write two distinct entries, and neither overwrites the other.

```
type Txid     = Hash             -- the cause-derived identifier of Chapter 4 (
    writing firing)
type PosFacts = Map Txid PosFact -- a pair's live facts, ONE entry per writing
    firing
data PosFact
  = Owed         Money      -- a determined entitlement; retires when its payment
    firing pays it
  | SettledValue Integer    -- a futures mark: the aggregate  sum q_i * level_i ,
    ONE per pair
-- retirement: an entry leaves the collection exactly when its obligation
    discharges.
```

The retirement rule. An entry is live for precisely the interval between the firing that determines it and the firing that discharges it. The record-date firing of the first dividend writes an `Owed` entry under its own identifier; the payment-date firing pays it and retires that entry, while the second dividend's entry — written under a different identifier — stays live until its own payment date. The collection at any moment is exactly the pair's outstanding obligations.

The futures specialisation. The multi-lot case does not need one entry per lot. The margined lots collapse into a single `SettledValue`: the aggregate settled value $\sum_i q_i \cdot \text{level}_i$ carried as one integer, from which day t's variation margin is $\text{settle}_t \times Q - \text{storedSettledValue}$ for total quantity $Q = \sum_i q_i$ (Chapter 5). A new lot is folded in at its trade price, so the old lots still mark from yesterday's level and the new lot from today's price — the multi-lot case falls out of one integer, which is what a clearing house actually keeps. This is why the event-keyed collection is the primitive and the aggregate rides inside it: two dividends in flight cannot be added into one number, because each discharges on its own date and must retire alone, so the collection must be keyed per firing; the futures lots all discharge against the same daily print, so they may be summed, and their sum is just one entry in that same collection.

Two disciplines make the homes trustworthy, and both are checkable.

Principle 6.1 (One fact, one home)**.** *Every non-balance fact has exactly one home, determined by what the fact is a property of: the unit alone (UnitStatus), the unit-and-wallet pair (PositionState), or the contract's terms (ProductTerms). No fact is kept in two homes, so no two homes can disagree about it.*

Invariant 6.2 (One legal writer)**.** *Every non-balance fact has exactly one legal writer — the smart contract of the instrument the fact belongs to — and no other contract may write it. The Transaction Executor enforces this at the door: a transaction that writes a fact outside its*

writer's authority is refused. The invariant joins the structural catalogue of Chapter 14 with constitution §11's writer discipline.

6.2 One fact, one home: the future exercises all three

The daily life of a cash-settled future carries all three kinds of fact, so it visits all three homes. FUT-IDX is a future on index IDX, multiplier 10, USD, settled-to-market; W-ALPHA holds one contract. On day 1 the official close prints 1,000.00, the Event Monitor emits, and the future's contract fires. Three facts are in play, and each has one home.

Fact	Property of	Home	Legal writer
Settlement print 1,000.00	the unit alone	UnitStatus	the print firing
W-ALPHA's applied level; VM USD 50	the unit–wallet pair	PositionState	the print firing
Multiplier 10	the contract's terms	ProductTerms	registration (v. 1)

The print is one number for every holder: it is a UnitStatus fact. The variation margin depends on the level W-ALPHA last applied, a fact of the wallet-and-unit pair: it is a PositionState fact, and the same print yields a different margin to a holder who entered at a different level. The multiplier was fixed at registration in ProductTerms and changes only by a new version. The print firing writes the first two facts in one transaction — two facts, two homes, one writer — and reads yesterday's applied level back from PositionState rather than holding it (Chapter 5).

Each fact has exactly one home, fixed by what it is a property of; the future, carrying three kinds of fact, exercises all three homes.

6.3 The homes suffice

The homes are not bookkeeping conveniences. Their reason for existing is a sufficiency requirement, and it is the load the rest of the architecture rests on.

Principle 6.3 (The homes suffice)**.** *The three homes, together with the balances, carry all the information the smart contracts require in order to execute. Every fact a firing must carry forward to a later firing is written into a home by an earlier firing, so the later firing finds its entire context on the record; if a contract would ever need to remember something, that something has a home.*

This is the precise reason a contract is stateless and never remembers (Chapter 5): statelessness is affordable only because every fact a firing would want to remember already has a home the firing can read. The sufficiency is what turns "never remembers" from an aspiration into a consequence.

Minimality is its other half. A home holds a *fact* — an input a later firing reads — never a *view*. NAV, profit and loss, availability, and encumbrance are projections, computed on demand from the homes and the log and never materialised into one (Chapters 7 and 12). The homes carry every fact a contract must read and no figure a valuation or a report recomputes — which is exactly why no store of "realised PnL" exists anywhere to be reconciled against the position record.

The variance swap exercises the requirement at full stretch.

Episode — the variance swap (T5): sufficiency, exercised. *Trigger.* The 127th scheduled IDX close is recorded and the Event Monitor emits VS-1's 127th fixing. VS-1 is a one-year OTC variance swap on IDX, 252 scheduled fixings against the recorded IDX official closes — the same series FUT-IDX settles on. *Contract.* VS-1's contract fires. To extend the realised variance it needs the whole path so far: the previous close and the running sum of squared

returns over fixings $1, \ldots, 126$. It holds neither, and it is forbidden to. *What it finds on the record.* The previous close is a recorded observation. The accrued statistic is a UnitStatus fact of VS-1 — a property of the unit alone, the realised path as the swap has seen it — written by the 126th firing for exactly this reader. At the mid-life checkpoint (fixing 126, half of the 252 fixings elapsed) it stands at $A_{126} = 1{,}805{,}000$, an integer at scale 10^{-8}: the 126 accrued squared returns compressed into one number. The firing reads A_{126} and the previous close, forms the 127th return, adds its square, and writes A_{127} into UnitStatus for the 128th firing. *The coupling.* The IDX close at fixing 126 is 1,005.00 — the same recorded close FUT-IDX takes as its day-3 final settlement; the closes $C_{124}, C_{125}, C_{126}$ read at fixings $124, 125, 126$ — the future's three settlement days — are 1,000, 990, 1,005. One recorded observation, two consumers: the future writes it as a settlement level into FUT-IDX's UnitStatus, the swap folds it into VS-1's accrued statistic in VS-1's UnitStatus, and neither reads the other's home. *Design point. Firing 127 finds its entire context on the record — 126 fixings compressed into one UnitStatus integer $A_{126} = 1{,}805{,}000$ — so a contract that spans a year of history remembers nothing between firings. The homes suffice.*

6.4 Two more facts finding their homes

The dividend and the split add one home each: the entitlement is per-holder, the split is a terms change.

Episode — the dividend (T3): an entitlement in PositionState. On 2026-05-15, the record date, ACME's contract fires and reads the owned positions as the ledger then stands: W-ALPHA holds 10,000 ACME. It records the entitlement $10{,}000 \times \text{USD } 1.50 = \text{USD } 15{,}000.00$. A dividend entitlement is a property of the unit-and-wallet pair — the same USD 1.50 per share yields a different amount to each holder — so it lands in PositionState. The transaction is moveless: no cash moves on the record date, but the position state now carries what W-ALPHA is owed, for the payment-date firing to discharge (Chapter 5). *Design point. An entitlement owed but unpaid is a fact with a home; the record date writes it into PositionState movelessly, and the payment date reads it back.*

Episode — the split (T4): a new version in ProductTerms. On 2026-06-01 ACME's 2-for-1 split becomes effective. In one atomic transaction the holding moves from 10,000 to 20,000 shares (a paired-leg move, Chapter 3) and ACME's terms gain a new version: ProductTerms ACME version 2 carries the split, and the recorded price frame reads 100.00 as 50.00 after it (the market data operator's work, Chapter 8). A split is a property of the contract's terms, so it lands in ProductTerms and nowhere else. The terms are not edited; a version is appended. *Design point. Terms change only by appending a version; the split is one ProductTerms fact, atomic with the move it accompanies, and the append-only discipline is what lets any past state be read against the terms in force then.*

6.5 Rebuildable by replay

Principle 6.4 (Homes are projections)**.** *Each home is a deterministic projection of the log — ProductTerms of the terms changes, UnitStatus of the unit-state changes, PositionState of the position-state changes. Any home may be discarded and rebuilt by replaying the log, without loss of information, so no home is a store that can drift from the record.*

This is the precise sense in which the homes are *materialised projections* and not second copies. A home cannot play that role: it has exactly one legal writer (the one-writer invariant above), it holds each fact in exactly one place (one fact, one home), and it is a pure fold of the one log everything else also folds. Materialising the accrued statistic into UnitStatus is therefore not the reintroduction of a store to reconcile; it is a cached read of the log, provably equal to the log it came from. What is materialised is a fact a contract must read; what is never materialised

is a view a projection can recompute. Between them the homes carry everything the contracts need and nothing they do not.

The accrued statistic is order-dependent: each firing reads the previous accrual and extends it, so a home rebuilds by replaying the fixings in execution-time order, not arrival order — a late print inserts at its execution-time index k and the tail $A_k, A_{k+1}, \ldots$ recomputes from k forward, identical to timely arrival (Chapter 5, VS-1; Theorem 14.5). Downstream figures already posted against a superseded tail — the future's variation margin on the same recorded closes — are corrected by authorised compensating transactions (Chapter 15, forward repair), never left as posted.

7 Valuation and Profit and Loss

Governed by C-8.1–8.9 (§16.5).

Net Asset Value (NAV) has an operational meaning before it has a formula. The initial NAV of a wallet is the cash it took to build the portfolio at inception; the current NAV is the cash you would hold if you unwound every position at prevailing prices; and the profit and loss (PnL) is the difference between the two. Nothing in this chapter is a new store: every number it produces is a projection of the immutable log, computed on demand and never written back.

7.1 Net Asset Value

The operational account becomes a formula in two steps. First, replay the log to recover the wallet's current composition — the signed coordinate vector, per unit, that the fold of Chapter 2 produces. Second, multiply that composition by the current price vector supplied by the pricing data layer, summing over the valued units only.

Definition 7.1 (Net Asset Value)**.** *The NAV of a wallet w at a log position is*

$$\mathrm{NAV}(w) \;=\; \sum_{u \text{ valued}} \mathrm{owned}(w,u) \cdot P(u, \sigma(u))\,,$$

where $\mathrm{owned}(w,u)$ *is the owned coordinate of w's balance in unit u (Chapter 3), $\sigma(u)$ is the current lifecycle state of u read from the ledger, and P is the price the pricing data layer supplies for that unit in that state.*

Only the owned coordinate enters. The lent and posted coordinates carry mass, never value (Chapter 3): a security posted as collateral is still owned by the poster and is already counted once, on its owned coordinate; counting the posted marker as well would value one economic reality twice. Non-valued units — a strategy rulebook, a mandate's governance shell — have no price and enter no sum; their value is undefined, not zero. A valued unit may nonetheless price at zero, and that is a different thing: when a lifecycle event has extinguished its exposure — a triggered one-touch, or a settled-to-market future after the day's margin (§7.3) — the unit stays valued and enters the sum at $P = 0$, so its contribution is defined and it is FUT-IDX, not a strategy shell.

Two disciplines are visible in the definition and are carried by the machinery of Chapters 3 and 8. The composition is one number per coordinate, obtained from one fold of one log, so no two readers can disagree about what is held. And a price is never combined with a quantity across a corporate-action event: after a two-for-one split, 2,000 shares are multiplied by the post-split frame 50.00, never by the stale 100.00 — consistency of reference, catalogued in Chapter 14.

```
type Points = Integer                          -- money in minor units, exact
price :: Unit -> State -> PriceVector -> Points   -- pricing is state-
    parameterised
```

```
nav   :: Composition -> PriceVector -> Points
nav c pv = sum [ q * price u (stateOf c u) pv     -- owned coordinate, valued
    units
               | (u, q) <- ownedBalances c, valued u ]
```

7.2 Profit and loss is the variation in NAV

Principle 7.2 (PnL is NAV variation)**.** *The profit and loss of any wallet between two log positions is exactly the variation in its NAV over that interval. PnL is therefore path-independent, and no derived figure a projection could recompute — no second copy of "realised PnL," no running valuation — is ever maintained alongside the position record.*

The principle is a consequence of the definition, not an additional convention. NAV is a function of the composition and the price vector alone; its variation between two positions depends only on the endpoints, never on the sequence of transactions that connected them. A stored realised-PnL figure would be a second record of a fact the log already determines — precisely the redundancy the framework exists to remove. Book cost is the opposite case, and the distinction is Chapter 6's: a home holds a fact a contract must read, never a view a projection can recompute. W-ALPHA's traded price 995.00, recorded into PositionState at the trade (Chapter 3, T1), is book cost — a fact the future's contract must read to compute each day's variation margin — so it is held, not recomputed; realised PnL is a view, so it is recomputed, never held. Realised and unrealised PnL remain available as *projections*, cut from the same log by a reporting convention (Chapter 12), but neither is a source of truth.

Episode — the future. FUT-IDX is a cash-settled future on index IDX, multiplier 10, USD, settled-to-market. Wallet W-ALPHA buys one contract at 995 on day 0. The official settlement prints on the three settlement days are 1,000, then 990, then 1,005; the contract's smart contract fires on each print and emits the day's variation-margin (VM) move.

Settlement day	Print	VM move (owned cash)	Running NAV change
Day 1	1,000	+50	+50
Day 2	990	−100	−50
Day 3	1,005	+150	+100

Each VM is a settlement — an owned-cash move that extinguishes the day's exposure, with no return obligation, because none exists in law for a settled-to-market contract (§8, case 1; developed in Chapter 9). The cumulative VM is $50 - 100 + 150 = 100$, and the position's PnL is $(1{,}005 - 995) \times 10 = 100$. The two agree, and the second computation never touches the intermediate prints 1,000 and 990: the path is irrelevant, only the endpoints 995 and 1,005 matter. *PnL is the variation in NAV, and it is path-independent.*

7.3 Pricing is parameterised by state read from the ledger

Because a unit's value depends on where it stands in its lifecycle, the price function is parameterised not only by the unit but by its state — and that state is obtained from the ledger, from UnitStatus and PositionState (Chapter 6), never from a store the pricing layer keeps of its own. The pricing layer computes; the ledger is the sole authority for the state it computes in.

Episode — the one-touch. OT-1 (Cast, §2.1) is marked USD 400,000 by the pricing layer while live. When OMEGA's official close prints 79.00, the Event Monitor emits the touch and the option's contract records the lifecycle transition: UnitStatus → *triggered*. The pricing layer, reading that state from the ledger, now prices the same unit in state *triggered* at 0: the

contingent claim has resolved, its economic value realised as the cash payout that Chapter 9 discharges.

The mark on OT-1 collapses from USD 400,000 to 0, but the mass does not move: one unit of OT-1 remains on W-ALPHA's owned coordinate, and any posted marker over it remains on its posted coordinate, until the payout completes and the unit retires from the zero vector. *A lifecycle event extinguishes value, never mass: coordinates carry mass, prices carry value, and the price turns on a state that lives on the record.*

Episode — the settled-to-market future. The same rule fixes the price of FUT-IDX, and fixes it where an undisciplined mark would double-count. W-ALPHA is long one contract and W-CCP is short it, so the two owned coordinates carry $+1$ and -1, and FUT-IDX is a valued unit that Definition 7.1 sums. But the day's variation margin has already settled as owned cash (the episode of §7.2). Were the pricing layer now to mark FUT-IDX at the futures level, it would count the day's move twice — once in the VM cash already owned, once in the mark — and W-ALPHA's NAV would be overstated by exactly that move, while W-CCP's would be understated. So in every committed state, after the day's settlement has been applied, a settled-to-market future prices at 0: its exposure is extinguished, and there is nothing left to mark. A collateralised-to-market future prices instead against the last settled level, because a return obligation survives the settlement, and that obligation is itself a unit (Chapter 9).

```
-- FUT s bundles the declared regime (a term of the unit) and PositionState.
price u s _ | isFuture u, stm u = 0                      -- STM: exposure
    extinguished; VM already owned
            | isFuture u, ctm u = markAt (lastSettled s)  -- CTM: return
    obligation survives; it is a unit
```

This is §9.5's regime key — the one declared bit that separates settled-to-market from collateralised-to-market — read on the valuation plane. The bit therefore carries a *second* consequence beyond the balance sheet: it decides not only whether a return-obligation unit is booked (the balance-sheet consequence of §9.5) but whether the future is marked at all. A misdeclaration corrupts NAV as well as the balance sheet — declare a settled-to-market future as collateralised-to-market and the pricing layer marks an exposure that is already extinguished, double-counting the day's move; declare the converse and it drops a mark it should carry. This sharpens §9.5's warning about the regime bit's misdeclaration, giving it one more symptom, rather than repeating it.

7.4 Deposits do not move profit and loss

For the variation in NAV to be exactly profit, an inflow of value must not, by itself, create profit. The constitution's guarantee (§8, as amended) is that every inflow of value is one of three things, and which one is not a matter of asset class but of the legal regime declared once on the governing agreement unit.

(1) **An exchange paired with an equal-valued outflow.** Settled-to-market margin is this case: an owned-cash move that settles the day's exposure with no return obligation, because the settlement extinguishes the exposure outright. The future's VM above is the worked instance.

(2) **A contribution or financing received against an equal obligation created in the same transaction.** Collateral received under title transfer, cash included, is this case: the receiver owns the inflow and owes an equivalent-return obligation that is itself a unit.

(3) **Value held in custody without being owned.** Collateral received under a security interest without right of use, segregated cash included, is this case: it is recorded on the received-collateral coordinate and never touches owned.

Case 1 adds equal and opposite owned value; case 3 adds no owned value at all. Case 2 is the one that could leak profit; whether it does turns on the declared financing rate, as the next proposition shows.

Proposition 7.3 (Deposit-neutrality of title-transfer financing, and its off-market limit)**.** *Constitution C-8.7 states case-2 deposit-neutrality conditionally: the inflow is received against an obligation* valued at fair value*, any day-one difference between the inflow amount and the obligation's fair value recognised as financing basis. This proposition discharges the clause by exhibiting both sides of the condition.*

The return obligation minted in case 2 is a valued unit. When the declared financing rate equals the fair rate for the term, its fair value is exactly the inflow amount — par plus accrued — so at receipt the owned cash and the obligation are equal and opposite in the owned sum, net owned value is unchanged, and NAV does not move. In that case deposit-neutrality is a theorem, not an approximation.

When the declared rate is off-market, the difference is measured, never hidden. Receive USD 100 of cash against an obligation whose fair value is USD 98 — financing taken below the fair rate — and net owned value rises by USD 2 at receipt. That USD 2 is the day-one financing basis *C-8.7 names: the correct measurement of the off-market terms, recognised in the open at receipt, exactly as the clause requires.*

In the fair-rate case, because the obligation is priced at par-plus-accrued, its value tracks the cash it returns for the life of the financing, and the receipt neither raises nor lowers NAV; only the subsequent accrual — a genuine financing cost or benefit — moves it, as it should. This is why PnL needs no regime branch. The regime decides *where* an inflow lands — owned plus an obligation unit, a posted marker, or an owned settlement — but once it has landed, PnL is computed uniformly.

Principle 7.4 (No regime branch in PnL)**.** *PnL is* $\sum \text{owned} \cdot P$ *over the valued units, with no branch on collateral regime. The regime is a property of the transaction shape and the agreement unit's declared terms, not of the valuation.*

A fair-value deposit therefore cannot move profit and loss — not because a formula subtracts it, but because it cannot change net owned value. Off-market financing is the one case that can, and that case is the parked conflict above. The full mechanics of the three regimes, the obligation and claim units, and the entitlement trap are Chapter 9's; this chapter uses only their valuation consequence.

7.5 Mid-life valuation is read from the record

State-parameterised pricing extends past a binary knock to any unit whose value depends on facts its own history has accrued. The point is that those facts have a home on the record (Chapter 6), so the pricing layer reads them; it does not remember them.

Episode — the variance swap. VS-1 is a one-year OTC variance swap on IDX, 252 scheduled fixings against the recorded official closes, variance strike $K = 400$ points, variance notional $N =$ USD 1,000 per point, no cap. Its declared convention annualises over 252 returns, so realised variance in points is $\sigma_R^2 = 10^4 \cdot \sum_{i=1}^{252} R_i^2$, where R_i is the i-th log return of the close series. Each firing writes the running sum-of-squared-returns statistic into UnitStatus as an integer at scale 10^{-8}, so firing k finds fixings $1, \ldots, k-1$ already on the record.

VS-1 fixes on the same recorded IDX closes the future settles against. The future's three settlement days are fixings 124, 125, 126 of the series (closes 1,000, 990, 1,005); the future's final settlement level, 1,005, is therefore the variance swap's mid-life checkpoint close at fixing 126 — one recorded fact, two consumers.

At the mid-life checkpoint (fixing 126, half the fixings elapsed) the accrued statistic is $A_{126} = \sum_{i=1}^{126} R_i^2 = 0.018050$, stored as the integer 1,805,000. Expressed in points and annualised to date this is $10^4 \cdot A_{126} \cdot (252/126) = 361$ points — a realised 19-vol over the first half. The mid-life mark to the variance receiver is that accrued statistic plus the remaining implied variance, both taken from the record: with remaining implied variance $I = 484$ points (22-vol) over the remaining half-year fraction 126/252,

$$V_{126} \;=\; N\left[\,10^4 \cdot A_{126} \;+\; I \cdot \tfrac{126}{252} \;-\; K\,\right] \;=\; 1{,}000\,[\,180.5 + 242 - 400\,] \;=\; \text{USD } 22{,}500.$$

The accrued term (180.5 points) is a projection of a UnitStatus integer; the implied term (242 points) is a market observation with provenance (Chapter 8); the strike is a declared term. No figure is stored as a mark. When the year completes, the remaining fixings accrue $10^4 \cdot \sum_{127}^{252} R_i^2 = 260.5$ points, the realised variance is $180.5 + 260.5 = 441$ points, and the settlement is $1{,}000 \times (441 - 400) = \text{USD } 41{,}000$ to the receiver — the mid-life mark and the endpoint cut from one accruing statistic. *A mid-life value is not a stored quantity; it is the record, priced.*

Episode — the total return swap. TRS-1's reference leg is defined as the NAV of a designated wallet, W-QIS, in a virtual ledger. Valuing that leg requires no new machinery: it is the identical fold, $\sum \text{owned} \cdot P$ over W-QIS, that values any wallet. The stamped NAV enters the true ledger as an observation, never a move (Chapters 8 and 10), and the TRS contract reads it at each reset. *The reference leg of a swap is a NAV projection, folded like any other.*

7.6 Dual valuation: two price vectors over one position set

A wallet is valued for two purposes that demand different prices. Settlement, margining, and reconciliation against a counterparty, a CCP, or a custodian demand the price at which value actually changes hands — the mark-to-market price, P^{mk}, which may differ by venue. Risk aggregation and PnL attribution demand one internally consistent price across the book — the mark-to-mid price, P^{md}. The ledger supplies both as projections over one and the same composition.

Principle 7.5 (One position set, two price vectors)**.** *Mark-to-market and mark-to-mid are two price vectors applied to the single composition produced by the fold. Position divergence is impossible: there is one position record, so only two money figures* — NAV^{mk} *and* NAV^{md} — *can differ, never the positions themselves.*

Settlement and risk read the same coordinates from the same log, so they cannot drift apart. The two numbers that differ are honest reflections of two questions — what would settle, and what is the consistent risk mark — and their difference is a basis, computed on demand, never a break to be repaired.

Example — three venue marks, one risk mid. A holding of 3,000 units of U sits in three custody wallets — W_A, W_B, W_C — one per venue: 1,500 in W_A, 1,000 in W_B, 500 in W_C. *Venue is not a new coordinate.* Sub-custody is nothing but the ordinary partition of a position across wallets, a dimension the fold already carries; "at venue A" means "in the custody wallet W_A," and the mark-to-market price is the official settlement close declared for that wallet's venue, read at the boundary like any other observation. Each venue publishes its own close, used for that venue's margin and reconciliation: $A = 100.20$, $B = 100.00$, $C = 99.60$. Risk marks the whole holding at one mid, 100.00. P^{mk} is thus a function of the custody wallet, applied wallet by wallet:

$$\text{NAV}^{\text{mk}} = \sum_{w \in \{W_A, W_B, W_C\}} P^{\text{mk}}(w)\,\text{bal}_{\text{owned}}(w, U) = 1{,}500(100.20) + 1{,}000(100.00) + 500(99.60) = \text{USD } 300{,}100,$$

$$\mathrm{NAV}^{\mathrm{md}} = 3{,}000(100.00) = \text{USD } 300{,}000.$$

The two money figures differ by USD 100, the cross-venue basis. The 3,000 is one number from one fold over the three custody wallets: each wallet's sub-position reconciles against its own venue statement under P^{mk}, and the book aggregates for risk under P^{md}, but the position cannot diverge because there is only one of it and no venue axis was added to hold it. The reduction is exactly *sub-custody is a wallet partition*: it holds when the venue distinction coincides with a custody split. An execution venue with no custody partition — one custody account marked against two exchanges' closes — is a boundary-reconciliation matter, not a second position, and a CCP-cleared position is a counterparty wallet with a clearing-level mark (Chapter 9), not a sub-custody partition of owned mass.

7.7 The pricing-stack requirements

The valuation discipline above forces a small set of requirements on the pricing stack and the pricing data layer. They are derived here as consequences of NAV being a projection; Chapter 16 restates them as the minima any implementation must meet.

P1. **The ledger records pricing outputs and never runs a model.** Model outputs re-enter only as observations carrying sufficient provenance; reproducing a valuation from recorded prices is unconditional, while reproducing a model's number requires the model itself, whose retention is a governance matter outside scope.

P2. **Valuation is state-parameterised, and the state comes from the ledger.** $P(u, \sigma(u))$ reads $\sigma(u)$ from UnitStatus and PositionState, never from a pricing-owned store — the one-touch and the variance swap both turn on this.

P3. **Valuations are reproducible from recorded inputs.** The composition is a fold of the log and the prices are identified on the record, so any party recomputes any NAV to the minor unit.

P4. **Prices and quantities are never mixed across an event.** Each is taken in the same state of the instrument — consistency of reference.

P5. **Two price vectors are both projections over one composition.** Mark-to-market and mark-to-mid share the single position record; positions never diverge.

P6. **Estimates are kept apart from entitlements**, and originals are preserved while adjusted and derived values are computed at read (Chapter 8).

Each is a property that can be checked mechanically over generated products and histories, and each is exercised by a thread episode in this chapter. Valuation adds no store to the ledger: it is the log, folded and priced.

8 Ingestion, Corporate Actions, and the Market Data Operator

Governed by C-9.1–9.3, C-4.12 (§16.5).

§9 governs the one place the closed system is not closed: the boundary where external data crosses in. Everything inside the ledger conserves quantity by construction (Chapter 3); value, by contrast, is supplied from outside — prices, dividend figures, index levels, fixings, corporate-action notices, stamped valuations. The conservation properties hold only because the system is closed, so the boundary is where they are won or lost. Data admitted without provenance, a corporate action that silently re-reads old data in a new frame, a fallback taken without a record: each corrupts the closed-system property without violating any single internal invariant. This chapter holds the boundary watertight, and it does so with two devices — the recorded observation and the market data operator — and no third.

8.1 The boundary, and corporate actions on it

Two things cross the boundary, and they are recorded the same way. *Market and reference data* — a price, a dividend figure, an index level, a fixing, a stamped NAV — crosses as a recorded observation. A *corporate action* — a split, a dividend, a spin-off, a merger — crosses first as a recorded notice and is then, on its effective date, a transaction like any other: a bundle of moves, unit-state changes, and a new version of the instrument's terms, applied atomically through the one Transaction Executor (Chapter 4). A corporate action is not a privileged operation on a store; it is admitted at the same door as a trade, and it writes ProductTerms, UnitStatus, and balances through the same four-kind transaction (Chapter 3).

A corporate action does one thing no trade does: it changes the meaning of all data recorded before it. After ACME's two-for-one split, the recorded close 100.00 still names the same economic fact, but a post-split reader who multiplies it by the post-split count 20,000 books a phantom. To carry pre-event data into the post-event frame a transformation must be applied, and the framework gives that transformation a name: the **market data operator**. Three principles govern it, taken from §9 and used throughout this chapter:

(i) the operator is *intrinsic to the pairing* of a kind of data with a kind of event, declared once when the kind of data is introduced and the corporate action is recorded, never improvised at the moment of reading;

(ii) the *ledger is the authority* and the pricing data layer merely applies: the operator is declared data on the record, and the pricing data layer computes with it, never around it;

(iii) *originals are never overwritten.* The observed data stays on the record exactly as observed; the adjusted value is computed at each read; derived quantities are recomputed from operator-adjusted inputs — until fresh post-event data is recorded, after which the operator has nothing left to adjust.

8.2 The observation door is the one door

No data enters the ledger as a bare read, and no data enters through a door of its own. All data crosses as a recorded **observation** carrying its provenance — source, observation time, basis frame. An untrusted ingestion contract *proposes* a moveless **observation-recording transaction** — empty move list, folding the observation into a home (Chapter 6) — and the one Transaction Executor admits or refuses it like any transaction. *The record is the log*: no separate observation store, and the single-writer discipline of §5 reaches every observation. This is the door Chapters 7 and 10 name when a mark "enters through the observation door." A bare read against a live feed — a pull that records nothing — is refused outright: nothing arrived, so there is nothing to quarantine, and an unreproducible read must not exist. A *pushed* arrival is the opposite case: it is captured under its envelope even when malformed (W4, Section 8.9). A recorded observation is a fact of the log, so every downstream number is a function of the record alone (§12).

```
data Provenance  = Provenance { source :: SourceId, observed :: Time, basis ::
    BasisRef }
data Observation = Observation { reading :: Reading, prov :: Provenance }
-- all data crosses as a recorded Observation; there is no bare read, and no
    second door.
-- an untrusted ingestion contract PROPOSES a moveless observation-recording
    transaction;
-- the one Transaction Executor admits or refuses it, like any transaction.
-- admission writes a moveless Transaction (empty move list) that folds the
    Observation
-- into a home; refusal is a returned value, never a partial write:
data Ingest      = Proposed Transaction   -- moveless: folds the Observation into
    a home
```

```
  | Refused  IngestError
```

The provenance frame is what makes "prices and quantities are never mixed across an event" (§8) enforceable rather than hoped for: the frame the data was observed in is on the record beside the data, so a read that would combine it with a quantity in a different frame is detectable, not silent. That detection is the invariance witness of Section 8.7.

Episode — the variance swap (T5): the fixing source is the record. *Trigger.* VS-1's scheduled fixing dates arrive; each is a watch on the recorded IDX official close (Chapter 3). *The door.* The fixing is not a second feed; it is a read of the same recorded official-close observation that FUT-IDX settles against. The three settlement days of the future — IDX closes 1,000.00, 990.00, 1,005.00 — are the closes $C_{124}, C_{125}, C_{126}$ read at fixings 124, 125, and 126 of the swap's 252-fixing series, each fixing being the log-return over the prior close (fixing 125 reads $\ln(990/1000)$, fixing 126 reads $\ln(1005/990)$). *Design point. One recorded observation source feeds two consumers; because the source is on the record, the fixing series is deterministic and replayable at the data boundary, and firing* 127 *recomputes the same statistic any party would (Chapter 7).*

Episode — the total return swap (T6): the bridge is an observation. *Trigger.* TRS-1's first quarterly reset arrives; virtual ledger V-1's replayable level is stamped at USD 10,300,000. *The door.* That level crosses as an observation with provenance — source the fold of V-1's own log, observed at the reset date, in the reference-currency frame — and is recorded. No move crosses: a move from a wallet of V-1 to a wallet of the true ledger has no paired leg that conserves in either ledger, so neither Transaction Executor can admit it (Chapter 10). *The door checks.* Provenance present; the crossing is a recorded observation, not a move. *Design point. The bridge between the virtual ledger and the true ledger is an observation, never a move; the reset and financing moves it triggers are the true ledger's own (Chapter 10).*

8.3 The data-kind registry

Market data is not one number per name. A dividend forecast is a composite: cash entries that carry a price dimension and scale under a split, and proportional entries that carry no dimension and stand. An operator that treated the data as a single scalar would either scale the proportional part, which is wrong, or leave the cash part, which is also wrong. The operator must therefore know the *dimension of each component*, and it learns it from the **data-kind registry**.

A kind is registered once, before any data of that kind crosses, with a component-wise declaration: each component's dimension — price-in-basis, proportional, or quantity — and the quotation convention it renders. Registration is immutable: a correction is a new kind plus a retirement of the old, never an edit, so no data ever changes dimension under a reader's feet. Data of an unregistered kind is not silently framed: the arrival is quarantined, its processing refused (W4, Section 8.9), never dropped.

The registry has a second column under the same immutable discipline: the **event kind** — the kind of thing that fires a contract: a split, a fixing, a fill, an external fail notice. The kinds of event the system can process form a finite, closed, declared set (constitution C-4.12); a kind is registered, on the record, before any event of it crosses, and registration names its **routing** — the contract or contracts that must fire on it, total at registration: a kind cannot register without a router. "Fires the responsible contract" (Chapter 4) is thereby a declared fact, not an assumption. One cause may route to many units — one ACME dividend, one firing per referencing unit — separated by the cause-derived identifier. The worked instance of a routed lifecycle event is `COUNTERPARTY-DEFAULT`: its router names the master agreement unit's own smart contract, one firing per master referencing the defaulted counterparty, the members under that master separated by the same cause-derived identifier (Chapter 9). Closed

and declared, the event-kind set is the alphabet of the paths that could have occurred: what makes the generated-event universe and the simulation sample space well defined (C-2.8).

```
data Dim      = Price | Proportional | Quantity          -- component dimensions
data Reading  = Reading { kind :: KindId, comps :: [(Dim, Integer)] }  -- exact
    minor units
data IngestError = UnknownUnit | UndeclaredConvention | UnregisteredKind
data Routing  = Routing { eventKind :: EventKind, fires :: NonEmpty ContractId }
-- second column: every registered event kind names the contract(s) that must fire
    ;
-- a kind cannot register without a router --- routing is total at registration.
-- the market data operator acts on Price components; Proportional stands;
    Quantity is
-- walled off to the move plane (Chapter 3) and is never carried on the value
    frame.
```

The wall between `Quantity` and the *market data* operator is the same wall as the move plane. A share count is not adjusted by the market data operator: the two-for-one split moves 10,000 ACME to 20,000 by a paired-leg move from the issuer's virtual wallet (Chapter 3), because quantity is conserved and only a move may change it. The market data operator acts on value-frame components only. This division is what makes the split's arithmetic come out invariant, and it is stated as the witness next. (One operator does carry a `Quantity`-dimensioned thing — the term-adjustment operator acting on a contract's multiplier, Section 8.6 — and there the wall is not breached, because a multiplier is a term, not mass.)

The registry is a named trust assumption, and it is the only one the operators rest on.

> **TA-KIND.** A kind declaration faithfully renders the publisher's quotation convention — which components are cash and which are proportional, in which basis. *Owner:* the party that governs kind registration (data governance). *Violation:* an operator scales a component it should not, or spares one it should scale, misvaluing across every subsequent corporate action. *Detection:* the invariance witness at the first crossing of a corporate action (Section 8.7), divergence against an independent source (failure mode W3 below), and §13 recomputation. *Residual:* a misdeclaration that never meets a corporate action is invisible from inside the boundary — a reconciliation matter at the perimeter, the exact sibling of the collateral regime bit of Chapter 9.

8.4 The market data operator and adjustment-schedule totality

An operator is a function of the registered kind and the event kind, declared once and recorded. It carries value-frame data from the pre-event basis to the post-event basis; it is the identity on proportional components and undefined on quantity components, which never reach it.

```
-- one operator per (event kind, data kind), declared once, never improvised at
    read:
operator :: EventKind -> KindId -> (BasisFrame -> BasisFrame)
```

The first mechanism is that this declaration is *total*.

Principle 8.1 (Adjustment-schedule totality)**.** *For every registered data kind and every event kind the boundary admits, an operator is declared — the identity operator where a kind is genuinely unaffected, but declared, not assumed. There is no (data kind, event kind) pair for which a read finds the adjustment undefined. A corporate action whose pairing has no declared operator is not processed: the arrival is recorded as a quarantined event and its adjustment refused (W4), never silently treated as identity.*

An improvised adjustment is precisely a pairing with no declared operator; forbidding the improvisation means requiring the declaration in advance, for the whole registered domain. The

refusal of an undeclared pairing is failure mode W4 (Section 8.9), and it fails closed, as §2's Correctness commitment demands: the system stops rather than guesses a frame.

Operators compose. Two corporate actions in sequence adjust data by the composition of their operators, and the composition is taken *in log order*, because the operators do not commute: a split then a special dividend is not the same adjustment as the dividend then the split. The log fixes the order, so the composed adjustment is well defined and replayable.

8.5 The operator menu: the split

The split shows the whole menu at once. On 2026-06-01 ACME's two-for-one split is effective; the recorded split notice had declared the effective-date watch, the Event Monitor emits the event, and one transaction carries the move and the new terms version (Chapters 3 and 6). What each recorded data kind reads as, in the new frame, is the operator's work:

Data kind	Operator for the 2-for-1 split	Read after the event
Official close (spot)	scale by $\frac{1}{2}$	$100.00 \to 50.00$
Cash-per-share dividend figure	scale by $\frac{1}{2}$	$1.50 \to 0.75$
Proportional (percentage) figure	identity — stands	$1.00\% \to 1.00\%$
Index divisor (a constituent splits)	adjusted to hold the level	level continuous

The share count is absent from the menu on purpose: $10{,}000 \to 20{,}000$ is a move, not an operator (Section 8.3). The spot and the cash-per-share figure are price-dimensioned and halve; the percentage figure is proportional and stands; the index divisor is adjusted so the published level does not jump when a constituent splits. Every original stays on the record; each "read after" is computed at the read, until a fresh post-split close prints and the operator retires for that data.

Episode — the dividend figure under the split (T3×T4). *Trigger.* On 2026-05-04 ACME's cash dividend of USD 1.50 per share is announced; the announcement is captured at the boundary as a recorded observation, and its declared per-share figure 1.50 enters the record (the three firings that pay it are Chapter 5's). *The operator.* The split is effective 2026-06-01, after the payment date but within the window in which the declared forward figure is still read; a read of the 1.50 figure in the post-split frame applies the split's cash-per-share operator and returns 0.75. *The door checks.* The data kind is registered as a cash-per-share (price) component, so the operator scales it; a proportional figure beside it would have stood. *Design point. One operator, one invariant, visible: the same halving that takes the spot from* 100.00 *to* 50.00 *takes the recorded per-share dividend figure from* 1.50 *to* 0.75*, so the dividend history reads in current-share terms — the scaling of a real figure is shown, not asserted.*

8.6 The term-adjustment operator: a derivative's own terms

The menu above adjusts *value-frame* data — a spot, a dividend figure, an index level. A unit whose terms *reference* the split stock has a second frame the same event must adjust: its own **declared terms**. OT-1's barrier of 80.00 names OMEGA's official close; when OMEGA splits two-for-one every OMEGA price halves, and a barrier left at 80.00 knocks on the first post-split close near 47.50 — a stock at half its former level meeting a barrier struck against the whole. The barrier is a declared term, not data, so no operator of the menu touches it. The framework gives the missing transformation the exact sibling of the market data operator: the **term-adjustment operator**. Where the market data operator carries *data* from a pre-event basis to a post-event basis, the term-adjustment operator carries a *declared term* across the same event; it is the action of the corporate-action out-edge every referencing unit declares (Chapter 3, Section 3.8), which appends a new ProductTerms version with each referencing term carried through its operator.

```
-- the sibling of  operator  (Section 8.4), acting on the declared-term frame, not
     value:
termOp :: EventKind -> TermKind -> (Term -> Term)   -- one per (event kind, term
    kind)
```

It is total in exactly the sense of Principle 8.1: for every (declared-term kind, event kind) pair an operator is declared — the identity where a term is genuinely unaffected, but declared, not assumed — and a pairing with no declared operator refuses the event at the door. Totality here is what forbids a silent stale barrier: an undeclared (term kind, event kind) pairing cannot cross, so a split can never leave a referencing term unadjusted by default.

The operator reuses the dimension machinery of the registry (Section 8.3): a term carries a dimension, and the dimension fixes how the event moves it.

Declared term	Dimension	Operator for the 2-for-1 split	Read after
Barrier (OT-1 on OMEGA)	`Price`	scale by $\frac{1}{2}$	$80.00 \to 40.00$
Strike (an OMEGA option)	`Price`	scale by $\frac{1}{2}$	$K \to K/2$
Multiplier (a single-name contract on OMEGA)	`Quantity`	scale by 2 — *doubles*	$100 \to 200$
Participation rate	`Proportional`	identity — stands	$150\% \to 150\%$
Fixed cash payout (OT-1)	declared absolute	identity — stands	USD 1,000,000 unchanged

Note the direction. A `Price`-dimensioned term — a barrier, a strike — halves, exactly as the spot halves, because it is quoted in the units the price is quoted in. A `Quantity`-dimensioned term — the multiplier of a single-name contract on the split stock — *doubles*, because the split doubles the number of underlying shares one contract commands and the payoff (level $- K$) $\times$ multiplier must stand: $(S - K) \times 100 = (\frac{S}{2} - \frac{K}{2}) \times 200$. A `Proportional` term — a participation rate — stands. And a fixed cash payout is neither `Price` nor `Proportional` but a *declared absolute*: it is registered as such, and its split operator is the identity — USD 1,000,000 is USD 1,000,000 whether OMEGA is whole or halved.

The multiplier is the one place a `Quantity`-dimensioned thing is carried by an operator. On the balance plane the wall of Section 8.3 is absolute: only a move changes a quantity, and no market data operator touches a share count. The term-adjustment operator carries a `Quantity`-dimensioned multiplier and doubles it, and this does not breach that wall, because *a multiplier is a term, not mass.* It names how many underlying units one contract controls; adjusting it re-describes the same contract in the post-split frame and moves no holding of anything. The share count the split really moves — 10,000 OMEGA to 20,000 — still travels by a paired-leg move and by nothing else (Section 8.3). Mass is carried by moves; a term is carried by its operator; the value frame and the balance plane never touch.

Declared identity is still a declaration: VS-1 and FUT-IDX on the index. Not every referencing unit adjusts under every event, but totality requires the non-adjustment be *declared.* VS-1's 252 fixings read the 253 IDX official closes $C_0, \ldots, C_{252}$, and FUT-IDX settles the IDX level at multiplier 10; both reference IDX, so both declare a corporate-action out-edge. When a *constituent* of IDX splits, the index divisor is adjusted to hold the published level continuous (the menu's divisor row above), so the level VS-1 and FUT-IDX read does not jump. Their term-adjustment operator for the (IDX-constituent split, fixing) and (IDX-constituent split, multiplier) pairings is therefore the *identity* — but declared, per Principle 8.1, not assumed: FUT-IDX's multiplier 10 stands because the level it multiplies is held continuous by the divisor, and VS-1's fixing reads the same continuous level. The contrast with the single-name multiplier is the whole point: a contract on OMEGA *itself* doubles its multiplier when OMEGA splits, because OMEGA's price halves; a contract on the IDX *level* leaves its multiplier alone when

a constituent splits, because the divisor already held the level. Both are declared operators; neither is silent.

8.7 State-aware reads: consistency of reference and cum/ex

Every operator read is state-aware: the frame data reads in is determined by the corporate-action state of its underlying at the read, and that state is on the record. Two readings of one recorded observation in two frames are two different values, and combining a quantity with a price drawn from a different frame produces a phantom. The check that refuses phantoms is the **invariance witness**: the operator on the value frame and the move on the quantity are bound so their product — total value — is invariant across the event.

For the split the witness is arithmetic. Before: $10{,}000 \times 100.00 =$ USD 1,000,000. After, read in the post-split frame: $20{,}000 \times 50.00 =$ USD 1,000,000. The phantom the witness refuses is $20{,}000 \times 100.00 =$ USD 2,000,000, the new count against the stale frame. This is the consistency-of-reference invariant; it is stated formally and catalogued in Chapter 14, and used here.

For a derivative the witness has no move to bind, and it is stated over the declared terms instead. OT-1 references OMEGA but carries no OMEGA quantity: its holder holds one option before OMEGA's split and one after, so no move fires. The witness is then that the *payoff* on the post-event term frame equals the payoff on the pre-event term frame. Under OMEGA's two-for-one split the term-adjustment operator (Section 8.6) carries OT-1's barrier $80.00 \to 40.00$ and leaves its USD 1,000,000 payout unchanged — a declared absolute stands. The witness is the knock predicate read in the new frame: a pre-split close at or below 80.00 pays USD 1,000,000, and a post-split close at or below 40.00 pays the same USD 1,000,000 — one economic event, two frames, one payout. The phantom the witness refuses is the dropped-edge failure of Chapter 3 (Section 3.8): the first post-split close prints ≈ 47.50; against the stale barrier 80.00 it reads $47.50 \leq 80.00$ and knocks, paying USD 1,000,000 on a stock that never fell; against the adjusted barrier 40.00 it reads $47.50 > 40.00$ and no knock fires. The stale-frame read is the phantom; the term-adjustment operator, appended as the corporate-action edge's new terms version, is what refuses it. Quantity-times-price for a holding and payoff-on-terms for a derivative are the two readings of one witness.

The same witness governs the cum/ex boundary of a distribution, where no quantity changes at all. Before the ex date a price is *cum* — it includes the coming distribution; on and after the ex date it is *ex* — it excludes it. The cum price and the ex price are one data kind read in two states separated by the ex-date event, whose declared operator subtracts the distribution's value. A cum price consumed as though it were ex, or an ex price consumed as though it were cum, is a phantom of exactly the split's kind — the right number in the wrong frame — and the same invariance witness refuses it, because the state that fixes the frame is the recorded ex-date event, not the wall clock. A read that ignores the ex-date state prices the instrument on the wrong side of its own corporate action, and the witness catches it before the number reaches a valuation.

One thing the ex-date does not carry is a promise about *when* an ex trade settles relative to the record date. That the ex-date is set so an ex trade settles only after the record date — keeping the registered holder and the entitled party the same party — is an external market convention, fixed by the corporate-action calendar and the settlement cycle, and it is nowhere on the ledger's record. The ledger records the ex-date as data with provenance and routes entitlement against it (Chapter 11); it does not enforce the convention that keeps registration and entitlement aligned. That convention breaks lawfully under a due-bills special dividend, whose ex-date falls after the payment date and lets an ex trade settle before the record date. The coherence is named as the trust assumption TA-EXDATE in Chapter 16, reconciled against at this boundary but not enforced here; the settlement interface handles the case it breaks with the mirror market-claim, so correctness does not rest on the convention holding.

8.8 Recompose and the aggregate weld

The split adjusts one unit's data in place. Two further mechanisms handle corporate actions that change which units exist: one splits a unit into several, the other merges several into one. Both preserve total value, and both are checked by the same witness.

Recompose (one unit becomes many): a spin-off. When an issuer distributes a new entity to its holders, one position becomes a basket, and the pre-event data of the parent must be carried onto the resulting units. *Recompose* is the operator that does so, per a declared allocation basis recorded with the event. Illustratively: HELIOS trades at 120.00 and spins off LUMEN, one LUMEN share per HELIOS share, with a declared allocation of 75% of pre-event value to the HELIOS stub and 25% to LUMEN. A holder of 1,000 HELIOS holds, after the event, 1,000 HELIOS and 1,000 LUMEN — the new LUMEN mass arriving by a paired-leg move from the issuer, not by an operator. Recompose carries the recorded pre-event close onto the two frames: HELIOS stub 90.00, LUMEN 30.00. The witness holds: $1{,}000 \times 120.00 = 1{,}000 \times 90.00 + 1{,}000 \times 30.00 =$ USD 120,000. The original 120.00 is preserved; the stub and spun-off frames are computed at read until fresh post-event closes print for each.

The aggregate weld (many units become one): an elective merger. When one unit is absorbed into another, the resulting position welds into the acquirer's aggregate, and the two data histories must weld into one continuous series so that valuation and profit and loss run unbroken across the event. The *aggregate weld* is the operator that carries the pre-event data of the absorbed unit onto the acquirer's frame, per the elected terms recorded at the boundary. An elective merger is captured as an observation — the election result is data with provenance, like any other. Illustratively: TARGET is acquired by ACQUIRER, each TARGET share electing either USD 50.00 cash or 0.4 ACQUIRER shares; ACQUIRER trades at 125.00. A holder of 1,000 TARGET who elects stock welds to 400 ACQUIRER, and the pre-merger TARGET series is welded onto the ACQUIRER frame at the elected ratio 0.4; the witness holds, $1{,}000 \times 50.00 = 400 \times 125.00 =$ USD 50,000. A holder who elects cash has the position retired at settlement and no weld occurs — the elected terms, recorded data, decide which mechanism runs.

Two further composite distributions — a dividend forecast and a special dividend — are governed by the same registry and the same operators: the forecast is a composite of cash and proportional components that the split scales component-wise, and the special dividend carries a cum/ex operator like the ordinary one. Their fully worked arithmetic is deferred to the Worked-Examples companion (register lines E71 and E72); the mechanism here is complete without them.

8.9 Failure modes at the boundary

The boundary has four failure modes W1–W4, a closed set distinct from the three collateral inflow *regimes* of Chapter 9: two closed enumerations, two names, never one word for both. Each mode has a determined, replayable disposition — a function of log state alone, so replay reproduces it bit-identically. None improvises; each fails closed or records a flag, never a silent pick.

	Boundary condition	Determined, replayable disposition
W1	missing data	the operator applies to the last recorded original, carrying a staleness flag; with no original, the price is undefined, not zero, and valuation over the unit defers (Chapter 7)
W2	late data	recorded at its observation-time coordinate; values derived over $[\text{observed}, \text{arrival})$ are recomputed — "as known at t" and "with corrections through t'" are both queryable
W3	contradictory sources	sources diverging beyond the declared threshold yield a flagged "aggregation failed" observation, never a silent pick; sources and threshold are recorded
W4	unroutable or unmappable event	the arrival is recorded as a **quarantined event**, a moveless provenance-carrying fact, and its *processing* is refused — the refusal on the record; fail-closed, nothing lost

W1 is the working state of §9's "until fresh post-event data is recorded": between the event and the first fresh observation, every read is the original under its operator, flagged stale, and the flag is a fact of the record, not a property of a cache. W2 makes the boundary bitemporal, and it is the machinery Chapter 2 (Section 2.5) and Theorem 14.5 rest on. Every observation carries two coordinates: its *execution-time* coordinate — its observation time, or the fixing index k it belongs to — and its *door-time* coordinate — the log position at which it was recorded. Data arriving late does not rewrite history: it is appended at its own door-time position while keeping its earlier execution-time coordinate, so the as-known-at view — the projection over any prefix that predates the arrival — is preserved, and the as-of view — the projection over the prefix that now carries the correction — recomputes the derived folds over the affected execution-time interval, bit-exactly (§2). Both views survive; neither overwrites the other. An amended or withdrawn corporate-action notice is this same case — superseding data recorded at its own door-time position, never an edit of the original notice, so the pre-amendment frame and the corrected one both survive. C-12.1 (v1.3) requires both honest answers — as known at t, and with corrections through t' — and delegates their indexing to this specification; W2 is where the two-axis indexing discharges that duty (entry PARK-2 closed, Chapter 17). W3 is the multi-source case: a single source is a single point of failure, so where data materially affects valuation it is drawn from more than one attested source, aggregated by a declared rule, and a divergence beyond the declared threshold produces a flagged output rather than a silently chosen one. The divergence threshold is a governance parameter, recorded and calibrated, not a silent constant: under correlated stress — every venue diverging at once — it defers valuation book-wide rather than choose a number, an intended fail-closed whose calibration is a governance matter, not a hidden behaviour. W4 is totality enforced without loss. An event of an undeclared kind, an unresolvable reference, a pairing with no declared operator (Principle 8.1), or a payload that fails validation — provenance incomplete included — is neither silently dropped nor silently guessed: it crosses the one door as a quarantined event — a moveless recorded fact carrying at least its capture envelope (Chapter 4), and the payload's own provenance where it has one — and its *processing* is refused, the refusal itself a fact of the record. The system still stops rather than improvises (§2); it no longer forgets what it stopped on. The quarantined event mints a visible, deadline-bearing open item and stands until its kind is registered or a person disposes of it (C-12.4 phase-0, constitution C-4.12). The guarantee runs from arrival: whether the world's events reach the boundary at all is the perimeter trust assumption TA-ARRIVAL (Chapter 16), reconciled against external authorities, never assumed.

When the kind is later registered, or a person disposes, the arrival is processed then: the processing transaction's cause is the recorded quarantined event — idempotent under the cause-derived identifier, carrying the quarantined event's own execution-time coordinate (W2 above). The registry is versioned declared data read in force at each log position, so replay across its

evolution is deterministic: every reader computes one answer.

8.10 The boundary-convention slot

One class of boundary fact is neither data nor an operator: a *convention* that some downstream projection needs and that must be stated once, as data, so that two readers of one log compute one answer. §9 places such conventions on the record beside the operators; this section declares the **slot** they occupy, and Chapter 12 fills and consumes one.

A convention in the slot is a total function of log state. The convention Chapter 12 consumes — the attribution of commingled received value across a reinvestment pool — has three components, and the slot's shape is exactly these three:

```
data Convention = Convention
  { pool     :: Predicate (Wallet, Move)   -- the pool over which value is
    attributed
  , basis    :: PeriodBasis                -- when the pro-rata ratio is struck
  , rounding :: RoundingRule }             -- remainders booked explicitly (
    Chapter 3)
attribute :: Convention -> LogPrefix -> Partition Amount   -- consumed in Chapter
    12
```

The slot declaration is itself a recorded event, so it versions with the log prefix: a report computed as of a past date uses the convention then in force, and a changed convention is a new recorded version, never an edit. Totality is the requirement that makes the slot safe to consume: for every log prefix the convention assigns every attributable quantity, the attributed parts and the explicit remainder partition the whole, and the rounding rule leaves no quantity silently dropped (§4's remainder discipline). A convention missing any component is refused, because two readers would then compute different figures from one log — internal reconciliation failure reintroduced through a report. Chapter 12 states the totality property and proves the figures it builds are projections of coordinates, units, and this declared convention alone; this chapter's duty is discharged by declaring the slot and its totality obligation.

8.11 What an auditor checks

A candidate implementation satisfies this chapter when seven things hold, each mechanically checkable over generated data and histories: all data on the record carries provenance, and no valuation reads data that does not (Section 8.2); every data kind is registered with component dimensions before it crosses, and an unregistered kind is refused (Section 8.3); an operator is declared for every (registered kind, event kind) pair, and an undeclared pairing is refused, not defaulted to identity (Principle 8.1); every original is preserved and every adjusted value is recomputed at read from the original under the declared operators (Sections 8.5–8.8); the invariance witness holds across every generated corporate action, and a stale-frame or wrong-side-of-ex read is refused (Section 8.7); each of W1–W4 produces its determined disposition on replay (Section 8.9); and every unroutable or unmappable arrival is recorded as a quarantined event, never dropped, and re-processes deterministically once its kind is registered (Sections 8.3, 8.9). These restate as minima in Chapter 16.

9 Collateral, Margin, and Lending

Governed by C-4.10–4.11, C-8.6–8.9, C-6.5 (§16.5).

9.1 One representation, keyed by declared regime

One representation serves every collateral, margin, and lending arrangement the ledger records. Every (wallet, unit) balance is the signed coordinate vector (owned, lent, posted) of Chapter 3; the outright holding is the degenerate case with all mass on owned; and any unit — cash, a bond, a structured note, a listed option — may sit on the non-owned coordinates. Which coordinate an inflow writes is decided by the legal regime declared once on the governing agreement unit, never by the asset class of what moves. Title transfer writes owned, plus a claim and an obligation that are themselves units, and touches no marker coordinate; a security interest writes marker mass on the posted plane and never touches owned; settled-to-market margin writes owned with no obligation at all.

The agreement is a unit — constitution §6: every right and obligation the legal contract creates is a unit. Its declared terms carry the regime (settlement, financing, or custody); the eligibility schedule and the per-line valuation percentages; the thresholds, deadlines, and close-out mechanics of its margining; the trapping conditions its distributions carry; and its watches — the valuation watch, the income watches — declared at registration like any other unit's. The agreement unit itself is non-valued: the claims and obligations it creates carry the value, and pricing the governing shell beside them would count the same economic reality twice (§4).

A claim or an obligation is a valued unit held positive by the obligee and negative by the obligor: one unit, one fact, both sides, and conservation makes disagreement between them unrepresentable. An obligation carries, as declared terms, a deadline, a discharge predicate, and a fallback (§14.1); by its deadline it is discharged, compensated, or declared defaulted — a duty that fails to fire is a visible, overdue item, never silence — and, like every unit, it retires only from the zero vector.

Eligibility and haircuts are declared data, checked at the door — the Transaction Executor's admission check — and never typed. Collateral is a role a balance plays under an agreement, not a kind of thing: the type system knows planes and agreement references; what may be posted, and at what valuation percentage, is a schedule in the agreement's terms. Admitting a new eligible asset class is a terms version, not a code change; an ineligible posting is refused at the door. The declared eligibility predicate may itself read recorded ledger state: a concentration limit makes eligibility portfolio-dependent, admitting a posting only while the resulting holding stays within the declared bound — read at the door exactly as the coverage check reads state, and never typed. What the type system does enforce is stronger than any schedule: a move writes exactly one coordinate of one unit, positive quantity, the same coordinate debited at the source and credited at the destination, and a move whose coordinate is not owned carries a mandatory reference to its governing agreement. Orphan collateral mass is not refused; it is unwritable.

```
newtype AgreementRef = AgreementRef UnitId  -- the governing agreement, itself a
    unit
data Plane = Lent | Posted                  -- the marker planes

data Placement                              -- where a move writes
  = OnOwned                                 -- ownership moves; no agreement
    needed
  | OnMarker Plane AgreementRef             -- marker mass: unwritable without its
     agreement

data Regime = Settlement | Financing | Custody  -- declared once, on the agreement
     unit
```

The `AgreementRef` that every `OnMarker` leg carries is exactly the agreement index G of the balance vector $\mathrm{bal}_{c,G}$ (Section 3.7): the coordinate a marker move writes is the pair (plane, agreement), never the plane alone, so a wallet's mass posted under G_1 and its mass received

under G_2 are two distinct coordinates that never cancel.

Everything computable from the coordinates — available quantity, encumbered quantity, mass in possession, mass re-used — is a projection, computed when needed and never stored (§4). Chapter 12 builds the reporting projections on exactly this surface.

9.2 The two guards

Two guards govern the marker planes, and they are never collapsed into one word. Both are *used* throughout this chapter; their formal statements and proof obligations are catalogued in Chapter 14.

Coverage is an invariant. No wallet delivers what it neither owns nor holds. It is stated once, as Invariant 14.8: checked at the door on every admission, per wallet and unit, the net over agreements G of the signed (posted, G) coordinates of Section 3.7 is bounded by possession, with the same bound for lent. The sign convention is *received-negative* — collateral posted out enters positive, collateral received in enters negative — so mass received *under a right-of-use agreement* offsets mass re-posted, and re-use of such collateral is bounded by possession, while received mass a segregation withholds from use grants no offset (the right-of-use gate, Chapter 14); the -40 chain, the refused segregated re-post, and the refused over-re-post are worked once under that invariant. The signed net is the right quantity *here*, where the question is whether the wallet can deliver; it is the wrong quantity for the encumbrance report, where the question is how much is encumbered, which reads the per-agreement coordinates gross (Chapter 12). The two never share a formula: coverage nets, encumbrance does not.

Sufficiency is an obligation. Collateral value at or above exposure is lawfully false intraday: a market move, a knock, a call in flight all leave it false with no party in breach. It is therefore an obligation, never an invariant: a deadline, a discharge predicate (the delivery amount met), and close-out as the declared compensation. A book below its sufficiency line is not a broken state; it is an open, deadline-bearing item on the record, and the deadline covers it.

9.3 Title transfer: owned re-books; the claim is a unit

Owned must be the coordinate the move machinery can act on. The taker of title-transfer collateral may lawfully sell it outright; a design that keeps the asset on the poster's owned makes that lawful sale a special constructor and makes the manufactured payment — the income the taker owes back to the poster — unrepresentable. So the poster's owned re-books to the taker, and what the poster keeps is what the poster legally has: in the same transaction the poster gains a **claim-for-equivalent** unit, and the taker's negative balance in that unit is the redelivery (or repurchase) obligation. Three rules complete the shape; each is a correctness requirement.

Pricing. The claim is priced through the ordinary pricing layer, parameterised — as valuation under §8 already is — by state: here the taker's. While the taker performs, the claim tracks the referenced asset (the accrued redelivery value) and bears the taker's credit; on the taker's default it decouples from the referenced asset and becomes a recovery claim on the taker's estate. It is not "priced identically to the asset": that rule would mark the poster long at par on the one day the mark exists for.

Identity. Claims are per-agreement units — per netting set. Close-out nets within a master agreement and never across masters, and because owned moves carry no agreement reference, the claim's netting-set identity cannot live on a move: it lives in the unit's own declared terms, which name both the referenced asset and the governing agreement. The unit registry grows by one claim per financing transaction. Accepted: fungible claims would break netting-set attribution at close-out, which is the one computation the claims exist to get right — and the counterparty-state pricing above is then natural, because the claim is a per-counterparty object.

Timing. The booking moment is itself a declared term of the governing agreement, not a constant of the ledger. Trade-date booking — owned re-books at instruction — is the default

this specification carries, and the instruction-to-settlement gap is carried explicitly, never elided. Where an agreement instead declares settlement-date booking, owned re-books only on the recorded settlement-confirmation event, so a bought-not-yet-settled position is not deliverable — its owned balance does not yet exist. Settlement-date booking changes only *which event writes owned*; it does not remove the cum buyer's entitlement, so it does not remove the market claim. The claim is a function of the trade's cum/ex status and settlement-versus-record-date timing (Chapter 11), never of the booking policy: a cum trade unsettled at a record date still raises the settlement-obligation unit's market-claim leg under either booking rule, because the buyer bought cum and is entitled while the register shows the deliverer. The ledger records whichever event the declared term names; the meaning of the owned coordinate is fixed by the constitutional definition of the atomic move and is unchanged by which event writes it (Chapter 11). Under trade-date booking, where a record date falls inside the gap, the paying agent pays the record holder — still the deliverer — while the ledger's owned shows the buyer. The reconciling leg is the **market claim**, carried by the settlement-obligation unit (Chapter 11) at its `instructed` node: first-class, recorded, deadline-bearing — an instrument, not a break. The settlement-obligation unit's lifecycle node is thereby load-bearing for entitlement routing: a correctness requirement of this design, not a reporting axis. Chapter 11 carries the settlement-side machinery; the mechanism is stated here because the re-booking rule creates the gap it must close. Re-booking at instruction also lets a wallet deliver or post a bought-not-yet-settled position; should that purchase later fail, the shortfall is not an impossibility but a fails cascade the model carries — order-forced at the door (an owned reduction that would drop owned below the mass already posted is refused), then the unmet return as a recorded obligation, then, on default, the supervised write-off path of this chapter's close — never a silent breach. That fails obligation is not a fresh unit minted symmetric to another: it *is* the settlement-obligation unit of Chapter 11 — the very unit the instructed purchase created — walked from its `instructed` node to its `failed` node when the settlement-fail event fires (the reversal having been refused; the projection writes nothing). One product graph governs that unit: its `instructed`, `settled`, and `failed` nodes, together with the market-claim leg the record-date firing raises, are that one graph's nodes and edges, so the market claim, the fails cascade, and the ordinary settlement are three walks of a single unit, not three mechanisms. With its paired settlement — delivery-versus-payment — cash leg it offsets the bought-not-yet-settled position at fair value, so net owned value — and NAV — is preserved across a failed purchase, on the same declared-rate-equals-fair-rate basis as the §8 case-2 return obligation below.

Cash under title transfer is the same shape, by §4's own admission test: a quantity earns a coordinate only when a distinct real-world action can change that coordinate independently of ownership — and taking cash under title transfer *is* the change of ownership, so it earns no marker coordinate. The receiver owns the cash, reinvests it, bears the result, and owes an equivalent-return debt: §8's second case, the obligation itself a unit. A custody marker over commingled cash would assert a per-dollar fact no bank statement can confirm — the nostro observes only the sum — and its intraday divergence from the statement is precisely the internal reconciliation failure the ledger exists to abolish. The return obligation is valued at the inflow amount — par plus accrued at the declared rate — so net owned value cannot move at receipt: §8's deposit-neutrality sentence holds by construction, not by formula. Balance-sheet presentation, encumbrance, and re-use figures are projections over claims, regimes, and coordinates — Chapter 12; no second store.

9.4 Pledge: marker mass under the coverage bound

A pledge changes possession independently of ownership, so it writes the marker plane and only the marker plane: one move on the posted plane — poster $+Q$, taker $-Q$, the taker's negative balance projecting as received as collateral — admissible under the coverage invariant, with owned untouched. That is the legal fact: the pledgor still owns. Because posting never

writes owned, $V = \sum \text{owned} \cdot P$ is unchanged by any post-and-return round trip, and §4's "only the owned coordinate carries economic value" becomes a machine-checkable property: insert a post-and-return pair within any watch-free interval of the referenced agreement and every later valuation is identical. The side condition is real — an insertion spanning the agreement's valuation watch lawfully changes later firings and cash — and the property, with it, is catalogued in Chapter 14.

Re-use of pledge-form collateral is a posting of mass held on the received ray, within the same coverage bound. Every re-pledge move carries a source-agreement reference beside its governing reference — the source is an observable fact of the custody instruction, so nothing fictitious is stored — and re-use therefore projects as per-(unit, agreement) actuals in Chapter 12.

9.5 Cash margin: four cases, one machine

Cash margin is four cases, keyed by the declared terms of the agreement unit, and the enumeration is total: a margin flow whose agreement declares no regime has no admissible booking and is refused at the door — the system stops rather than improvises (§2).

The agreement declares	The inflow is	The transaction writes
Settled-to-market (STM) variation margin	settlement (§8 case 1)	owned; no obligation — the day's exposure is extinguished
Collateralised-to-market (CTM) or title-transfer credit support	financing (§8 case 2)	owned, plus a return-obligation unit accruing the declared rate
Security interest without right of use (segregated)	custody (§8 case 3)	posted-plane marker mass; owned untouched
Security interest with right of use	financing upon exercise of the right	owned, plus the return-obligation unit, upon exercise

The custody case raises no fungibility objection, because the segregated account reconciles one-for-one against the marker. For cash under a right of use, receipt into the receiver's own accounts is the exercise: commingled right-of-use cash books as financing, never as custody — the fourth row collapses into the second at the moment commingling makes the custody claim unverifiable.

Episode (future): the settled-to-market margin day. FUT-IDX is a cash-settled listed future on index IDX, multiplier 10, settled-to-market by the declared terms of the central counterparty (CCP) agreement unit; wallet W-ALPHA is long one contract, and day 1 settled at 1,000.00. *Trigger.* The day-2 official settlement print, 990.00, is recorded; the Event Monitor emits the settlement event. *The contract.* The future's contract fires, reading the last applied level, 1,000.00, from PositionState (Chapter 6). *The transactions.* One transaction:

(1) move owned(USD) $(1{,}000.00 - 990.00) \times 10 =$ USD 100, W-ALPHA $\to$ W-CCP;
(2) PositionState update: margin applied, settlement level 990.00.

No marker coordinate, no obligation unit: the payment settles the day's exposure, and on unwind nothing is returned, which is correct. W-ALPHA's net asset value falls by exactly 100, the day's loss. *The door.* Conservation per (unit, coordinate) on the paired-leg move; an owned move, so no agreement reference is required; idempotence under the cause-derived identifier — a duplicate settlement event proposes nothing new. *The contrast (CTM).* The episode is settled-to-market: the day's exposure is extinguished and nothing is owed back. Had the CCP agreement declared CTM, the identical cash move would be paired with a return-obligation unit — the returnable margin liability — valued at 100 and accruing the declared rate. That single unit is the whole difference: STM and CTM share one machine and diverge by one declared term

and one returnable-liability obligation unit, not by a different case. *The regime is a declared term of the agreement unit; the machine is the same.*

Episode: the segregated inflow and the refused re-post. Wallet W-SEG receives USD 100 of USD as variation margin under agreement G-SEG, whose declared regime is custody and whose declared terms grant *no* right of use — a security interest withholding use (§8 case 3, the third row above); the regime and the right-of-use term are two declared bits, not one. The inflow writes marker mass on the posted plane: W-SEG's posted coordinate under G-SEG is -100, received-negative, owned untouched. *The attempted re-post.* W-SEG proposes to re-pledge USD 60 under a second agreement G2, a move carrying G-SEG as its source-agreement reference (Chapter 12). *The door.* The coverage check reads G-SEG's declared right-of-use term through that reference. G-SEG grants none, so its received -100 contributes zero to the gated net (Invariant 14.8): the net is $60 + 0 = +60 > \max(0, 0) = 0$ and the re-post is *refused*. Nothing is written. Had the same USD 100 arrived under a right-of-use agreement, that -100 would count, the net would read $60 - 100 = -40 \leq 0$, and the identical re-pledge would be *admitted* within the coverage bound. One declared bit — the source agreement's right of use — decides, and no new unit or type marks the mass: collateral is a role, read at the door.

The regime is one declared bit carrying the largest balance-sheet consequence in this chapter, and its misdeclaration is a class of error the recomputation check of §13 cannot catch: recomputation reproduces the declared bit and faithfully reproduces the wrong answer. A margin agreement misdeclared as STM omits the returnable-margin liability and its interest accrual; the converse fabricates them. Detection is boundary reconciliation against the counterparty's or the clearing house's margin statement — nothing internal detects it, and the reconciliation is therefore a named duty of the boundary, not a redundancy. The repair is defined now, not improvised at the first discrepancy: a ProductTerms amendment — a new terms version declaring the true regime — *plus* authorised compensating transactions rebooking the obligation units and accruals of the affected interval. Both, not one: the amendment without the rebooking leaves the interval wrong; the rebooking without the amendment repairs history and re-breaks tomorrow.

The regime bit is the worked instance of a rule that governs every declared term of the agreement. The eligibility schedule, the valuation percentages, the thresholds and deadlines, and the trapping predicates are all **boundary data**, transcribed once from the executed agreement, which is their provenance. A transcription error in any of them is not an inconsistency the ledger can catch by recomputation — recomputation reproduces the transcribed term and faithfully reproduces the wrong answer — but a perimeter-reconciliation matter, the same class as the §13-invisible regime bit above and the market-data trust assumption of Chapter 8. It is detected by boundary reconciliation against the counterparty's own record of the agreement, and repaired by the same two-step path: a terms amendment declaring the corrected term *plus* authorised compensating transactions rebooking the affected interval, never a silent edit. The regime bit carries the largest consequence and earns the worked treatment; the duty is general over every declared term.

9.6 Line valuation

Collateral value under an agreement is the sum, over the lines the agreement holds, of market value times the declared valuation percentage of that line. The derivation is §6 faithfulness: the ledger values what the agreement declares, and the agreements declare lines — an eligibility schedule with a valuation percentage per line — not floored packages. An agreement whose declared terms define package-level valuation prices the package through the ordinary pricing layer, admissible as declared data like any other term; and a portfolio floor, where computed, is a projection — never a stored valuation, and never a haircut the agreement does not grant.

9.7 Entitlements: determination and payment

Entitlement *determination* always reads the owned plane. The instrument's contract is possession-blind and regime-blind: who is owed a dividend, a coupon, a payout is a question about ownership, and ownership lives on exactly one coordinate. Entitlement *payment* routes through a **conditional obligation unit** created by the determination, whose discharge predicate carries the governing agreement's declared trapping terms — the sufficiency and default conditions under which the agreement withholds a distribution. Where no agreement encumbers the owner's mass, the predicate is vacuously true and the obligation discharges at once: in the ordinary state the *condition* vanishes, never the *leg*. An unconditional issuer-to-owner pass-through is forbidden: the agreements trap distributions exactly when a delivery amount or a default stands, and §6 binds the ledger to the terms as declared. The predicate traps or releases the gross determined entitlement; where an agreement instead declares an *amount* transformation — a manufactured dividend net of withholding, or a contractually reduced manufactured rate — that amount is a declared term of the agreement, read like any other declared data, not a new mechanism.

Episode (dividend): the unencumbered case. W-ALPHA holds 10,000 ACME; the declared dividend is 1.50 per share; the record-date firing has already determined and recorded the entitlement, USD 15,000, in PositionState (Chapters 5 and 6). Nothing encumbers W-ALPHA's ACME. *Trigger.* The payment-date watch fires; the Event Monitor emits the event. *The contract.* The stock's contract, on its third firing, pays against the recorded entitlements. *The transactions.* One transaction:

(1) the conditional payment obligation is created against the recorded entitlement — ACME's issuer wallet owes W-ALPHA USD 15,000; its predicate carries the trapping terms of every agreement encumbering the holder's mass, and there are none, so it holds vacuously;
(2) move owned(USD) USD 15,000, ACME issuer wallet → W-ALPHA; the obligation is discharged and retires at the zero vector.

The door. Conservation on both paired-leg moves; idempotence — a duplicate payment-date event finds the entitlement discharged and proposes nothing; the obligation retires only from the zero vector. *In the ordinary state the condition vanishes, never the leg.*

9.8 The knock while pledged

The centrepiece assembles every rule above on one instrument: OT-1 (Cast, §2.1), owned by W-ALPHA. It is pledged under security-interest agreement G to counterparty wallet B at a declared valuation percentage of 50%: W-ALPHA posted +1, B posted −1; collateral value to B USD 200,000 against an exposure of USD 150,000. Then OMEGA prints its barrier close of 79.00.

1. The low print is recorded; the Event Monitor emits the touch. The knock is now on the record; no ledger state has changed — the Monitor emits events and cannot write state.

2. OT-1's contract fires on the event, and the Transaction Executor admits its proposal: UnitStatus → *triggered*; the entitlement is *determined* from the owned plane — W-ALPHA; and the conditional payment obligation is created — W-BANK owes W-ALPHA USD 1,000,000, discharge predicate carrying G's declared trapping terms. Mass on the posted plane is unchanged. A duplicate emission finds the unit triggered and proposes nothing.

3. The pricing layer, state-parameterised, prices triggered OT-1 at 0. Value has collapsed; mass is intact — coordinates carry mass, prices carry value.

4. G's contract, at its declared valuation watch, values B's received projection: 0. The delivery amount is USD 150,000; it fires a margin-call obligation with a deadline.

5. The payment obligation discharges under its predicate — release only to the extent no delivery amount stands. One transaction, two moves: owned(USD) USD 1,000,000 W-

BANK → W-ALPHA, and posted(USD) USD 150,000 W-ALPHA → B under G, delivered as credit support and discharging the call; USD 850,000 stays free with W-ALPHA. Coverage holds within the transaction: owned cash 1,000,000 ≥ posted 150,000. Had G declared no trapping terms, the predicate would be vacuous and the full payout would release; an agreement declaring proceeds attachment would have G's contract post the cash instead — declared terms decide.

6. The pledge closes: the marker return clears the posted plane of OT-1 (B/W-ALPHA ±1 → 0); then the extinguishment move owned(OT-1) W-ALPHA → W-BANK takes every wallet's vector for OT-1 to zero — **the ordering is forced by the coverage invariant**, which refuses extinguishing owned while posted mass remains; and OT-1 retires at the zero vector.

The door, across the six steps: idempotence at step 2; conservation per (unit, coordinate) on every move; coverage on step 5's transaction and again as the ordering constraint of step 6; retirement only from the zero vector. The residuals are named, not hidden. Between steps 2 and 5 the book is lawfully under-collateralised: sufficiency is an obligation, and the deadline covers it. The knock–revalue–call–discharge ordering is temporal, outside the move algebra, and carries the safety and liveness model of Chapter 15. And a worthless unit that no voluntary machinery will bury exits by the supervised write-off path below. *One move-and-coordinate algebra, no special case: the agreement's declared terms — never the machinery — decide where the payout lands.*

9.9 The lent plane: securities lending

The title-transfer derivation applies to the lending axis verbatim. Owned is the coordinate the move machinery can act on, and the borrower's purpose is to sell: a design that keeps the security on the lender's owned and marks the loan as lent-plane mass makes the borrower's short sale either unrepresentable — the borrower has no owned balance to sell — or coverage-violating — a lent-out marker standing against an owned balance the loan has emptied. A securities loan that passes title therefore books as financing: owned re-books at instruction; the lender holds the per-agreement claim-for-equivalent; the borrower's negative balance in it is the redelivery obligation; income is manufactured through the conditional obligation of the entitlement rule. The lent plane itself remains what the posted plane is: the marker plane for arrangements under which possession moves without ownership passing, written under the same coverage bound. One derivation, two planes — the regime key applied to lending, not a second rule.

Episode (securities loan): open, dividend, recall, return. Stock SIGMA trades at 40.00; wallet W-LEND holds 1,000 shares and lends them to W-BORR under title-transfer lending agreement L, whose declared terms carry the financing regime, an income watch, and the redelivery window. The collateral leg of the loan follows the regime key of this chapter unchanged and is not repeated here.

Trigger 1 — open. The executed loan instruction is recorded; L's contract fires. One transaction:

(1) move owned(SIGMA) 1,000, W-LEND → W-BORR — re-booked at instruction;
(2) issuance of the per-agreement claim-for-equivalent CLM-L: move owned(CLM-L) 1,000, W-BORR → W-LEND; the borrower's −1,000 is the redelivery obligation, and CLM-L's declared terms name SIGMA and L.

W-BORR may now sell outright — the purpose of the borrowing. CLM-L is priced by the borrower's state: it tracks SIGMA while W-BORR performs (USD 40,000 at inception) and decouples to a recovery claim on default.

Trigger 2 — a dividend mid-loan. SIGMA declares 2.00 per share. Determination reads the owned plane: W-BORR, holding 1,000. On the payment date, move owned(USD) USD 2,000, SIGMA issuer wallet → W-BORR. The same recorded event fires L's contract on its income

watch: the manufactured payment is the conditional obligation — W-BORR owes W-LEND USD 2,000, predicate carrying L's declared conditions (no default, no undischarged margin deficiency). In the ordinary state it discharges at once: move owned(USD) USD 2,000, W-BORR → W-LEND. The lender's cash offsets the dividend drop in CLM-L's price: a stockholder's profit and loss, exactly, while the borrower performs.

Trigger 3 — recall. W-LEND's recall notice is recorded at the boundary; L's contract fires, and the redelivery obligation acquires its deadline — the declared redelivery window — with redelivery of 1,000 SIGMA as its discharge predicate and close-out compensation as its declared fallback. By §14.1 it is discharged, compensated, or declared defaulted by the deadline; an unmet recall is a visible, overdue item, never silence.

Trigger 4 — return. The redelivery is recorded. One transaction:

(1) move owned(SIGMA) 1,000, W-BORR → W-LEND;
(2) move owned(CLM-L) 1,000, W-LEND → W-BORR, taking every wallet's vector for CLM-L to zero; CLM-L retires at the zero vector, and the redelivery obligation is discharged within its deadline.

The door, across the four triggers: conservation per (unit, coordinate) on every paired-leg move; the claim retires only from the zero vector; the deadline-bearing obligation is visible on the record until discharged. *A securities loan is title-transfer financing on the lending axis: the same re-booking, the same claim, the same conditional leg — no new machinery.*

The operational lifecycle of lending — locates, buy-ins, and the securities-financing reporting and settlement-discipline regimes that bind it — is the subject of the *SBL Operations Companion*; the ledger shape is complete here.

9.10 Retirement, and the supervised write-off

A lifecycle event extinguishes value, never mass (§4). A knock, a default, an expiry changes UnitStatus and price — never a coordinate balance — and a unit leaves the ledger only from the zero vector, the agreement's machinery clearing the marker planes first, in the order the coverage invariant forces.

Zero-vector retirement has no voluntary exit for a defaulted issuer's marker mass: nobody returns a worthless unit, and dead units would otherwise accumulate — polluting the encumbrance projections and drowning the obligation-liveness queue in items that can never discharge. The **supervised write-off path** is the exit: an issuer-default lifecycle event permits clearing the marker planes by compensating moves against explicit, simultaneous loss recognition — recorded, authorised, and open, without the voluntary agreement machinery, and never silent. The centrepiece's step 6 needs the voluntary machinery; a book that cannot run it runs this one.

9.11 Close-out and the netting algebra

A counterparty default is the one lifecycle event that must value every claim and obligation under a master agreement at once, net them to a single figure, and turn the residue into a recovery claim. The machinery is already built: the event-kind registry (Chapter 8), the claim priced by the taker, the coverage-forced ordering of §9.8, retirement from the zero vector, and the supervised write-off path above. This section is their assembly. It adds one unit — the **close-out amount claim** (CLM-CO) — and nothing else.

The event and its router. `COUNTERPARTY-DEFAULT` is a registered event kind (Chapter 8, Section 8.3) whose routing names the master agreement unit's own smart contract — the settlement-obligation pattern of Chapter 11, the master's contract the one legal writer of the close-out. It fires once per master referencing the defaulted counterparty; the members under that master are separated by the cause-derived identifier, one leg each; the default notice enters as a moveless observation-recording transaction. Routing to the master is what makes "net

within a master" a fact of *which* contract fires: no contract spans two masters, so no firing can net across them.

Taker-state valuation under the declared convention. Each member is valued at the close-out determination through the ordinary state-parameterised pricing layer (§8), priced by the taker — the claim-for-equivalent rule of this chapter, generalised from one claim to the netting set. The valuation convention — the side (bid, offer, or mid), the method (market quotation, loss, or dealer poll), and the termination currency — is declared data on the master agreement unit's close-out-mechanics terms, transcribed once from the executed agreement and never improvised at default. It is boundary data under TA-TERMS (Chapter 16, B7): a misdeclaration is a perimeter reconciliation, not an internal break, and the declared convention keeps the figure recomputable from the record.

The netting set, and the refusal across masters. The netting set is the set of units whose declared terms name one master agreement unit; the net is a projection over those units, never a stored figure. Cross-master netting is refused at the door, not made unrepresentable: every leg of one close-out transaction must name one master, read from each unit's declared terms, and a transaction whose legs name two masters is refused — a door check of coverage's shape (Invariant 14.8). Typing the master into every unit would contradict the per-agreement-via-declared-terms design of this chapter and add machinery the refusal already supplies.

One figure, one claim, recovery decoupled. The net mints as a single unit, CLM-CO: the surviving party positive, the defaulter's estate negative, par the net figure. Each member claim retires from the zero vector by close-out compensation — the declared §14.1 fallback the redelivery and margin obligations already carry. Recovery then decouples exactly as the claim-for-equivalent decouples on a taker default: CLM-CO's UnitStatus walks to *defaulted*, and its mark tracks recoverability, not par. Two marks across one status transition on one immutable unit are the loss recognised by construction (C-8.4) — the fall in the mark *is* the fall in NAV, and no second copy of a realised loss is kept beside it.

The write-down is lawful loss, not error repair. The par-minus-recoverable gap is recognised on the **supervised write-off path** above — person-authorised, evidenced, recorded, and open. It is a lawful economic loss by construction (C-8.4), not the forward repair of a wrong recorded event: C-12.4's discipline — human authorisation and evidence — is borrowed for the authorisation, but the write-down corrects nothing, because nothing was recorded wrong.

The ordering, forced by coverage. The admitted transactions run in one sequence, and the order is forced, not preferred: the default notice, moveless, routed to the master; the marker planes clear, because coverage refuses extinguishing owned while posted mass stands (the ordering of §9.8); the determination observations, moveless; net-and-mint, each member claim retired from the zero vector and CLM-CO minted at the net; the recovery decoupling to *defaulted*; the authorised write-down. Marker clearing precedes member retirement because a member claim is an owned holding, and coverage refuses its extinguishment while collateral marker mass still stands.

Close-out is itself an obligation (§14.1): its deadline is the declared close-out period, its discharge predicate is the amount determined and CLM-CO claimed, and its fallback is the supervised write-off. A stalled close-out is a visible, overdue item on the record, never silence and never a dead unit in the liveness queue.

Episode (close-out): the borrower defaults mid-loan. The loan of the previous episode stands: W-LEND lent 1,000 SIGMA to W-BORR under master agreement L, holding CLM-L (W-LEND +1,000, W-BORR −1,000), priced by the borrower's state. W-BORR has posted USD 41,000 of cash collateral under L's security-interest terms *without* right of use — posted-plane marker, received-negative (W-LEND −41,000, W-BORR +41,000), W-BORR's owned untouched. L's close-out-mechanics terms declare offer-side market quotation in USD. SIGMA now prints 44.00, and W-BORR defaults.

The walk. Seven admitted transactions, each through the one door:

(1) the `COUNTERPARTY-DEFAULT` notice is recorded, moveless, and routes to L's contract; the members cascade by the cause-derived identifier, and a duplicate notice proposes nothing;
(2) the collateral marker clears first — posted(USD) ±41,000 (W-BORR, W-LEND) → 0, and owned(USD) USD 41,000 W-BORR → W-LEND — one coverage-ordered transaction enforcing the security interest;
(3) the determination print 44.00 is recorded, moveless; CLM-L, priced at the taker's (W-BORR's) state, values at $1{,}000 \times 44.00 =$ USD 44,000;
(4) net within L: CLM-L's close-out value — the projection over L's netting set, USD 44,000 — less the USD 41,000 collateral applied at step (2), leaves USD 3,000 owed to W-LEND; the projection and the collateral offset are two operations, exactly as the recomputation property reads them (Chapter 15);
(5) CLM-CO is minted at par USD 3,000 (W-LEND +3,000, W-BORR's estate −3,000), and CLM-L retires from the zero vector by close-out compensation, its declared §14.1 fallback;
(6) CLM-CO's UnitStatus walks to *defaulted*, decoupling its mark from par — the claim-for-equivalent decoupling generalised;
(7) the authorised write-down records the evidenced recovery of 40% on the supervised write-off path, so CLM-CO marks at $0.40 \times 3{,}000 =$ USD 1,200 and NAV falls by USD 1,800 — the loss, by construction (C-8.4), with no second copy booked.

The door. Conservation per (unit, coordinate) on every paired-leg move; coverage orders step (2) before step (5)'s retirement; CLM-L and CLM-CO retire only from the zero vector; idempotence under the cause-derived identifier at every step. The loss is the fall in CLM-CO's mark and nothing besides. *One default, one door, one figure per master: the algebra the per-netting-set claim units were made first-class to carry.*

10 Virtual Ledgers, Strategies, and the Total Return Swap

Governed by C-10.1–10.5 (§16.5).

Nothing in this architecture is specific to real holdings. The machine of Chapters 2 through 7 — units, wallets, balances, an immutable log, and the three machines that map events into transactions and fold transactions into state — makes no reference to whether the positions it records will ever settle. A *virtual ledger* is a complete instance of that same machine whose holdings are notional: it settles nothing, and none of its wallets ever reaches a settlement instruction or the balance sheet. Everything else is identical: it is subject to the same eight commitments as a true ledger — above all to full reproducibility, so that any party holding the record recomputes its every number to the minor unit, and to time travel, so that every past state is reconstructible from that record alone.

The word *virtual* carries exactly one meaning throughout this specification, and no other: on the far side of the settlement boundary. A virtual ledger is not an approximation, a draft, or a lower grade of record. It is held to the identical standard of correctness; it differs from a true ledger in one respect only — its balances are claims about a hypothetical world, so no move it records is ever projected into a payment. This is the same boundary the settlement interface draws for a true ledger, read from the other side: the true ledger says what settles, the virtual ledger says what nothing settles.

10.1 The strategy as a smart contract

The motivating case is a quantitative investment strategy: a rulebook that, reading market observations, decides a hypothetical portfolio and rebalances it on a schedule.

A deterministic strategy is a smart contract, and its rulebook is the declared terms of a *strategy unit* — a non-valued unit (Chapter 3), whose price is undefined, not zero, because pricing the governance shell beside the portfolio it governs would count one economic reality

twice. The hypothetical portfolio the rulebook manages is an ordinary wallet in a virtual ledger. Each rebalancing is a recorded transaction: the rebalancing date is a declared watch on the strategy unit; the Event Monitor emits the event; the strategy contract fires, reads the stamped market observations and the portfolio as the virtual ledger then stands, and proposes the moves the rules require; the Transaction Executor admits them onto the virtual log. The contract never waits and never remembers — whatever it must carry from one rebalancing to the next it writes into the three homes, exactly as every other contract does.

The strategy's *level* is that wallet's Net Asset Value: the same NAV projection of Chapter 7, the same fold of the same log against a price vector, computed on demand and never stored. Because the strategy ledger is replayable and the rebalancing contract is a pure function of recorded rules and recorded observations, any party holding the recorded rulebook and the recorded observations recomputes the level exactly — which is the whole content of the claim that the level is well defined.

An index restatement follows the auditability commitment without exception. When the level of a published index is corrected — a late-arriving observation, a provider's amended fixing — the virtual ledger does not rewrite its log. It records a compensating entry that names the transaction it supersedes, so that both the original computation and its correction survive on the record. A restatement is an entry, never a rewrite; the historical level as first published and the historical level as corrected are both reconstructible, and which one a consumer read on which date is itself a fact of the record.

10.2 Two disciplines

Two disciplines make the arrangement safe, and both are checkable.

The bridge between ledgers is an observation, never a move. A move transfers a positive quantity within one ledger; conservation is stated per ledger, per unit, per coordinate, and it is closed by the virtual wallets that represent every outside party (Chapter 3). No move therefore spans two ledgers, and none can: a move from a wallet of `V-1` to a wallet of the true ledger has no paired leg that conserves in either, so the Transaction Executor of neither can admit it. The only thing that ever crosses is a number — a stamped level, admitted into the receiving ledger through the observation door of Chapter 8, with its provenance recorded like any other observation. The two ledgers touch at exactly one point, and at that point mass does not flow; a fact is copied.

The wall between models and the ledger holds level by level. A ledger records the outputs of its models and never runs one (Chapter 16); the true ledger does not execute the strategy, it consumes the level as an observation. This is not a property of the top ledger alone. Virtual ledgers may nest — a strategy over strategies — and at each level the same wall stands: each ledger relates to the one it references only through stamped observations admitted at its boundary, never by reaching inside to run the other's machinery. Because the wall is re-erected at every level, no amount of nesting lets a model's execution leak into an accounting record; each ledger remains a pure fold of its own log.

A short type sketch makes both disciplines mechanical:

```
-- a virtual ledger has the very same type as a true one;
-- only the settlement projection distinguishes them.
type VirtualLedger = Ledger

-- the sole thing that crosses the boundary is a stamped level:
observeLevel :: VirtualLedger -> Wallet -> Observation Money

-- a move lives inside one ledger and has no cross-ledger form;
-- none can be given a conserving type, so none can be written.
```

10.3 Real execution: orders, benchmark, and slippage

When the strategy's trades are meant to be real, the strategy's output toward the world is an *order* — an instruction, not a fact. An order records an intention to trade; it changes no balance, because nothing has happened yet. What happens is external: the market fills the order, wholly, partly, or not at all, at a price the strategy did not set. Those fills return as external events, captured at the boundary, mapped by the responsible contracts, and folded into a real wallet of the true ledger — the ordinary path of any executed trade.

The virtual ledger now becomes the benchmark. It holds what the rules say the portfolio should be; the real wallet holds what execution achieved. Both are NAV projections, each a fold of its own log against the same prices. Their difference is the slippage:

$$\text{slippage} \;=\; \text{NAV(benchmark wallet in } V) \;-\; \text{NAV(real wallet in the true ledger)}.$$

It is the exact difference of two folds, computed from two records, never an estimate — every cost that separates the idealised portfolio from the realised one is captured here in full, because both sides are recorded to the minor unit and neither is a model output.

10.4 The total return swap

A total return swap (TRS) is then one definition and no new machinery: its reference leg is the NAV of a designated wallet or virtual ledger. The swap itself is an ordinary unit of the *true* ledger. Its two counterparties hold wallets there; its smart contract, at each reset, reads the stamped NAV observation and emits the reset and financing moves. The reference NAV is not computed by the TRS — it enters through the door of Chapter 8, exactly as the strategy level does, and the wall between the two ledgers is the discipline above.

Episode TRS-1. TRS-1 has notional USD 10,000,000. Its reference leg is the NAV of wallet `W-QIS` in virtual ledger `V-1`, managed by the declared rulebook of the non-valued strategy unit `S-QIS`. Its financing leg is fixed at 4% per annum, reset quarterly ($\Delta t = 0.25$). At inception the reference NAV observation is USD 10,000,000, and the reference level is set to that figure.

The trigger. The first quarterly reset date arrives. `V-1` has been running: `S-QIS`'s rebalancings are recorded transactions on its log, and its level is the replayable NAV of `W-QIS`. On the reset date that level is stamped — USD 10,300,000 — and admitted into the true ledger as an observation with provenance (Chapter 8). The Event Monitor emits the reset event for TRS-1.

The contract that catches it. The Events Executor fires TRS-1's smart contract, handing it the reset event and the current projection of the true ledger. The contract reads the stamped reference NAV and the reference level carried since inception, and computes one net move.

The transaction it produces.

#	Component	Quantity (USD)
1	Total-return leg: stamped NAV 10,300,000− reference 10,000,000	300,000
2	Financing leg: $10{,}000{,}000 \times 4\% \times 0.25$	100,000
3	Net move — `owned(USD)`, payer → receiver	200,000
4	Reference level reset (UnitStatus): 10,000,000 → 10,300,000	—

The total-return leg is the appreciation of the reference over the period, USD 300,000; the financing leg accrues on the period's opening reference, USD 100,000; their difference nets to a single move of USD 200,000 of `owned(USD)` from the payer's wallet to the receiver's wallet. In the same transaction the contract writes the new reference level, USD 10,300,000, into the swap's UnitStatus, arming the next period against it. One reset, one net move, one state change.

What the door checks. The Transaction Executor admits the transaction on its structure alone. The USD 200,000 move is a paired-leg `owned(USD)` transfer, so conservation holds by

construction. The reference-level write has a single legal writer — the swap's UnitStatus — and lands in its one home. The proposal carries a cause-derived identifier keyed to the reset observation, so a retried or duplicated reset commits once and no second time. The stamped NAV that the contract read is itself on the record, with its provenance, so the whole reset is reproducible: re-running the map on the recorded observation reproduces the identical USD 200,000.

The design point. The reset moves cash in the true ledger, and the strategy that produced the number never touched it: the only thing that crossed the settlement boundary was a stamped observation, and a move never can.

10.5 The simulated ledger

Concrete case first. At the close of a recorded trading day the true ledger stands in a definite state: its log has a last position. Take that position as a **branch point**, copy the log up to it, keep every contract and every machine, and continue it — not with the day's real events, but with a generated stream of clock ticks and boundary observations. The result is a **simulated ledger**: a virtual ledger (this chapter) whose log extends a recorded prefix of another ledger's log, driven onward by generated events. Branch `V-SIM` at that close and drive OT-1 — the one-touch on OMEGA struck at barrier 80.00 (Chapter 3) — down 1,000 generated OMEGA paths. Each path is a sequence of generated official-close observations admitted through the observation door (Chapter 8); on each, OT-1's contract fires unchanged, and a path whose close touches 80.00 pays the declared USD 1,000,000 while one that never does pays nothing. The price estimate is the mean payout across the thousand paths. No new machinery produced it: the same door, the same contract, a generated stream in place of the real one.

The generator is outside the ledger. The stream of ticks and observations is produced by a **generator** — a model, and a model lives outside the fold (the wall between models and the ledger, Chapter 16). The generator's one non-record input is the **seed**. The seed is recorded, so a **path** is a deterministic function of the branch point, the generator version, and the seed: fix those three and the path replays bit for bit. Nothing else enters. This is the confinement the constitution's simulability commitment demands (C-2.8) — one non-record input, recorded, so every path replays exactly.

No simulation-only variant exists. This is the section's discipline, and it is absolute. There is no simulation build of any contract, and none of the Event Monitor, the Events Executor, or the Transaction Executor. The generated stream is admitted through the one door that admits real events and folded by the same contracts into the same three homes (Chapter 6). A contract that behaved one way in production and another under simulation would be an untested second implementation — the exact divergence the one-record architecture abolishes. The simulated ledger is held to the same eight commitments as any ledger; it differs from the true ledger in one respect only, that its driving events were generated rather than observed.

Results cross back as observations. A simulated ledger settles nothing, exactly as any virtual ledger. A number computed on it re-enters the true ledger only as an observation with provenance — branch point, generator version, seeds, path count — through the observation door of Chapter 8, so the figure is reproducible from the record; no move ever crosses from the simulated ledger to the true one.

One engine, two measures. Chapter 15's generator regime is this same engine. Its generated products, events, and histories are simulated paths branched from recorded states and folded by the same contracts and the same door; only their purpose differs. A path driven to price OT-1 runs to ask what the instrument is worth; a path driven by the adversarial scheduler runs to ask whether an invariant can be broken. The machinery is identical — one generator universe, one door, one fold — and the two uses are two measures over the same paths. Refutation and pricing are one computation aimed at different questions.

```
-- a branch point is a recorded prefix of a ledger's log:
branch        :: Ledger -> LogPrefix -> BranchPoint
-- a path is a deterministic function of (branch point, generator version, seed),
-- run through the same contracts and the same door as production:
simulate      :: BranchPoint -> GeneratorVersion -> Seed -> Path
-- a number computed on a path re-enters the true ledger only as a stamped
-- observation with provenance -- never as a move (Chapter 8):
observeResult :: Path -> Observation Money
```

11 The Settlement Interface

Governed by C-Scope.9 (§16.5).

11.1 Settlement as a projection

The settlement interface is where the closed ledger meets the open infrastructure of settlement. It is a projection like any other (§3): a function of the committed log that reads the moves the ledger has admitted and emits *settlement instructions*. The division of labour is exact — the ledger records *what* settles; the external infrastructure decides *how*, and how is out of scope (settlement finality, payment-system access, and central-securities-depository connectivity are external authorities the ledger reconciles against, never functions it performs). An instruction is the boundary object: it names the transfer to be performed and carries none of the machinery that performs it. Settlement in size may confirm in part: the ledger records the settled quantity each confirmation carries, as a new external fact, while how a delivery is partialled, tranched, or auto-split under settlement discipline belongs to the settlement layer — the *how*, out of scope.

What the interface never does is write. It is not the Transaction Executor and holds none of its privilege; it proposes nothing to the door and changes no balance. Every fact the interface consumes is already on the record, and every fact it produces is an instruction, never a move.

- *Pure.* It reads the recorded log alone — no clock, no call to the outside world — so any party recomputes the same instructions from the record (§2, §3).
- *Total.* It is defined on every committed transaction. A transaction that carries no boundary-crossing move — a moveless corporate-action component, a holding in a virtual ledger (§10) — has the empty instruction list as its image, by construction, not by a special rule.
- *Deterministic.* The same log yields the same instructions, always.
- *Idempotent.* Because it is a function of the recorded log and the settlement-obligation unit's recorded lifecycle node, re-running it produces the same instructions, and an instruction whose transfer the record already shows as settled is recognised and not performed twice.

```
settle :: Ledger -> [Instruction]   -- pure, total, deterministic, idempotent;
    writes nothing

data Instruction                    -- the boundary object: what settles, never
    how
  = Cash     WalletId WalletId UnitId MinorUnits  -- from, to, currency, amount
  | Delivery WalletId WalletId UnitId Quantity     -- from, to, security, quantity
```

One consequence is stated once and relied on throughout: *settlement failure is a recorded event, never a reversal.* A settlement that does not complete returns as an external event, recorded as a new fact; the instruction's underlying move is not undone. History is never rewritten (§2): a trade ultimately unwound is unwound by a compensating transaction, through the same door, never by deleting the booking that instructed it.

11.2 Four episodes

Dividend: cash projects to one instruction. W-ALPHA holds 10,000 ACME; the declared dividend is 1.50 per share, and nothing encumbers the holding. The payment-date firing has already produced its transaction (Chapter 9): the conditional payment obligation discharges vacuously, and one move crosses the boundary — owned(USD) USD 15,000, ACME's issuer wallet → W-ALPHA. *The projection* reads that committed move. *The instruction:*

(1) Cash USD 15,000, ACME issuer → W-ALPHA.

The moveless component of the same transaction — the obligation's creation and vacuous discharge — crosses nothing and projects to the empty list. *The door* is not the interface's: the move already passed the Transaction Executor, and the projection only reads what was admitted. *The ledger determined and recorded the entitlement; the interface instructs only the cash that leaves the ledger's edge.*

Future: each day's margin, and the expiry unwind. FUT-IDX is settled-to-market, multiplier 10; W-ALPHA is long one contract bought at 995.00. On each settlement print the future's contract fires and emits the day's variation-margin move (Chapter 9); the interface projects each to one instruction, and the expiry's final settlement to a settlement instruction that also closes the contract.

Recorded print	Variation margin	Committed move	Instruction
day 1 1,000.00	+50	owned(USD) 50, W-CCP → W-ALPHA	Cash 50 to W-ALPHA
day 2 990.00	−100	owned(USD) 100, W-ALPHA → W-CCP	Cash 100 to W-CCP
day 3 1,005.00	+150	owned(USD) 150, W-CCP → W-ALPHA	Settlement 150 to W-ALPHA

At expiry the position unwinds: the final settlement of USD 150 projects to the settlement instruction above, while the unit-state change that retires the contract carries no move and projects to nothing. *The door* again belongs to the Transaction Executor; the interface reads three admitted moves and emits three instructions. Cumulative variation margin is $50-100+150 = 100$, the total profit and loss (Chapter 7) — but the interface never sees the profit and loss, only the cash that crosses each day. *The ledger nets the day to a single move; the interface settles exactly that move, and no valuation reaches the boundary.*

Split: the moveless corporate action projects to nothing. ACME splits two-for-one, effective 2026-06-01. One transaction carries the re-denomination and the new terms (Chapters 3 and 8):

(1) move owned(ACME) 10,000, ACME issuer → W-ALPHA — W-ALPHA's 10,000 shares become 20,000;

(2) a new ProductTerms version: price frame 100.00 → 50.00, with the market data operators declared (spot scales by $\frac{1}{2}$; the cash dividend figure scales 1.50 → 0.75; proportional figures stand).

The projection. Component (2) carries no move; its image is the empty list by the projection's totality. Component (1) is the ledger mirroring a re-denomination the external register performs of its own accord as the corporate action; the interface issues no delivery or payment instruction for it, because a split settles nothing between counterparties. *The door* admitted a well-formed transaction whose consistency of reference is checked in Chapter 14 (20,000 against the fresh 50.00, never the stale 100.00); the interface adds no check and emits nothing. *What crosses the boundary is a transfer; a corporate action that re-denominates terms crosses nothing, so it projects to nothing — by construction.*

Variance swap: the terminal settlement projects to one instruction. VS-1 is a one-year OTC variance swap on IDX, 252 fixings, strike 400 variance points at USD 1,000 per point.

Through its life each fixing is a log-return the ledger computes from recorded IDX closes, not a transfer (Chapters 8 and 7); none crosses the boundary, and each projects to the empty list. At expiry the realised variance is 441 points and the contract proposes one settlement move — owned(USD) $1{,}000 \times (441 - 400) =$ USD 41,000, from the strike payer to the realised-variance receiver. *The projection* reads that one committed move. *The instruction:*

(1) Cash USD 41,000, strike payer → realised-variance receiver.

A whole year of fixings accrues one statistic and settles once; only the terminal cash crosses the edge, and the interface instructs exactly it.

11.3 The settlement-obligation unit and the market claim

One recorded fact upstream of the projection is load-bearing for its correctness: the settlement level of a trade. A determination turns on it — who receives a distribution when a trade is instructed but unsettled across a record date — and the fact must be read, not assumed away. That fact is not floating free with nowhere to live. Every right and obligation a legal contract creates is a unit (§5.6), and an instructed-but-unsettled trade is exactly an obligation: a delivery-versus-payment obligation, the seller owing the security and the buyer the cash. So it is a unit — the **settlement-obligation unit**. Its lifecycle node — `instructed`, then `settled` or `failed` — is a property of the unit alone, the same for every reader, so it lives in that unit's UnitStatus (Chapter 6), which is a home like any other. There is no free-standing settlement-state kept somewhere the three homes do not reach: the routing below reads the unit's recorded node exactly as it reads any other recorded fact, and sufficiency (Chapter 6) holds with the three homes, not a fourth.

Owned re-books at instruction. Title moves on the instruction event, on the record (§4), not on settlement confirmation — the trade-date booking of Chapter 9. The instant a trade is instructed the owned plane shows the buyer; the settlement instruction the interface projects is the promise that the external register will catch up. Between the two lies the instruction-to-settlement gap, and the gap is carried explicitly — never elided, never back-filled at read time.

A record date inside the gap. Economic entitlement to a distribution is fixed by the *ex-date*, not by the record date alone. A trade executed before the ex-date is *cum* — the price it paid still includes the coming distribution — and one executed on or after the ex-date is *ex*, its price already excluding it; the ex-date operator that separates the two frames is the one Chapter 8 declares, and the cum/ex invariance witness that forbids reading a price on the wrong side of it is catalogued in Chapter 14. The routing predicate reads the trade's execution date against that recorded ex-date; the record date only fixes the registration snapshot the paying agent pays against. Whom the paying agent actually credits turns on a second recorded fact — whether the trade has *settled* by the record date, so the external register shows the buyer, or is still *unsettled*, so it still shows the deliverer. Two recorded facts, each binary — cum or ex, settled or unsettled at the record date — make four quadrants. The paying agent pays the registered holder in all four; in two the registered holder is the entitled party and nothing more is owed, and in the other two the two diverge and a leg of the settlement-obligation unit reconciles them.

The cum case. A *cum* sale still unsettled at the record date leaves the owned plane showing the buyer — who is cum-entitled — while the paying agent credits the *record holder* at the external register, still the deliverer, the seller. The two disagree by construction, and the disagreement is not a break to be reconciled at report time: it is a recorded obligation. It is carried by the settlement-obligation unit at its `instructed` node: across a record date in the gap, the record-date firing walks that unit onto its market-claim branch, raising the **market claim** — the reconciling leg whose obligee holds it positive and whose obligor holds it negative, the cum buyer's claim-for-equivalent on one side and the deliverer's duty to remit on the other, discharged when the deliverer receives the distribution.

The ex case. An *ex* sale unsettled at the record date raises no buyer claim at all: the buyer bought ex, is not entitled to this distribution, and the registered holder — the deliverer — keeps it, matching the ex-buyer's economic expectation exactly. The owned plane shows the buyer here too, so owned read alone would misroute; entitlement keys on the ex-date, and the record-date contract stays silent.

The fourth quadrant: an ex sale settled before the record date. The two cases above both leave the trade unsettled at the record date. Settle it, and the register catches up. For a *cum* sale settled by the record date the register now shows the buyer, who is cum-entitled — registered holder and entitled party are the same, and no claim arises. The last corner is the one the earlier enumeration missed. Take an *ex* sale — the buyer bought without the coming distribution and is not entitled — that settles *before* the record date, so the register shows the buyer as the record holder. The paying agent credits the buyer, who is not entitled, while the seller, who is, has been de-registered. This is the exact mirror of the cum case: there the registered holder lagged the entitlement, here it leads it. The repair is the mirror of the market claim — the same settlement-obligation unit, its market-claim leg raised at the `settled` node with the signs reversed: the **mirror market-claim** from the wrongly-paid buyer to the entitled seller, obligee the seller, obligor the buyer, discharged when the buyer receives the distribution and remits it.

When does an ex sale settle before the record date at all? Under the ordinary calendar it cannot: the ex-date is set relative to the record date and the settlement cycle precisely so that a trade executed ex settles only *after* the record date, landing back in the ex-unsettled case. That coherence is an external market convention, not a fact the ledger enforces; it is named TA-EXDATE in Chapter 16 and reconciled against at the boundary (Chapter 8). It breaks *lawfully* under a due-bills regime for a large special dividend, whose ex-date falls *after* the payment date: there an ex sale can settle before the record date, the fourth quadrant is reached, and the mirror market-claim leg routes the distribution to the entitled seller — on the record, by a leg the ledger raises, not by a convention it assumes.

Here is the seam that keeps the projection pure: the market claim is created not by the projection, which writes nothing, but by the contract that fires at the record date. That contract reads the owned plane, the trade's cum/ex status against the recorded ex-date, and the settlement-obligation unit's recorded lifecycle node, and proposes the reconciling leg — where the cum case calls for one — as one component of the determination transaction, which the Transaction Executor admits like any other. The projection then reads the resulting committed moves; it originates none of them.

One product graph governs the settlement-obligation unit, and three mechanisms that were once specified apart are its nodes and edges. The ordinary settlement is the spine, `instructed→settled`; the fails cascade (Chapter 9, §9.3) is the edge `instructed→failed`, on which the very same unit *becomes* the fails-cascade obligation; and the market claim is the branch raised at the `instructed` node when a cum trade's record date falls in the gap, with its mirror raised at the `settled` node when an ex trade's record date follows settlement — one mechanism, reflected across the settlement node. Being a product graph like any other, its consistency is the four executable properties of Invariant 3.2, tested over generated settlement histories in Chapter 15; no new machinery is added, and no fourth home.

```
-- The settlement-obligation unit (S5.6): an instructed-but-unsettled trade is a
-- delivery-versus-payment obligation, so it is a unit like any other.
data SettlementObligation = SettlementObligation
  { deliveryLeg   :: (WalletId, UnitId, Quantity)    -- Q, the security the seller
     owes the buyer
  , paymentLeg    :: (WalletId, UnitId, MinorUnits)  -- the cash the buyer owes
    the seller
  , dueDate       :: Date
  , dischargePred :: Predicate  -- satisfied by settlement confirmation on the
    external register
```

```
  }
-- ONE product graph governs it (Ch. 3). Its lifecycle node lives in UnitStatus (
    Ch. 6):
--   instructed --confirmed--> settled     the ordinary walk; the unit retires
    unless a
--                                          record date straddles the settled node
     (see below)
--   instructed --fails------> failed      the fails-cascade obligation (S9.3):
    the SAME unit, later node
--   instructed --partial(q_s)-> SPLIT     on a partial confirmation 0 < q_s < Q
    (= deliveryLeg
--                                          quantity): mint a settled leg (q_s, at
     settled) and a
--                                          failed-residual leg (q_r = Q - q_s, at
     instructed); the
--                                          parent retires from the zero vector,
    q_s + q_r = Q
--   at INSTRUCTED, a CUM trade across a record date raises the market-claim leg (
    deliverer -,
--     cum-buyer +), valued at par, discharged by the deliverer's receipt of the
    distribution.
--   at SETTLED, an EX trade across a record date raises the MIRROR market-claim
    leg, signs
--     reversed (ex-buyer -, seller +): the fourth quadrant, discharged by the
    buyer's receipt
--     and remittance to the seller. Same mechanism, reflected across the
    settlement node.
-- Routing reads this node from UnitStatus; there is no free-standing settlement-
    state type.
```

Proposition 11.1 (Entitlement routing)**.** *The recipient of a distribution is a total function of the recorded owned plane, the trade's cum/ex status against the recorded ex-date, and the settlement-obligation unit's recorded lifecycle node at the record date. The last two facts are each binary — cum or ex, settled or unsettled at the record date — and partition into four quadrants; the paying agent credits the registered holder in all four, and a market-claim leg reconciles exactly the two in which the registered holder is not the entitled party.* (i) Cum, unsettled: *the trade is still at its* `instructed` *node, so the register shows the deliverer, and the settlement-obligation unit's market-claim leg carries the reconciling claim from the deliverer to the cum-buyer.* (ii) Cum, settled: *the register shows the buyer, who is entitled, so no leg arises.* (iii) Ex, unsettled: *the register shows the deliverer, who is entitled, so no leg arises.* (iv) Ex, settled — the fourth quadrant: *the register shows the buyer, who is* not *entitled, so the same settlement-obligation unit, at its* `settled` *node, raises the* mirror *market-claim leg, signs reversed, carrying the reconciling claim from the wrongly-paid buyer to the entitled seller. No distribution reaches any wallet except by a recorded move against a determined entitlement or the discharge of a market-claim leg — direct or mirror — of the settlement-obligation unit. On a partial fill the settlement-obligation unit splits into a settled leg and a failed-residual leg, each a settlement-obligation unit at its own node, so the four quadrants apply leg-wise: a cum sale part-settled across the record date raises the market-claim leg on the unsettled quantity alone. The proposition is thereby total over partial fills, with no new routing logic.*

Worked micro-example (dividend). Stock ACME pays a cash dividend of 1.50 per share; the record date is 2026-05-15. Wallet W-ALPHA holds 10,000 ACME and sells them to a buyer, wallet W-BETA: the sale is instructed on 2026-05-14 for settlement on 2026-05-16, so the record date falls in the gap. The 2026-05-14 execution precedes the ex-date associated with the 2026-05-15 record date, so the sale is *cum* — which is why the distribution routes to the buyer;

an otherwise identical trade executed on or after the ex-date — and, as here, still unsettled at the record date — would be *ex* and raise no market claim. (Had that ex trade instead settled before the record date, the fourth quadrant, it would raise the mirror market-claim to the seller. The cash consideration settles symmetrically and does not bear on the entitlement.)

2026-05-14 — the sale; owned re-books at instruction. One transaction:

(1) move owned(ACME) 10,000, W-ALPHA → W-BETA — re-booked at instruction; the settlement instruction, due 2026-05-16, is recorded as unsettled.

The owned plane now shows W-BETA holding 10,000 ACME and W-ALPHA holding none.

2026-05-15, record date — determination and the market claim. The record-date watch fires; the stock's contract determines the entitlement and, reading the trade's cum status against the ex-date and the settlement-obligation unit's recorded node, proposes the reconciling leg. One transaction:

(1) determination reads the owned plane — W-BETA owns 10,000 — and records the entitlement, 10,000 × 1.50 = USD 15,000, to W-BETA in PositionState;
(2) the sale is cum and its settlement-obligation unit is still at `instructed` across the record date, so the market-claim branch is walked: the paying agent will credit the record holder, the deliverer W-ALPHA. Raise the market-claim leg MC-1 — the settlement-obligation unit on its record-date branch — valued at par: move owned(MC-1) 1,500,000 (minor units, = USD 15,000), W-ALPHA → W-BETA — W-ALPHA holds −1,500,000 (the duty to remit once received), W-BETA holds +1,500,000 (the claim-for-equivalent). MC-1's discharge predicate is the deliverer's receipt; the obligation is contingent on it and is matched by the deliverer's record-holder receivable, so the conduit's owned value is neutral. While W-ALPHA performs the claim is valued at par, and it decouples to a recovery claim on W-ALPHA's default.

This transaction is admitted through the door; MC-1 exists because the record-date firing wrote it, not because the projection did.

The sale settles on 2026-05-16, recorded as an external confirmation; this updates the register but not the record-date snapshot that governs the distribution — the paying agent still credits the record holder as of 2026-05-15.

2026-05-22, payment date — the distribution and the discharge. The payment-date watch fires. One transaction, two moves:

(1) move owned(USD) USD 15,000, ACME issuer wallet → W-ALPHA — the paying agent credits the record holder;
(2) move owned(USD) USD 15,000, W-ALPHA → W-BETA — the market claim discharges under its predicate, and MC-1 retires at the zero vector.

W-ALPHA nets to zero across the two moves — a conduit, not an owner; W-BETA ends with the USD 15,000 it economically owns. *The door*: conservation per (unit, coordinate) on every paired-leg move; MC-1 retires only from the zero vector; idempotence — a duplicate payment-date event finds MC-1 discharged and proposes nothing. Each of these two cash moves is what the interface then projects to a cash instruction; the market claim itself never crosses the boundary — it is the internal record that keeps the two boundary-crossing payments consistent.

Settlement failure is a recorded event, never a reversal. Should the sale fail to settle on its due date, the settlement-obligation unit walks to its `failed` node — and the instruction's owned re-booking is not undone and the market claim is not withdrawn. The walk has exactly one writer, and it is named: the settlement-obligation unit's own smart contract, the one legal writer of that unit's lifecycle node (Chapter 6, Invariant 6.2). It proposes the transition on one of two recorded triggers — an external fail notice, a registered event kind whose routing names this settlement-obligation unit's contract (Chapter 8), admitted through the one door as a moveless observation-recording transaction, or the due-date watch firing and finding the cumulative confirmed settled quantity short of the instructed quantity by the deadline. The second trigger reads quantity, not mere presence, and a partial settlement is not carried on one overloaded node:

it walks a declared **partial edge** that splits the unit. On a partial confirmation — cumulative confirmed quantity q_s with $0 < q_s < Q$, where Q is the instructed quantity — the edge's action mints two successor settlement-obligation units of the same product graph and retires the parent: a **settled leg** at the `settled` node carrying q_s, and a **failed-residual leg** carrying $q_r = Q - q_s$, at `instructed` until its deadline and then walked `instructed`→`failed` on the residual alone. Each leg carries the parent's identity and its own cause-derived txid; the split is one paired-leg transaction that moves the parent's delivery and payment mass into the legs, so $q_s + q_r = Q$ on the delivery coordinate, the payment is pro-rated across the two, and the parent leaves the ledger from the zero vector (§4) — its mass conserved, none created and none destroyed. The fails cascade then fires on the residual leg's q_r only; the settled leg is untouched. The split adds no machinery: each leg is a settlement-obligation unit at its own node, so the four-quadrant routing below applies leg-wise and stays total over partial fills — and a further partial confirmation on the residual leg splits it again by the same edge, recursively, each round conserving its own $q_s + q_r$. In both cases the Transaction Executor admits the transition like any other transaction; no other machine writes the node, and no `failed` node exists without one of the two triggers on the record. The settlement-obligation unit's market-claim leg carries a deadline, a discharge predicate, and a fallback (§14.1); by its deadline it is discharged, compensated, or declared defaulted — a failed distribution is a visible, overdue item, never a silent balance edit.

Worked micro-example (partial fill across the record date). Same cast, same dividend: W-ALPHA sells its 10,000 ACME to W-BETA, instructed 2026-05-14, due 2026-05-16, and the 1.50 dividend has record date 2026-05-15. On 2026-05-15, before the record snapshot, the external register confirms 6,000 of the 10,000 settled. The settlement-obligation unit walks its partial edge and splits: a settled leg SO-s carrying $q_s = 6{,}000$ at `settled`, and a failed-residual leg SO-r carrying $q_r = 4{,}000$ at `instructed`. $6{,}000 + 4{,}000 = 10{,}000$, and the parent retires from the zero vector.

Record date — determination, leg-wise. The watch fires on each leg. Determination reads the owned plane — W-BETA owns 10,000, title having moved at instruction — and records the full entitlement $10{,}000 \times 1.50 =$ USD 15,000 to W-BETA in PositionState. Routing then splits by leg:

(1) SO-s is *cum, settled*: the register shows W-BETA for its 6,000, who is entitled, so the paying agent credits W-BETA the $6{,}000 \times 1.50 =$ USD 9,000 directly and no leg arises (quadrant (ii));

(2) SO-r is *cum, unsettled*: the register still shows the deliverer W-ALPHA for its 4,000, so the market-claim leg MC-r is raised on the unsettled quantity alone, $4{,}000 \times 1.50 =$ USD 6,000: move owned(MC-r) 600,000 (minor units, = USD 6,000), W-ALPHA → W-BETA — W-ALPHA −600,000, W-BETA +600,000 (quadrant (i)).

The two paths sum to W-BETA's USD 15,000: USD 9,000 credited directly, USD 6,000 by the claim. The claim is on 4,000, not 10,000, because only 4,000 is unsettled at the record date.

Due date — the residual fails. On 2026-05-16 SO-r's cumulative confirmed quantity is still 0 against its 4,000, so its contract walks it `instructed`→`failed` on the 4,000; SO-s, already `settled`, is untouched. The fails cascade carries the 4,000 residual as a recorded obligation, and MC-r stands to its deadline. Nothing is reversed, and every quantity is conserved.

Load-bearing, not a reporting axis. The settlement-obligation unit's lifecycle node is therefore a correctness input, not a presentation dimension. A design that read owned alone would instruct the venue to pay the buyer directly — an instruction it cannot honour, since the buyer is not yet the record holder — and the cash would arrive at the deliverer with nowhere on the record to go, resurfacing as exactly the reconciliation break the ledger exists to abolish. Reading the settlement-obligation unit's recorded node, and carrying the market claim on that unit, makes the routing both faithful to what the venue will do and reproducible by any party from the record alone (§2, §3). The construction is symmetric across the settlement node, and that symmetry is the fourth quadrant: where a cum trade unsettled at the record date raises the

market claim from deliverer to buyer, an ex trade settled by the record date raises the **mirror market-claim** on the same settlement-obligation unit, its signs reversed, from the wrongly-paid buyer to the seller — one mechanism, total over all four quadrants of the gap.

The ledger says what settles and, across the gap, who is owed; the interface projects that settled activity into instructions, writing nothing and inventing nothing.

12 Presentation and Reporting Projections

Governed by C-2.1–2.2 (§16.5).

12.1 The projection principle

Every reporting figure is a projection: a pure, total, deterministic function of the record and of the declared terms the record carries, and of nothing else. There is no reporting store and no parallel reporting pipeline; a report is computed when asked for and discarded when read.

The principle is forced by §1: a reporting store would be a second store of facts the record already holds, the reconciliation failure the architecture abolishes.

Three commitments of §2 are discharged at once, by construction rather than by apparatus. *Auditability*: a projection is a fold over the immutable log (Chapter 2), so every figure decomposes mechanically into the transactions that entered the fold — drill-down is the fold's own trace. *Reproducibility*: the function is pure and its inputs are recorded, so any party holding the record recomputes the same figure to the last minor unit; where a figure involves value, the prices come from the pricing layer, itself reproducible from recorded inputs (Chapter 16). *Time travel*: a report as of any past date is the same function applied to the log up to that date, and a restated report is a new projection of a longer log containing recorded corrections — never an edited document.

One input class stands outside the record: reference data — identifiers, classifications, counterparty attributes — owned by external authorities, per the constitution's scope. It is supplied at the boundary by the authority that owns it and joined to the projection as the report is built. The join attaches labels and groupings; it can never alter a quantity. Gating admission and altering a quantity are different acts, and neither contradicts the other: an eligibility predicate may read declared or reference data to refuse a transaction at the door (Chapter 9), but a refusal admits or rejects, it never rewrites a booked number — and the report-time join does not even gate, it only labels. A figure carrying a wrong identifier is a labelling defect at the boundary, repaired by re-joining fresh reference data to the unchanged projection — never a ledger defect. The same division of labour governs format: the ledger computes what the figures are; a submission gateway, outside the boundary, arranges them into whatever shape a supervisory template requires. Per-regime field inventories therefore live outside this specification and are recorded as exclusions in the Exclusions Register; where a template requires a figure, the figure is derived here from the record and shown to satisfy the template — the template is never the derivation.

```
-- One arrow, no state of its own, total in every argument.
-- Declared terms and conventions travel inside the prefix.
report :: LogPrefix -> BoundaryRef -> Report
```

12.2 The presentation projection

Chapter 9's regime key decides what a collateral inflow writes; the presentation projection recovers from those writes the figures that a balance sheet or a supervisory template asks of a firm that finances, pledges, and re-uses assets. It reads coordinates, units, and declared terms alone. Four figures cover the surface.

Transferred assets and associated liabilities. Under title-transfer financing the poster's asset re-books to the taker; the poster holds a claim-for-equivalent unit and the taker a repurchase or redelivery obligation unit, both per governing agreement (Chapter 9). The projection maps each claim held under a financing agreement to a transferred asset at fair value — the claim priced by the pricing layer, parameterised by the taker's state (Chapter 7) — and maps the paired obligation to its associated liability. The presentation is complete within the ledger's fair-value scope.

Encumbrance, poster side. Under pledge the asset never leaves the poster's owned coordinate; its encumbered mass is the posted-as-collateral ray, read *per agreement and summed gross*, never as the signed net across agreements. The distinction is not pedantic: a wallet that posts 100 of a unit as collateral under agreement G_1 and receives 100 of the same unit as collateral under a different agreement G_2 is fully encumbered — 100 is pledged away under G_1 — yet its net posted coordinate, $\mathrm{bal}_{\mathrm{posted},G_1} + \mathrm{bal}_{\mathrm{posted},G_2} = (+100) + (-100) = 0$, is zero. A projection reading that net would report the wallet as unencumbered, which is false. So the poster-side encumbrance reads the per-agreement coordinates $\mathrm{bal}_{\mathrm{posted},G}$ (Section 3.7) gross:

$$\mathrm{encumbered}(w,u) \;=\; \sum_{G} \max\big(\mathrm{bal}_{\mathrm{posted},G}(w,u),\,0\big),$$

the posted-as-collateral mass summed over agreements; the received-as-collateral mass $\sum_G \max(-\mathrm{bal}_{\mathrm{posted},G}(w,u),0)$ is the separate receiver-side figure of the re-use line below. Under title-transfer financing the encumbered position is instead the claim held under the financing agreement. Available mass — the quantity the wallet can still deliver — is a different question, answered by the coverage net $\max(\mathrm{bal}_{\mathrm{owned}},0) - \sum_G \mathrm{bal}_{\mathrm{posted},G}$ (Chapter 14), the same $\max(\cdot,0)$ the coverage invariant carries, and non-negative for *every* wallet because that invariant bounds the signed posted sum: a negative-owned wallet — a plain short, or an issuer wallet conservation itself creates — has zero available free mass, never a negative figure. Written without the max the figure would go negative for exactly those wallets; the max keeps the two chapters one statement. Encumbrance sums gross, availability nets — the two are never one figure.

Re-use, receiver side. Mass held on the received-as-collateral ray is available for re-use *where the governing agreement grants a right of use.* Actual re-use is read from re-pledge moves, each carrying a source-agreement reference beside its governing reference (Chapter 9), so received, re-used, and available project as per-(unit, agreement) actuals. Collateral received *without* right of use (Constitution C-8.8, segregated included) cannot be re-used, and that is not a projection convention but an admission bound: the coverage invariant is gated by the source agreement's right-of-use term (Chapter 14, the right-of-use gate), so received mass under a no-right-of-use agreement contributes zero to the coverage net and a re-post of segregated mass is refused at the door. The projection therefore reports re-use only where the door admitted it — there is no re-use it could report that the gate did not already permit.

Fungible received mass. Title-transfer received cash and securities land on owned, commingled with proprietary holdings of the same unit. The record cannot say which dollar was re-invested, because no bank statement can say it either: a per-dollar "actual" over commingled mass would be a stored fiction. Figures over this mass — how much of the received value was re-used, and where — are attributions, computed by the declared convention of the next subsection.

Reported figure	Read from — and from nothing else
Transferred asset / associated liability	claims and obligation units under financing agreements
Encumbrance under pledge	posted-as-collateral ray, per agreement, summed gross — never the s
Encumbrance under financing	claims held under financing agreements
Re-use actuals (pledge form)	source-agreement references on re-pledge moves
Re-use of received fungibles	owned mass, obligation units, the declared convention

12.3 The attribution convention is total

The convention is data, not code: it is declared once, in the declared-data slot of Chapter 8, and consumed here — behaviour carried as declared data, per §2. It has exactly three components:

1. the *pool predicate* — a declared wallet and move predicate defining the pool over which received value is attributed (for instance, the wallets and moves constituting the reinvestment book); it is total over recorded log state, ranging only over wallets and moves already on the log, so an input not on the record is not admissible and there is no hidden manual-tagging step outside the projection;
2. the *period basis* — the timestamp or period on which the pro-rata ratio is struck;
3. the *rounding rule* — with remainders booked explicitly, per §4's remainder discipline; nothing is silently dropped.

Principle 12.1 (Attribution totality)**.** *The declared convention is a total function of log state: for every log prefix it assigns every attributable quantity; the attributed parts and the explicit remainder partition the whole; and two readers of one log and one declared convention compute identical figures to the last minor unit.*

A convention missing any of the three components is a defect: two readers of one log would compute different figures, which is internal reconciliation failure reintroduced through a report. The projection therefore fails closed — it refuses to produce the figure rather than improvise a ratio (§2, Correctness).

```
data Convention = Convention
  { pool     :: Predicate (Wallet, Move) -- who is in the pool
  , basis    :: PeriodBasis              -- when the ratio is struck
  , rounding :: RoundingRule }           -- remainder explicit
attribute :: Convention -> LogPrefix -> Partition Amount
```

12.4 The reconciled surface

An attributed figure is single-sided: it is the firm's own aggregate, and no counterparty computes a matching number to break against. For such figures the correctness standard is Principle 12.1 — totality and determinism — not agreement with a second party, because no second party exists.

One figure on this surface is genuinely two-sided: the collateralisation of net exposure under a governing agreement. It is the specification's single reconciled surface, present by design and named beside the projections, not a concession extracted from them after the fact. The counterparty computes the same figure from the same agreement, and the two are compared at the boundary. It is therefore sourced from the governing agreement unit's declared terms — valuation percentages, thresholds, netting — consistently with the counterparty, and it is named beside the single-sided aggregates in every presentation, so that a reader never mistakes an attribution for a reconciled fact. Where the two sides disagree, the disagreement is a boundary reconciliation item against an external authority — the class the constitution's scope leaves at the perimeter — never an internal break: inside the boundary there is one log and one declared convention.

12.5 Episode: the pledged one-touch in the encumbrance projection

Take OT-1 (Cast, §2.1) pledged to B under agreement G exactly as in §9.8. Before the knock, the encumbrance projection reads, with no store of its own: under G, mass posted 1; price USD 400,000; collateral value $400{,}000 \times 50\% =$ USD 200,000 against an exposure of USD 150,000 — sufficiency holds.

The trigger. OMEGA's official close 79.00 is recorded, at or below the declared barrier; the Event Monitor emits the touch.

The contract. OT-1's contract fires on the event with the current projection of the ledger and proposes one moveless transaction.

The transaction.

1	UnitStatus of OT-1: *live* → *triggered*
2	conditional payment obligation created: writer owes W-ALPHA USD 1,000,000, predicate G's trapping te

No moves; no coordinate changes anywhere.

The door. Conservation holds trivially — there are no moves; writer discipline — the unit's contract is the sole writer of its UnitStatus; a duplicate emission finds the unit already *triggered* and proposes nothing.

The read after. The same projection, one transaction later: mass posted under G is still 1 — a lifecycle event extinguishes value, never mass (§4) — but the pricing layer, parameterised by unit state, now prices the triggered OT-1 at 0, so collateral value reads $0 \times 50\% =$ USD 0 against the unchanged exposure of USD 150,000. The shortfall does not live in the report: it surfaces as G's margin-call obligation at G's declared valuation watch, with a deadline (Chapter 9), and the report shows both the collapsed line and the open obligation from the record alone.

The projection reads mass from the coordinates and value from the pricing layer, so the collapse is on the report the instant it is on the record — and there was no report-side store to go stale.

12.6 Verification and the audit gate

Per the Testability commitment, the projections of this chapter enter the executable-check regime of Chapter 15 as properties over generated logs, agreements, and conventions:

1. *Presentation completeness* — every claim held under a financing agreement appears exactly once as a transferred asset, and its paired obligation unit exactly once as the associated liability.
2. *Encumbrance bound* — per (wallet, unit), reported encumbered mass never exceeds $\max(\text{owned}, 0)$: the coverage invariant of Chapter 14, read back from the report.
3. *Gross, not net* — reported poster-side encumbrance equals $\sum_G \max(\text{bal}_{\text{posted},G}, 0)$, the per-agreement coordinates summed gross, on every generated history — including the cross-agreement branch where a wallet posts under G_1 and receives under G_2 so the signed net is zero (Chapter 15). A report reading the net would pass vacuously there, showing zero encumbrance for a fully encumbered wallet; the gross read catches it. The branch is generated and its firing witnessed, not hoped.
4. *Attribution partition* — attributed parts plus the explicit remainder equal the whole, for every generated log and declared convention.
5. *Re-use bound* — per source agreement, re-pledged mass never exceeds the mass received under it.
6. *Mass–value separation* — a lifecycle event inserted into a generated history changes reported values, never reported mass, exactly as the episode above exhibits.
7. *Recomputation* — every published figure equals the figure recomputed from the log prefix and the boundary join: the reporting analogue of §13's economic verification.

Two readers of one log compute identical reports; the properties above make that sentence executable rather than asserted. One gate remains before the specification freezes: the auditor bench signs the faithfulness of the presentation projection — that the mapped figures state what a fair-value presentation asks — against these property tests. The sign-off is recorded here as a Phase 4 gate of the drafting programme. It confirms that the derivation is faithfully presented; the derivation itself rests on the constitution alone.

13 Alignment with the Common Domain Model

Governed by C-6.5, C-2.6 (§16.5). The industry's Common Domain Model (FINOS CDM, version 6.0.0) maps onto the ledger's declared-data vocabulary; the direction of authority is the constitution's and does not turn. The ledger's objects are derived from first principles; the CDM is exhibited *against* them, never consulted as their source. Where the two models disagree, the disagreement is a finding about the mapping — recorded, and carried by a lineage convention — and never a reason to alter a ledger primitive.

13.1 Unit maps to product; a bespoke instrument maps to the trade

A ledger unit's declared terms map to a CDM product. The counterparty-agnostic template — the CDM `NonTransferableProduct`, whose `EconomicTerms` carry the `Payout` legs, the effective and termination dates, and the notional — is the image of the unit's product-graph terms (Chapter 3). For a listed, fungible instrument this template is the whole image: the same terms serve every holder.

For a bespoke bilateral instrument the faithful image is not the template but the CDM `Trade`. The reason is economic individuation. In CDM the collateral terms live on `Trade.collateral`, whose type `Collateral` carries the `CollateralProvisions`; so two executions sharing one `NonTransferableProduct` but differing in their collateral terms are distinct `Trade`s. The ledger reaches the same conclusion from the other direction and for the same reason: collateral eligibility and the governing regime are declared terms of a distinct agreement unit (Chapter 9), and a unit whose economics are inseparable from those terms is a distinct unit. Unit identity therefore corresponds to the CDM `Trade` layer, not to the `NonTransferableProduct` template — collateral terms individuate a trade economically in both models. The individuation is not the identifier: a CDM trade's identity is carried by its `TradeIdentifier`, and collateral terms individuate the economics that identifier then names. The one structural difference is placement: CDM co-locates the collateral terms — `Trade.collateral`, a `Collateral` carrying the `CollateralProvisions` — inside the `Trade`, whereas the ledger factors the same terms into a separate agreement unit and relates the two by projection. That is a finding about the mapping, carried by the agreement-reference lineage of Chapter 9; it changes no primitive.

13.2 Business event maps to transaction

The CDM `BusinessEvent` — a before-state, a set of `PrimitiveInstruction`s, and the resulting `TradeState` — maps to the ledger's recorded *transaction*, the atomic bundle that advances the fold. The correspondence is structural on both sides. CDM composes a `BusinessEvent` from primitive instructions (quantity change, termination, transfer, reset, observation, and their siblings); the ledger composes a transaction from its four kinds of change (moves, unit-state changes, position-state changes, terms changes). The map from ledger transactions to CDM business events has the shape the constitution already requires of the ledger's own arrows, and it inherits their properties:

- **total** — every recorded transaction has a CDM image, because every transaction is one of the four declared kinds of change and each maps to a primitive instruction or a qualified composition of them;
- **deterministic** — the image is a function of the record alone;
- **composing** — a transaction's four change-kinds map component-wise to the event's primitive instructions, so composition of changes is composition of instructions;
- **conserving** — the ledger's transaction conserves value per (unit, coordinate) by construction; the CDM event does not carry this law, and the ledger supplies it;
- **idempotent under the cause-derived transaction identifier** — each transaction commits exactly once, keyed by its $txid = H(causeEventId, contractId, unitId, seq)$ (Chapter 4);

CDM has no native idempotence key, and the ledger's key is the lineage carried across the map. A retry of one leg collapses to its own *txid*, while the distinct legs of one cascade carry distinct identifiers and all commit.

The division of labour is exact and worth naming: *the ledger keeps the economic substance, the CDM carries the business meaning.* The conserved moves and the state written into the three homes are the substance, guaranteed by the Transaction Executor; the CDM qualification — that this event *is* a partial termination, a reset, a variance fixing — is the meaning, layered on top of substance the ledger has already made unforgeable.

Variance swap. Unit VS-1 (one-year OTC variance swap on IDX, 252 fixings, strike 400 variance points, 1,000 USD per point) maps to a product whose `EconomicTerms` carry one `PerformancePayout`; its underlier is the index IDX as an `Observable`, and the variance computation sits in the payout's `returnTerms` as `VarianceReturnTerms` — the field on which the CDM equity-variance qualification depends. Each of the 252 fixings is a daily log-return $r_i = \ln(C_i/C_{i-1})$ over two consecutive recorded IDX official closes, so the 252-fixing series reads 253 closes — the effective-date reference close C_0 fixed at inception, then closes $C_1, \ldots, C_{252}$, one per fixing date. Each recorded close is a CDM `Observation`, an observed value on the record; the ledger's accrual firing, which writes the accrued sum-of-squared-returns statistic into UnitStatus for the next firing to find, maps to a `BusinessEvent` whose `Reset` lands in the trade's `resetHistory`. The two models agree: each recorded close is a CDM observation, each accrual a recorded business event. Final realised variance 441 points settles $1{,}000 \times (441 - 400) =$ USD 41,000 to the realised-variance receiver: one CDM transfer primitive, one ledger owned-cash move, one conserved fact.

Total return swap. Unit TRS-1 (notional USD 10,000,000, financing 4% quarterly) maps to a two-legged product: a `PerformancePayout` for the reference leg, whose underlier is the NAV of wallet W-QIS as an `Observable`, and an `InterestRatePayout` for the financing leg. The first reset is a CDM `BusinessEvent` carrying a `Reset` (reset value: the stamped NAV observation USD 10,300,000) and a transfer; the ledger records it as one transaction with a single net move of USD 200,000, payer to receiver. The alignment makes a reporting point the netting would otherwise erase: the report derives from the *retained business event* — total-return leg USD 300,000 against financing leg USD 100,000, both recoverable by recomputation from the recorded reset and NAV observation — never from the USD 200,000 net move alone. Substance is the net move on the record; meaning is the gross legs the business event retains.

13.3 Eligibility as declared data restates the CDM collateral model

The ratified collateral ruling makes eligibility and haircuts declared terms of the collateral-agreement unit, checked at the single door and never encoded in a type. This is the CDM collateral model restated in the ledger's primitives. CDM carries eligibility as an `EligibleCollateral` schedule — asset class, currency, and a per-line valuation percentage after the shape of CSA Paragraph 13 — read as data, not as a subtype hierarchy of collateral kinds. The ledger's line-level valuation default (Chapter 9) is the same choice: collateral value is the sum of per-line market value times the declared percentage, package-level valuation admissible only where an agreement declares it. That the two models, derived independently, land on *eligibility as declared data valued line by line* is the alignment this section exhibits; it is shown, not borrowed.

13.4 Closed enumerations and the product graph

Two further alignments serve later chapters. First, the generator universe of Chapter 15 rests only on the enumerations that are genuinely closed. The CDM `Payout` is not one of them: its components *compose* rather than exclude. A product's `EconomicTerms` co-populate one payout

component per leg — `EconomicTerms.payout` has cardinality `(1..*)`, so an interest-rate swap carries two interest-rate legs and a total return swap an interest-rate leg alongside a performance leg — and a product is therefore a composition of payout components, not a choice of one from a fixed arity. The closed enumerations that do seed the generator universe are the day-count fraction, the option type (call or put), and the primitive-instruction set, together with the ledger's own closed sets: the regimes {title-transfer, pledge, STM/CTM}, the coordinate planes {owned, lent, posted}, the trigger and event kinds, and the product-graph node and edge sets closed at registration (Chapter 3). These enumerations seed the generator universe of Chapter 15, so that the property tests range over the industry's own catalogue of product and event shapes together with the ledger's, not an invented one.

Second, the product graph of Chapter 3 — nodes the closed lifecycle states, edges the guarded transitions that emit transaction templates — is the natural translation target for CDM lifecycle representation. A CDM `BusinessEvent` is an edge traversal: the before-`TradeState` is the source node, the after-`TradeState` the target, the primitive instructions the edge's action. The CDM lifecycle and the ledger graph are the same object described twice; the graph is where a CDM event round-trips without loss.

13.5 Gaps

Where CDM 6.0.0 has no native form for a situation the ledger must represent, the mapping states the gap and names the lineage convention the ledger carries instead. None is a defect in the ledger.

Situation	**CDM 6.0.0 native form**	**Lineage convention carried**
Securities-lending recall	None — SL lifecycle events are not primitive instructions	Lent-plane transaction keyed to the loan agreement unit (Chapter 9): open → recall → return
Locate-style pre-trade confirmation	None — CDM is a post-trade model	The locate is a declared obligation on the agreement unit, with deadline and discharge predicate; the pre-trade act itself is out of ledger scope
Collateral re-use / rehypothecation event	Collateral is modelled; re-use is not a first-class event	Posting of mass on the received ray with a mandatory source-agreement reference, from which per-agreement re-use projects
The coordinated cascade (one cause, many units)	None — events are per-trade	The legs share one *causeEventId* — the lineage that ties the cascade together — yet differ in (*contractId*, *unitId*, *seq*), so each carries a distinct cause-derived identifier (Chapter 4): every leg commits, and each exactly once

No gap adds a mechanism: the mapping is total on the ledger's side because the residue is absorbed by primitives the design already carries.

14 The Invariant Catalogue

Governed by C-11.1–11.5, C-12.1–12.5 (§16.5).

The chapters before this one *use* invariants; this chapter states each one once, as an executable property, and discloses the inner ground for it — the reason it holds, not merely the fact that it does. The catalogue divides in two. A **law** holds by construction: no check runs, because the property is a theorem about the fold, and a state violating it is unrepresentable. A **guard** is

enforced at the door: the Transaction Executor computes it on every admission, and a violation is refused, never written. Each property below is stated in the form a test consumes; the executable-check regime of Chapter 15 (Testability and the Executable-Check Regime) exercises all of them over generated products, events, and histories, together with the product-graph consistency invariant of Chapter 3 — restated there, not here.

14.1 Conservation per (unit, coordinate, agreement)

The central law is a conservation law, and it is stated at the finest grain the coordinate vector has: per (unit, coordinate, agreement), not merely per unit or per coordinate. For a wallet w, a unit u, a coordinate $c \in \{\text{owned, lent, posted}\}$, and an agreement G governing the non-owned coordinates, write $\mathrm{bal}_{c,G}(w,u)$ for the balance obtained by folding the log (Section 3.7; Chapter 2). Owned is unqualified, so its agreement is null, and there the finer grain and the per-coordinate grain coincide; on the marker coordinates G is the reference every marker move carries (Chapter 9).

Theorem 14.1 (Conservation)**.** *For every unit u, every coordinate c, every agreement G, and every position in the log,*

$$\sum_{w} \mathrm{bal}_{c,G}(w,u) \;=\; 0,$$

the sum running over all wallets — virtual counterparty and issuer wallets included, because the system is closed (Chapter 3).

Issuance is the first paired-leg owned move out of the issuing wallet; retirement is the log position after which every wallet's vector for u is the zero vector, every (c,G) coordinate zero. Two coarser laws follow as corollaries, never the primitive. Summing over agreements gives conservation per (unit, coordinate), $\sum_w \mathrm{bal}_c(w,u) = 0$; summing further over coordinates gives the per-unit law $\sum_w \sum_c \mathrm{bal}_c(w,u) = 0$. The finest grain is primitive for a reason the encumbrance report makes concrete: a wallet posting 100 under G_1 and receiving 100 under G_2 has $\mathrm{bal}_{\text{posted},G_1} = +100$ and $\mathrm{bal}_{\text{posted},G_2} = -100$, two separately conserved coordinates whose aggregate $\mathrm{bal}_{\text{posted}}$ is 0. Summation is many-to-one: the fully encumbered state and a wholly unencumbered one share that image, so a law stated only on the image cannot tell them apart, and the encumbrance projection built on the image would read zero for a fully encumbered wallet (Section 3.7; Chapter 12). The finer law, and the finer state it certifies, keeps them distinct — that is the exact sense in which it is strictly stronger. The per-coordinate grain in turn forbids a cross-coordinate rebooking that fabricates custody on the posted plane while conserving the gross sum; each grain forbids one class of fabrication the coarser one admits. This theorem is the discharge of the constitution's third admission test for the coordinate schema — conservation at the finest declared grain (C-4.10) — for the (coordinate, agreement) schema Section 3.7 declares.

Proof. Induction on the log. *Base:* the empty log gives every (u,c,G) coordinate a sum of zero. *Step:* the only operation that writes a balance is an atomic move, which debits one wallet and credits another in the same coordinate of the same unit under the same agreement by the same positive quantity q — a marker move carries exactly one agreement reference and an owned move carries none, writing the null-agreement coordinate (Chapter 9) — so it adds $(-q)+(+q)=0$ to that one (u,c,G) coordinate's sum and nothing to any other; the other three kinds of change a transaction bundles — unit-state, position-state, terms — touch no balance. The sum on every (u,c,G) coordinate is therefore invariant across every admitted transaction, and remains 0. □

The per-(unit, coordinate) law is the image of the theorem under summation over agreements: for each (u,c),

$$\sum_{w} \mathrm{bal}_{c}(w,u) \;=\; \sum_{w}\sum_{G} \mathrm{bal}_{c,G}(w,u) \;=\; \sum_{G}\Big(\sum_{w} \mathrm{bal}_{c,G}(w,u)\Big) \;=\; \sum_{G} 0 \;=\; 0,$$

the middle step exchanging two finite sums. Summation carries the finer conserved quantities to the coarser one and so preserves conservation; but it is many-to-one, so the coarser law is strictly weaker, and the finer law is the primitive.

```
conserved :: Log -> Unit -> Coord -> Agreement -> Bool
conserved l u c g = sum [ balance c g w u (fold l) | w <- wallets l ] == 0
-- a law, not a guard: the fold cannot reach a prefix where this is False.
-- owned is the null-agreement coordinate; markers carry their agreement (Ch. 9).
```

The word *conservation* is exact: the vanishing sum is the quantity a symmetry of the log preserves. That symmetry is stronger than conservation alone. A transaction's *footprint* is everything its fold step — the door's admission checks included — reads or writes: it writes the (wallet, unit, coordinate, agreement) balance cells its moves touch and the three homes' targets its other changes touch, and it reads more — a marker move at (w, u) is admitted against the coverage net over *all* agreements (Invariant 14.8), so its read-set takes the whole (w, u) posted and lent fibres with (w, u) owned, *and the declared right-of-use term of every agreement those fibres range over* — the gate reads rou(G) from each source agreement's ProductTerms, so those terms are in the read-set too, and a terms change amending an agreement's right of use shares a footprint element with every guarded move under it: amend-then-re-post and re-post-then-amend reach different states, and the shared footprint keeps that reorder forbidden. An owned reduction symmetrically reads the marker fibres it must not breach. Two moves on one (w, u) under different agreements therefore never have disjoint footprints, though their cells differ — the rehypothecation chain is order-sensitive for exactly this reason. Two adjacent admitted transactions whose footprints — reads and writes alike — are disjoint leave the folded state bit-identical when swapped, not merely every coordinate sum unchanged.

Proposition 14.2 (Independent commutation)**.** *Let t_1 and t_2 be adjacent admitted transactions with disjoint footprints — disjoint read-sets and write-sets, at fibre grain on the guarded coordinates as defined above. Then applying t_1 then t_2 and applying t_2 then t_1 yield the bit-identical* folded state *— balances and the three homes; the log's byte-image differs under the swap, because the hash chain is position-dependent, and is no part of the claim. Transactions that share any footprint element may not be presumed to commute: buy shares before a dividend's record date and the holder is entitled; meet the record date before buying and the holder is not, so the two orders reach different state (Chapter 2).*

The proof is the disjointness: the fold step is a function of exactly the cells it reads and writes, so on disjoint footprints each step sees the same inputs in either order and writes the same outputs, and the two compositions agree cell by cell. This discharges C-2.7's obligation — reorder nothing unless harmlessness is proved — because disjoint footprints are that proof, and the shared-footprint case stays forbidden, the record-date pair its witness. Conservation is the corollary on the balance plane: independent moves share no balance cell, so every coordinate sum survives their swap. Permutation invariance and the vanishing coordinate sum are one fact seen twice; the first is itself a checkable (metamorphic) property.

Episode — the future (T1): variation margin is conservation, not a check. *Trigger.* FUT-IDX (multiplier 10, settled-to-market) prints official settlements 1,000, 990, 1,005 on its three days; W-ALPHA is long one contract, bought at 995 on day 0, against the central counterparty wallet W-CCP. *Contract.* Each print fires the future's contract, which proposes one owned(USD) move between the two wallets (Chapter 9). *Transactions.* Three, one per print:

(1) day 1, 995→1,000: move owned(USD) USD 50, W-CCP → W-ALPHA;
(2) day 2, 1,000→990: move owned(USD) USD 100, W-ALPHA → W-CCP;
(3) day 3, 990 → 1,005 (final): move owned(USD) USD 150, W-CCP → W-ALPHA; the position unwinds.

The door. Each move is paired-leg owned(USD): whatever W-ALPHA gains W-CCP loses, so $\mathrm{bal}_{\mathrm{owned}}(\text{W-ALPHA}, \text{USD}) + \mathrm{bal}_{\mathrm{owned}}(\text{W-CCP}, \text{USD})$ contributed by these moves is zero at every

print. The cumulative variation margin to W-ALPHA is $50-100+150 = 100 = (1{,}005-995)\times 10$, exactly the position's profit and loss (Chapter 7). *Design point. Variation margin is zero-sum because it is conservation per (unit, coordinate, agreement) — here the owned coordinate, whose agreement is null, so the finer grain and the per-coordinate grain coincide; no reconciliation enforces the zero-sum — the paired-leg move makes any other outcome unrepresentable — and the cumulative* 100 *W-ALPHA gains is exactly what W-CCP pays, and equals the profit and loss.*

14.2 The log's structural invariants

Four further properties are laws of the log itself, guaranteed by the single-writer discipline of the Transaction Executor (Chapter 4). Each is stated once, in the form a test consumes.

```
atomic       :: Tx  -> Ledger -> Bool   -- apply writes the whole bundle or
    nothing
appendOnly   :: Log -> Log    -> Bool   -- a later log extends an earlier one; no
    edit
verifyChain  :: Log -> Bool             -- each entry carries its predecessor's
    hash
idempotent   :: Tx  -> Ledger -> Bool   -- re-applying a seen txid is a no-op
```

- **Atomicity.** A transaction applies in full or not at all: the Transaction Executor computes the whole four-kind bundle against the current ledger and either appends all of it or writes nothing. A refusal is a returned value, never a half-applied state — no log position exists at which some but not all of a transaction's changes are present. The split shows it: no instant exists at which 20,000 shares stand beside pre-split terms (Chapter 3).
- **Append-only and hash-chained.** An admitted transaction is never edited or removed; a mistake is repaired forward by a compensating transaction that names what it supersedes and is agreed with the counterparty and carries a recorded human authorisation before it fires (constitution C-12.4; the forward-repair discipline of Chapter 15). Each entry carries the hash of its predecessor, so any alteration of history breaks the chain and is detectable (Chapter 3).
- **Determinism of every arrow.** Each of the three arrows — watch, contract, apply — is a pure function of its recorded inputs (Chapter 2): the same record and clock emit the same events, the same event and ledger propose the same transaction, the same transaction and ledger reach the same ledger. Determinism is the ground on which replay and economic recomputation (Chapter 15) stand.
- **Idempotence.** Every proposed transaction carries the cause-derived identifier $txid = H(causeEventId, contractId, unitId, seq)$ of Chapter 4; the door keys on the *txid*, so a retried or late-arriving proposal recomputes the identical identifier and is committed once, while the distinct legs of one cascade carry distinct identifiers and all commit. Applying a transaction whose identifier is already on the log changes nothing.

14.3 Consistency of reference

A quantity and a price name value only when both refer to the same state of the instrument. All recorded data carries the basis frame it was observed in (Chapter 8); the guard refuses a read that would combine a quantity with a price drawn from a different frame.

Invariant 14.3 (Consistency of reference)**.** *A quantity Q and a price P may be combined only when Q and P are read in the same corporate-action frame. Across an event carrying a quantity change (a move) and a value-frame change (an operator), total value is preserved: $Q_{pre} \times P_{pre} = Q_{post} \times P_{post}$. This binding is the* invariance witness*, and a product taken across mismatched frames is refused as a phantom.*

Episode — the split (T4): the phantom the witness refuses. *Trigger.* ACME's 2-for-1 split is effective; W-ALPHA's 10,000 shares become 20,000 by a paired-leg move, and the recorded price frame reads 100.00 as 50.00 under the split's operator (Chapter 8). *The witness.* Before: $10{,}000 \times 100.00$ = USD 1,000,000. After, in the post-split frame: $20{,}000 \times 50.00$ = USD 1,000,000. The phantom is $20{,}000 \times 100.00$ = USD 2,000,000 — the new count against the stale frame. *The door.* The 20,000 is stamped in the post-split basis and the 100.00 in the pre-split basis; the frames do not match, so the product is refused. The count changes by a move, the frame by an operator, and the two are bound so their product is invariant. *Design point.* 20,000× *stale* 100.00 *is a phantom USD 2,000,000 against a true USD 1,000,000; the witness refuses it because the quantity and the price are read in different frames — the same witness Chapter 8 leans on for every corporate action.*

The same witness governs a distribution's cum/ex boundary, where no quantity changes at all. A *cum* price includes the coming distribution; an *ex* price excludes it; the two are one data kind read in states separated by the recorded ex-date event, whose declared operator subtracts the distribution's value. A cum price consumed as ex, or an ex price consumed as cum, is a phantom of the split's kind — the right number in the wrong frame — and the same witness refuses it, because the frame is fixed by the recorded ex-date state, not the wall clock. Chapter 8 cross-references this invariant for both readings.

The witness has a third reading, for a unit whose terms reference another and that carries no move at all. A derivative — OT-1's barrier on OMEGA, an option's strike, a single-name contract's multiplier — changes no holder's quantity when its underlying acts: the holder holds one contract before and after. Here the witness is over the *declared terms*: across a corporate action on a referenced underlying, the payoff read on the post-event term frame equals the payoff read on the pre-event frame, and the binding is the term-adjustment operator that the corporate-action out-edge appends as a new terms version (Chapters 3 and 8). Under OMEGA's two-for-one split OT-1's barrier is carried $80.00 \to 40.00$ and its USD 1,000,000 payout stands; the phantom is a knock read against the stale 80.00, on which the first post-split close ≈ 47.50 fires a payout on a stock that never fell, while against the adjusted 40.00 no knock fires. Quantity-times-price for a holding, and payoff-on-terms for a derivative, are the two readings of one witness: value is invariant across the event, and a stale frame — whether a stale price or a stale term — is refused.

14.4 Writer discipline

Invariant 14.4 (One legal writer)**.** *Every non-balance fact has exactly one legal writer — the smart contract of the instrument the fact belongs to — and the Transaction Executor refuses a transaction that writes a fact outside its writer's authority (Chapter 6, constitution §11).*

Each of the three homes — ProductTerms, UnitStatus, PositionState — is a *materialised projection*: a cached, one-writer, rebuildable read of the log, provably equal to the fold it caches, and *not* a second store of any derived quantity. No figure a valuation or a report recomputes is materialised anywhere — NAV, profit and loss, availability, encumbrance are computed on demand (Chapters 7 and 12) — so there is nothing derived to reconcile against the record. Writer discipline over facts, and the refusal to store views, are jointly the condition under which replay reproduces the homes exactly.

14.5 Time travel and replay

Section §12's guarantees are theorems of the fold, and the fold runs along two axes, not one. C-12.1 names both honest answers and delegates their indexing to this specification; the two axes are that indexing.

Theorem 14.5 (Time travel)**.** *Every historical state of the ledger is the fold of the log along two axes: a door-time prefix p — how far down the immutable log the reconstruction reads, the axis of what was known — and an execution-time horizon h — which recorded observations are in force, the axis of what those observations say (Chapter 2). Fixing the pair (p, h), the ledger is* fold step *over the committed transactions of p with h's observations in force, and every view — a balance, a NAV, a report — is a further fold of that same pair. Two facts hold.* (a) As-known-at permanence: *the view at (p, h) is never rewritten by a correction admitted after p, because such a correction is a transaction appended strictly beyond p and the prefix is fixed.* (b) As-of exactness: *let p' be the prefix that includes a back-dated observation recorded at execution-time index k, and h' the horizon putting it in force; the view at (p', h') recomputes bit-exactly from p', because the fold reads only committed transactions, never re-executes contracts, and is deterministic. The historical state is thus independent of the machinery that produced it and reconstructible at any time, forever.*

Proof. The ledger is the fold of the log (Chapter 2); restricting to a prefix folds the transactions that prefix selects, and determinism of apply (the determinism of every arrow, above) makes the result a function of those transactions alone. *(a):* fix (p, h). A later correction is an admitted transaction at a log position strictly beyond p (append-only, above), so it is not among the transactions p selects; a function of a fixed argument does not change when a value outside that argument is added, so the view at (p, h) is unchanged. *(b):* the back-dated observation enters p' as a recorded observation carrying its execution-time index k. Folding p''s observations in execution-time order re-derives the order-dependent accrual tail $A_k, A_{k+1}, \ldots$ from k forward; every input to that fold — the recorded closes, the accrued integers (Chapter 6) — is on the record, so the result is a function of p' alone and reproduces bit-for-bit. □

The as-of fold of clause (b) is not the log-order fold of the next theorem, and the difference is the crux of the two axes. Replay folds the committed transactions of a prefix; the as-of view additionally orders the recorded observations by their execution-time index, so a back-dated observation — admitted late in door time, hence last in log order — is folded at execution-time index k, re-sequencing the tail $A_k, \ldots$ from k forward. The two folds traverse the same observations in two different orders, and each is a deterministic function of the prefix; a party recomputes either one and obtains the same bits. Confusing them — folding a back-dated close in log order, or asserting the log-order replay already carries the as-of answer — is exactly the single-axis error the discharge below closes.

Theorem 14.6 (Replay determinism)**.** *Discarding the ledger and the three homes and replaying a log prefix reproduces them bit for bit. The apply arrow is deterministic (the determinism of every arrow, above), each home is a materialised projection with one legal writer (Invariant 14.4), and a retried or late proposal is idempotent under its cause-derived identifier; so replay adds nothing and drops nothing.*

Proof. The ledger is defined as the fold of the log (Chapter 2); restricting to a prefix folds fewer transactions, and determinism of apply makes the result a function of the prefix alone. Each home is the fold of its own kind of change over the same prefix (Chapter 6), hence reproduced by the same replay. Idempotence guarantees a duplicated cause contributes once. □

These two theorems are the executable ground of the constitution's Full Reproducibility and Time-Travel commitments. Chapter 15 (Testability and the Executable-Check Regime) exercises them — regenerating a mid-life firing from the recorded fixings and comparing, and inserting a back-dated observation to witness that the as-known-at view holds while the as-of view recomputes — and this catalogue does not restate the test.

The discharge. Constitution C-12.1 (v1.3) names both answers — what the book said then, never rewritten by later arrivals, and the numbers with a later correction in force —

as deterministic replays of the one log, and delegates their indexing to this specification. Theorem 14.5 is that discharge: the door-time prefix and execution-time horizon are the indexing the constitution now leaves here, and the two both-correct answers are its two clauses.

14.6 The authorised fork

The forward repair of Chapter 15 corrects a wrong *event*. It does not answer a corrupted *record* — a log damaged by systemic failure, where the bytes themselves are no longer trustworthy. The remedy for that is the **authorised fork** (constitution C-12.5): replay to a chosen point, and rebuild forward on a new *lineage* that names its parent point. This is the rollback of a bad delivery, not a rewrite of history; the abandoned lineage is retained, never erased, so the log is append-only *across* lineages as well as within one.

The fork is a recorded, authorised fact: the new lineage names its parent point and carries a recorded human authorisation — the forward-repair gate of Chapter 15, counterparty agreement and personal authorisation alike, raised from a single move to a whole lineage — so which lineage is authoritative is itself a recorded fact, and exactly one is authoritative at any position. The primitive is shared with the simulated ledger (Chapter 10, Section 10.5): replay to a point, build forward on a fresh lineage. A simulated ledger branches to *explore* — never authoritative, nothing settles; a fork branches to *repair* — authoritative by the recorded authorisation. One mechanism, two readings.

Invariant 14.7 (Lineage append-only)**.** *No committed transaction of any lineage is ever destroyed. A fork adds a lineage and an authorisation and removes no byte; exactly one lineage is authoritative at any position, and which one is itself a recorded fact.*

14.7 The record is the log

Replay reconstructs more than balances and the three homes. Every guaranteed observation enters the log as a moveless observation-recording transaction (Chapter 8), so replaying a prefix reconstructs every recorded observation as well — the record *is* the log, no second store, and the single-writer discipline (Invariant 14.4) covers every observation. Chapter 15 exercises this as `prop_replayReconstructsRecord`, rebuilding the whole record from the log alone. The observation stream is no second half of the record with a door of its own: there is one door, and it is the Transaction Executor.

14.8 The coverage invariant and the two-guard contrast

Collateral introduces one guard and one obligation that look alike and are structurally opposite.

Invariant 14.8 (Possession coverage)**.** *No wallet delivers what it neither owns nor holds, and no wallet re-posts received mass its source agreement does not release. On every admission, per wallet w and unit u,*

$$\sum_{G} \max\big(\mathrm{bal}_{\mathrm{posted},G}(w,u),\,0\big) \;+\; \sum_{G:\,\mathrm{rou}(G)} \min\big(\mathrm{bal}_{\mathrm{posted},G}(w,u),\,0\big) \;\le\; \max\big(\mathrm{bal}_{\mathrm{owned}}(w,u),\,0\big).$$

The sign convention on the posted coordinate is received-negative*: collateral posted* out *enters with positive sign, collateral received* in *enters with negative sign (the received ray, poster $+Q$, taker $-Q$; Chapter 9). Posted-out mass — the positive part — always counts; received mass — the negative part — offsets, and so grants re-posting headroom, only under an agreement whose declared terms grant a right of use,* $\mathrm{rou}(G)$ *read at the door from the source agreement G like every other declared term. Received mass under a no-right-of-use agreement — a segregation, Constitution C-8.8 — contributes zero: it grants no headroom and cannot be re-posted. This is* ***the right-of-use gate*** *— not a second guard beside coverage but the one coverage net read under one qualifier. The analogue holds with lent in place of posted.*

The gated net lets received mass offset re-posted mass *only where the source agreement releases it.* Worked once, for citation from Chapter 9: take a wallet with owned 0. It receives 100 under a right-of-use agreement, so its posted coordinate there is -100 and, the agreement granting use, that -100 counts; it re-posts 60 under another, coordinate $+60$; the net is $60 - 100 = -40 \leq \max(0, 0) = 0$, within the bound, so a rehypothecation chain passes and is bounded by possession. Had the same wallet re-posted 200 instead, the net would be $200 - 100 = +100 > \max(0, 0) = 0$ — refused at the door: a wallet may re-post no more than it received under a releasing agreement plus what it owns. Had the 100 instead arrived under a *no-right-of-use* agreement — a segregation (C-8.8) — that received -100 contributes zero, so re-posting even the same 60 gives a net $60 + 0 = +60 > 0$ and is refused: segregated mass grants no headroom. The max binds only the posting direction, so a negative-owned wallet with nothing posted — an issuer wallet that conservation itself creates, a plain short — is admissible, while a poster with owned $-5{,}000$ attempting to post 5,000 is refused at the door. Coverage is never lawfully false, because a transaction is atomic and possession is instantaneous within it: that is what makes it an invariant.

Coverage binds *instructed* ownership, and the qualifier is load-bearing. Owned is booked at instruction — trade-date (Chapter 9) — so a wallet may post or deliver a security it has bought but not yet settled: that state is reachable, admitted at the door because instructed ownership covers it. The invariant forbids only *recording* a state whose posted mass exceeds instructed ownership. A later involuntary reduction of owned — a failed or cancelled purchase that would drop owned below the mass already posted — is therefore not a silent breach but a transaction *refused at the door*: it cannot be recorded while it would violate coverage. The unmet return is carried instead as a recorded, deadline-bearing obligation (Chapter 9), and on default it escalates to the supervised write-off path there. The fails cascade thus lives inside the model — order-forced at the door, then obligation, then supervised write-off — and the invariant is never false in any committed state.

On fungible cash coverage is a per-move check: a cash posting is admitted against owned cash at the instant it is recorded, and because the same owned USD backs every obligation drawn on it, aggregate cash adequacy across all obligations is not coverage at all but the sufficiency obligation — a deadline, not an invariant.

Guard	Statement	Lawfully false?	Kind
Possession coverage	$\sum_G \text{bal}_{\text{posted},G} \leq \max(\text{bal}_{\text{owned}}, 0)$, received mass offsetting only under right of use (the right-of-use gate)	never (atomic, instantaneous)	invariant, checked at the door
Value sufficiency	collateral value $\geq$ exposure	intraday, routinely	obligation: deadline, discharge predicate, close-out as compensation

Value sufficiency is the guard coverage is never to be confused with: an intraday shortfall with no party in breach is an open, deadline-bearing item, not a broken state (Chapter 9, §14.1).

Episode — the one-touch (T2): coverage on the discharge; value extinguished, mass intact. *Setup.* OT-1 (Cast, §2.1) pledged to B under agreement G as in §9.8 (collateral value to B USD 200,000 against exposure USD 150,000). OMEGA prints its barrier close, $79.00 \leq 80.00$. *The discharge transaction.* The payout obligation discharges under its trapping predicate in one transaction, two moves: owned(USD) USD 1,000,000 W-BANK $\rightarrow$ W-ALPHA, and posted(USD) USD 150,000 W-ALPHA $\rightarrow$ B under G; USD 850,000 stays free. *The door.* Coverage holds within the transaction: W-ALPHA's owned cash 1,000,000 $\geq$ posted 150,000, so the posting is

admitted — a posting of collateral W-ALPHA did not possess would be refused. *Value versus mass.* The knock extinguishes value, never mass: pricing the triggered OT-1 at 0 collapses its value, but its coordinate balance is unchanged — coordinates carry mass, prices carry value. Retirement is only from the zero vector: the marker return clears the posted plane of OT-1 first — coverage forces the order, refusing to extinguish owned while posted mass remains — then owned(OT-1) moves W-ALPHA → W-BANK, taking every wallet's vector for OT-1 to zero, and OT-1 retires. *Design point. Coverage is checked at the door on the discharge (owned* 1,000,000 ≥ *posted* 150,000*) and again as the order of close-out; the lifecycle event extinguishes OT-1's value but not its mass; and between knock and call the intraday shortfall is carried by the sufficiency obligation, not by a broken invariant.*

14.9 The post-and-return metamorphic property

The last property makes "only the owned coordinate carries economic value" (Chapter 3, constitution §4) machine-checkable.

Proposition 14.9 (Post-and-return)**.** *A pledge posting writes the posted plane and never owned, so* $V = \sum \text{owned} \cdot P$ *is unchanged by any post-and-return round trip. Inserting a post-and-return pair within any watch-free interval of the referenced agreement leaves every later valuation identical.*

The side condition is real and named: an insertion spanning the agreement's valuation watch lawfully changes later firings and cash, so the property owns the move algebra, not the watched case. The watched case is the temporal residual carried by the safety and liveness model of Chapter 15 (Testability and the Executable-Check Regime). As a test the proposition is metamorphic: insert the pair into a generated history and assert that every subsequent NAV is unchanged to the minor unit.

14.10 What the catalogue buys

The properties above, with the product-graph consistency invariant of Chapter 3, are the surface Chapter 15 tests over one generator universe. Their joint effect is the constitution's claim in §11: the classes of inconsistent state that internal reconciliation exists to detect — unbalanced books, a fact written twice into disagreement, a quantity valued in the wrong frame, collateral posted in excess of the poster's instructed ownership — are not caught here; they are unreachable. The residual economic exposures that remain — a bought-not-yet-settled posting whose purchase later fails, an intraday shortfall of value against exposure — are reachable and real, but the model carries them as recorded, deadline-bearing obligations (Chapters 9 and 11), never as silent breaches of an invariant.

15 Testability and the Executable-Check Regime

Governed by C-13.1–13.3, C-2.5, C-4.12 (§16.5). The ledger is correct because it is built to be tested. Every invariant catalogued in Chapter 14 is an executable check run over generated products, events, and histories, never a property defended only in prose; and correctness composes, because the steps are pure and their composition is fixed — the map, then the fold — so verifying each part verifies the whole. This chapter does not restate the invariants. It makes them the surface a test regime exercises, states the regime, and turns the specification's three binding conditions into executable gates rather than intentions.

15.1 Two layers of correctness

Correctness rests on two layers, and each is checked by its own means.

Structural correctness holds by construction, at the door. An illegal state is not detected and rejected; it is unrepresentable — no program writes it. A balance changes only through a move (Chapter 3); a marker coordinate is unwritable without its governing agreement reference (Chapter 9); a knocked note has no un-knock constructor (Chapter 3, Invariant 3.2); conservation is automatic because every move is paired-leg and the system is closed. These properties hold whatever the contract that proposed the transaction did — correct, careless, or malicious — because the Transaction Executor admits on structure alone and is the sole writer (Chapter 4). The test obligation for this layer is negative: no generated event, however adversarial, drives the ledger into a state the type discipline forbids or the door should refuse.

Economic correctness is not guaranteed at admission; it is made checkable by recomputation. A contract may emit a perfectly well-formed transaction that books the wrong economics. The check is to re-run the map on the recorded event and compare the result with what was recorded.

```
-- the economic oracle: re-run the map on the recorded event and compare
prop_recompute :: Recorded (Event, Transaction) -> Property
prop_recompute (e, txRecorded) = contract e (ledgerBefore e) === txRecorded
-- a mismatch is a defect of the CONTRACT, not the ledger; it is repaired FORWARD
    by an
-- authorised compensating transaction through the same door -- never by an edit
    to history.
```

Purity is what makes this oracle exist at all: a contract reads only the event and the record (Chapter 5), so its recomputation is exact and unconditional. A mismatch is a defect of the contract, never of the ledger, and it is repaired forward the way every money change is made — by an authorised compensating transaction through the same door (the forward-repair discipline below). The regime-bit repair of Chapter 9 (a new terms version *plus* the authorised compensating rebooking of the affected interval) is the worked instance; the valuations of Chapter 7 are recomputable under the same discipline. The two layers divide the labour exactly and neither substitutes for the other: structure is proved once and tested negatively, economics is re-derived per recorded event and tested by comparison, and a structurally valid transaction can still be economically wrong.

Forward repair: the correction event and the authorisation gate. A wrong recorded event — a wrong price, a wrong payoff — is repaired forward, never edited (constitution C-12.4). The discovery is itself a recorded *correction event* that names the wrong event's identifier, admitted through the one door like any event. It mints a visible, deadline-bearing *open item* — a deadline, a discharge predicate, and a fallback, the obligation pattern of Chapter 14 — so the discrepancy stands on the record until repaired. Two things then part company. A *view* recomputes automatically: the as-known-at projection stands, the as-of projection recomputes over the corrected prefix (Theorem 14.5). *Money* does not: because money already moved cannot be unsent, a compensating transaction that repairs a balance is agreed with the counterparty and authorised by a person before it fires. The authorisation is a recorded observation; the compensating transaction references both the correction event and its authorisation, or the door refuses it. In phase 0 no correction fires without human intervention.

```
data Correction    = Correction    { wrongEventId :: EventId,  reason   :: Text }
-- the discovery: a moveless recorded event, through the one door, naming the
    wrong event.
data Authorisation = Authorisation
  { corrects :: EventId, approver :: PersonId, counterpartyAssent :: ObservationId
    }
-- a recorded approval carrying BOTH C-12.4 conditions: agreed with the
    counterparty
-- (the assent, a recorded observation) and authorised by a person (the approver).
compensating :: CorrectionId -> AuthorisationId -> [Move] -> Transaction
```

```
-- door-checkable definitions: isCompensating = carries a Correction (wrongEventId
    )
-- reference; hasAuthorisationRef = references an Authorisation whose corrects
    field
-- names that wrongEventId, carrying approver AND counterpartyAssent.
-- the guarantee is the DOOR's, not the constructor's -- an untrusted proposer can
-- bypass `compensating` -- so the property asserts the refusal:
prop_noRepairWithoutAuth :: Transaction -> Property
prop_noRepairWithoutAuth tx =
  isCompensating tx && not (hasAuthorisationRef tx) ==> refusedAtDoor tx
genUnauthorisedRepair :: Gen Transaction  -- ADVERSARIAL: reference a correction,
    omit
  -- the authorisation, bypass the constructor -- as the untrusted Executor could.
prop_noRepairAuth_fires =                 -- the refusal witness, asserted, not
    hoped
  checkCoverage $ cover 1.0 unauthorisedRepair
    "correction-referencing repair without authorisation -> refused"
    genUnauthorisedRepair
```

A view is free because it stores nothing and settles nothing; a compensating transaction moves owned mass, so it passes the same human gate every real move passes.

15.2 The generator universe and the adversarial scheduler

The enumerations this specification builds on are genuinely closed. On the ledger's side: the product-graph node and edge sets — the lifecycle states and guarded transitions — fixed at registration (Chapter 3); the collateral regimes {title-transfer, pledge, STM/CTM} and the coordinate planes {owned, lent, posted} (Chapter 9); the component dimensions of the data-kind registry and the boundary failure modes W1–W4 (Chapter 8); and the trigger and event kinds of a contract (Chapter 5). On the CDM side the genuinely closed enumerations are the day-count fraction, the option type (call or put), and the primitive-instruction set (Chapter 13). These are closed choices, and they seed the generator universe; a co-populated, cardinality-many composite is not a closed choice and is not one of them. Because the closed enumerations are closed and declared, one generator universe ranges over the whole domain, and its coverage of each enumeration is itself a checkable fact — every inhabitant is reachable by the generators, so a property that never exercises a case is a visible gap, not a silent one.

```
genProduct :: Gen ProductGraph          -- a well-typed graph over the closed node
    /edge set
genEvent   :: ProductGraph -> Gen Event -- on-menu guards, and off-menu
    adversarial events
genHistory :: Gen [Transaction]         -- interleaved admitted walks of many
    products
-- every generator is paired with a shrinker: a violation shrinks to a minimal
    history,
-- and every counterexample is replayable from its seed (a defect is a
    reproduction).
```

This generator regime is the simulated ledger of Chapter 10 (Section 10.5) run under the test measure: the same branch-point, seed, and one-door discipline, aimed at refutation rather than pricing.

Generation is adversarial by design, because the test environment must be harder than production. The Event Monitor and the Events Executor are outside the trust boundary and may duplicate, delay, and reorder firings; integrity owes them nothing (Chapter 4). The scheduler therefore explores every legal interleaving the Transaction Executor's total order permits — duplicated proposals, late firings, events delivered out of arrival order — and injects pathological-but-legal behaviour rather than only the happy path: this is precisely what exercises idempotence

and the temporal model below. One discipline guards the generators against vacuity: a property whose precondition is never generated passes without witnessing anything. A generated history that never knocks a pledged note, never re-posts received collateral, never delivers a duplicate is not evidence; the generators are measured by the states they reach, not the cases they cover. Throughout this chapter each property's firing witness asserts its branch at $\geq 1\%$ of generated histories under `checkCoverage`; an unwitnessed branch is a *failed* run, not a silent green, by §2's zero-firings rule. This standing gate is not restated per property.

15.3 The test surface: the invariant catalogue and the product graph

The invariants of Chapter 14 are the test surface, in a ramp from the universal to the domain-specific. At the base sit the properties no history may violate: no transaction half-applies (atomicity), no balance is created or destroyed off the paired-leg move (conservation per (unit, coordinate, agreement)), the log is append-only and hash-chained, and every arrow is deterministic. Above them sit the structural invariants — consistency of reference, writer discipline — and above those the safety properties of the collateral planes: coverage is an invariant, checked at the door on every admission, while sufficiency is an obligation, lawfully false intraday and bounded by a deadline, and the test regime must assert the first as an invariant and the second only as a liveness obligation. Stating sufficiency as an invariant would be a false theorem the generators refute at once, and insisting on it would forbid lawful states: the two guards are never collapsed, in the tests least of all.

Conservation at agreement grain, and the netting collapse it catches. Conservation is checked at the finest grain the coordinate vector has — per (unit, coordinate, agreement), Theorem 14.1. The teeth are the cross-agreement history: post 100 under G_1, receive 100 under G_2, aggregate posted axis zero. A property reading only the aggregate sees zero and passes *vacuously*, reporting a fully encumbered wallet as unencumbered (Chapter 12); the finer law keeps $+100$ and -100 as two coordinates, and the gross-encumbrance property reads them as 100, not 0.

```
prop_conservation :: History -> Unit -> Coord -> Agreement -> Property  -- Theorem
    14.1
prop_conservation h u c g = sum [ balance c g w u (fold h) | w <- wallets h ] ===
    0

prop_encumbranceGross :: History -> Wallet -> Unit -> Property          -- Ch. 12,
    gross
prop_encumbranceGross h w u =
  reportedEncumbrance h w u === sum [ max (postedG h w u g) 0 | g <- agreements h
    ]
  -- the signed net across agreements is NEVER the encumbrance figure.

genCrossAgreement :: Gen History  -- draw a netting-flag with rate r; when it
    fires,
  -- post q of u under G1 (bal_posted,G1 = +q) and receive q of u under G2 (= -q),
  -- so the AGGREGATE posted axis is 0 while the two coordinates are +q and -q.
prop_netting_fires =                      -- the firing witness, asserted, not
    hoped
  checkCoverage $ cover 1.0 isCrossAgreementNetting
    "post under G1 and receive under G2 (aggregate posted = 0)" genCrossAgreement
```

The netting branch is a genuine precondition, not a tautology: a generator that never posts and receives the same unit under two agreements would satisfy `prop_encumbranceGross` without ever witnessing the collapse it exists to catch. `genCrossAgreement` constructs the branch by construction, and its firing witness asserts it (§2). On every history that clears

the precondition, `prop_conservation` fires on each of G_1 and G_2 and finds each coordinate conserved, and `prop_encumbranceGross` fires and finds the reported encumbrance 100 where a net-reading report would show 0 — the single-axis property passing vacuously while the agreement-grain pair catches the collapse.

The coverage sign convention, and the over-re-post it refuses. The rehypothecation bound nets the signed posted coordinates against possession (Invariant 14.8), and the whole bound rests on one sign convention: on the posted coordinate collateral posted *out* is positive and collateral received *in* is negative — received-negative. Get the sign backwards and the bound inverts: a wallet that received collateral would appear able to re-post it without limit. The property asserts the convention by asserting the refusal it forces. Take a wallet with owned 0; it receives 100 under one agreement (posted coordinate -100) and attempts to re-post 200 under another ($+200$). The net is $200 - 100 = +100 > \max(0, 0) = 0$, so the admission is refused at the door; under the inverted sign the same net would read $-100 \leq 0$ and admit — the exact defect the property catches. The precondition — a zero-owned wallet that receives and then re-posts *more* than it received — is a genuine one, not a tautology.

```
prop_coverageSignConvention :: History -> Wallet -> Unit -> Property  -- Invariant
    14.6
prop_coverageSignConvention h w u =
  admitted h ==> coverageNet h w u <= max (ownedBal h w u) 0
  where coverageNet h w u = sum [ postedG h w u g | g <- agreements h ]
  -- received IN contributes NEGATIVE, posted OUT contributes POSITIVE; here every
     received
  -- coordinate is under a right-of-use agreement, so this plain sum equals the
    gated net of
  -- Invariant 14.6 (no-right-of-use received mass would instead contribute 0; see
     the gate below).

genOverRepost :: Gen History  -- draw a re-post-flag with rate r; when it fires,
    take a
  -- wallet with owned 0, receive 100 of u under a RIGHT-OF-USE agreement G1 (
    bal_posted,G1 = -100;
  -- the gate lets it count), then attempt to re-post 200 of u under G2 (
    bal_posted,G2 = +200):
  -- net +100 > max(0,0)=0, REFUSED. (G1 must grant use, else -100 contributes 0
    and the
  -- sign-convention inversion below has nothing to invert.)
prop_coverageSign_fires =                  -- the refusal witness, asserted, not
    hoped
  checkCoverage $ cover 1.0 isOverRepostOnZeroOwned
    "owned 0, received 100, re-post 200 -> net +100, refused" genOverRepost
```

When the re-post-flag fires the generator constructs the over-re-post by construction — owned 0, received 100, re-post attempt 200 — and its refusal witness asserts the branch (§2). Flip the convention to received-positive and the refused net $+100$ reads $-100 \leq 0$ and admits: `prop_coverageSignConvention` reports exactly that inversion, while the lawful chain of Invariant 14.8 — received 100 *under a right-of-use agreement*, re-post 60, gated net $-40 \leq 0$ — stays admitted.

The right-of-use gate refuses a no-right-of-use re-post. The coverage net offsets a re-post by received mass, but only received mass whose source agreement grants a right of use (Invariant 14.8). Received mass under a segregation contributes zero, so a re-post the plain sum would "cover" is refused. The property asserts the gated net at the door.

```
prop_rouGateRefusesSegregated :: History -> Wallet -> Unit -> Property  --
    Invariant 14.6, gated
prop_rouGateRefusesSegregated h w u =
  admitted h ==> gatedNet h w u <= max (ownedBal h w u) 0
  where gatedNet h w u = sum [ contrib g | g <- agreements h ]
        contrib g | b >= 0        = b            -- posted OUT: always counts
                  | rou (agmt g)  = b            -- received under right-of-use:
    offsets
                  | otherwise     = 0            -- received under segregation: NO
     headroom
                  where b = postedG h w u g

genSegregatedRepost :: Gen History  -- draw a gate-flag with rate r; when it fires
    : owned 0, receive
  -- 100 of u under a CUSTODY (no-right-of-use) agreement G_seg (bal_posted,G_seg
    = -100), then attempt
  -- to re-post 60 under G2 (+60): gated net = 60 + 0 = +60 > max(0,0)=0, REFUSED
    at the door.
prop_rouGate_fires =                    -- the refusal witness, asserted, not hoped
  checkCoverage $ cover 1.0 isSegregatedRepostOnZeroOwned
    "owned 0, received 100 segregated, re-post 60 -> gated net +60, refused"
    genSegregatedRepost
```

When the gate-flag fires the generator builds the segregated re-post by construction — owned 0, received 100 under custody, re-post attempt 60 — and the refusal is witnessed on at least 1% of runs (§2). Flip the mass to a right-of-use agreement and the identical 60 admits: the declared bit alone decides.

The right-of-use gate admits a bounded re-use. Under a right-of-use agreement the received −100 counts, so a re-post within the bound is admitted — the certified −40 chain, now explicitly under a releasing agreement.

```
prop_rouReuseAdmittedBounded :: History -> Wallet -> Unit -> Property
prop_rouReuseAdmittedBounded h w u =
  admitted h ==> gatedNet h w u <= max (ownedBal h w u) 0
  -- same door invariant; here the received coordinate IS under a right-of-use
    agreement.

genRouReuse :: Gen History  -- owned 0, receive 100 of u under a FINANCING (right-
    of-use) agreement G1
  -- (bal_posted,G1 = -100), re-post 60 under G2 (+60): gated net = 60 - 100 = -40
     <= 0, ADMITTED.
prop_rouReuse_fires =               -- the admission witness, asserted, not hoped
  checkCoverage $ cover 1.0 isRouReuseWithinBound
    "owned 0, received 100 RoU, re-post 60 -> gated net -40, admitted" genRouReuse
```

The same 60 re-post carries opposite verdicts under the two regimes — −40 admitted under right of use, +60 refused under segregation — and only the declared right-of-use term differs; the admission branch fires on at least 1% of runs (§2).

A refused re-post leaves the re-use projection unchanged. A refusal appends nothing to the log, and the Chapter 12 re-use figures fold committed state alone. So a refused segregated re-post cannot move any re-use figure: there is no second computation to disagree, and the check needs no oracle.

```
prop_refusedReuseNoWrite :: History -> Property  -- refused = no write;
    projections read committed state
prop_refusedReuseNoWrite h =
```

```
  reuseProjection h === reuseProjection (h `afterAttempting` segregatedRepost)
  -- the attempt is refused at the gate, writes nothing, so the re-use figure is
    byte-identical.
```

The projection is invariant under a refused attempt by construction: refusal writes no transaction, and a fold over an unchanged log returns an unchanged figure — the property is oracle-independent (§2).

Carried onto this surface are the four properties of the product-graph consistency invariant (Invariant 3.2, defined in Chapter 3), stated here as executable checks over generated products and histories:

```
-- for every unit, at every commit, the three interpreters of its graph agree:
prop_watchesAreOutEdges   :: History -> Property  -- (i) watch list == out-edges(
    current node)
prop_firedMatchesOneEdge  :: History -> Property  -- (ii) fired event -> exactly
    one edge, lands on its target
prop_actionEqualsRecorded :: History -> Property  -- (iii) emitted tx == edge
    action applied to recorded inputs
prop_traversalExpiresSibs :: History -> Property  -- (iv) traversal expires
    exactly the declared siblings
```

Property (i) is checked at every log position: the Event Monitor's live watch set is folded from the record and compared with the out-edge set of the unit's current node. Property (ii) rejects any firing that matches no edge or more than one, or that lands the unit off the edge's declared target. Property (iii) is the graph-level face of the economic oracle above: the transaction the contract emitted must equal the edge's declared action applied to the recorded inputs. Property (iv) walks the sibling set and confirms that a traversal expires those edges and no others. The graph declares watches, state schema, and actions from one object; these four properties are the statement, made executable, that its three interpreters never disagree.

Term adjustment on a corporate action, and the phantom knock it refuses. Across a split of a referenced underlying, the corporate-action edge (Section 3.8) appends a new ProductTerms version through the term-adjustment operator (Chapter 8), and it carries its own executable witness (Invariant 14.3): the payoff read on the appended terms equals the payoff on the pre-event terms, and no guard fires that the pre-event terms would not have fired. The precondition — a generated derivative *and* a generated corporate action on its referenced underlying — is genuine, not a tautology.

```
prop_termAdjustmentInvariance :: History -> Property
prop_termAdjustmentInvariance h = hasCAonUnderlying h ==>
       payoffOn (postEventTerms h) (postEventFrame h)
   === payoffOn (preEventTerms  h) (preEventFrame  h)      -- payoff invariant
    across the CA
  .&&. not (spuriousGuardFires h)  -- no edge fires that the pre-event terms would
     not

genDerivWithCA :: Gen History
-- 1. draw a derivative whose terms REFERENCE an underlying (OT-1's barrier on
    OMEGA);
-- 2. draw a CA-flag with rate r; when it fires, emit a two-for-one split of that
    referenced
--    underlying inside the derivative's live window, then record the next
    underlying close
--    in the GAP between the adjusted and the stale barrier (e.g. 47.50, with
    80.00 -> 40.00)
--    -- the exact phantom-knock witness the edge must refuse.
```

```
prop_termAdj_fires =                        -- the firing witness, asserted, not
    hoped
  checkCoverage $ cover 1.0 hasCAonUnderlying
    "split on a referenced underlying, next close in the phantom gap"
    genDerivWithCA
```

When the CA-flag fires the generator constructs the split and prints the next close in the phantom gap, and its firing witness asserts the branch (§2). On every history that clears the precondition the payoff is invariant across the appended terms and the stale-barrier knock — $47.50 \leq 80.00$, a payout on a stock that never fell — does not fire, because the live edge reads the adjusted 40.00 and $47.50 > 40.00$. Strip the corporate-action edge and the barrier stays 80.00, the knock fires, and `prop_termAdjustmentInvariance` reports the phantom the edge exists to refuse.

The settlement-obligation graph, and the fail branch it must walk. The settlement-obligation unit (Chapter 11) is a product graph like any other, tested by the four properties above (Invariant 3.2) — no new machinery. What the generator must not leave unwitnessed is the off-happy-path walk: the fail edge, the cum record-date branch, and the fourth-quadrant mirror branch, each constructed below; a generator that only ever settles would satisfy the four properties without exercising any of them.

```
genSettlementHistory :: Gen History
-- 1. instruct a trade -> mint the settlement-obligation unit at node 'instructed
    ';
-- 2. draw a fail-flag with rate r; when it fires, deliver a settlement-fail event
     so the unit
--    walks instructed -> failed (the SAME unit at its fails-cascade node, never a
     fresh one);
-- 3. draw a cum record-date flag with rate r; drop a record date inside the
    instruction-to-
--    settlement gap on a CUM trade, so the unit raises the market-claim leg (
    deliverer -,
--    cum-buyer +) at its 'instructed' node;
-- 4. draw an ex-settled flag with rate r; settle an EX trade BEFORE the record
    date, so the
--    register shows the buyer and the unit, at its 'settled' node, raises the
    MIRROR
--    market-claim leg (ex-buyer -, seller +) -- the fourth quadrant, reachable
    under a
--    due-bills regime whose ex-date falls after the payment date.
prop_settlementFail_fires =                 -- the firing witness, asserted, not
    hoped
  checkCoverage $ cover 1.0 reachesFailedNode
    "settlement-obligation unit walks instructed -> failed" genSettlementHistory
prop_mirrorClaim_fires =                     -- the fourth-quadrant firing witness
    , asserted
  checkCoverage $ cover 1.0 raisesMirrorClaim
    "ex trade settled before record date raises the mirror market-claim (buyer ->
    seller)"
    genSettlementHistory
```

When the fail-flag fires the generator walks the unit to its `failed` node by construction, so `prop_firedMatchesOneEdge` confirms the fail event lands it there and `prop_actionEqualsRecorded` confirms the emitted transaction is the edge's declared action — the fails-cascade obligation, never a reversal of history. The firing witness asserts the fail branch (§2).

The fourth quadrant needs its own witness. A generator that only ever leaves an ex trade

unsettled at the record date — the coherent-calendar case — raises no mirror leg and never exercises the ex-settled corner. Branch 4 settles an ex trade before the record date by construction, so the record-date firing reads the buyer as registered holder against an ex entitlement and raises the mirror market-claim leg from buyer to seller (Chapter 11); `prop_mirrorClaim_fires` asserts `raisesMirrorClaim` (§2). The branch models the lawful due-bills regime, where an ex trade can settle before the record date, and the property confirms the distribution routes to the entitled seller on the record, not to the registered buyer — the reflection, across the settlement node, of the cum-unsettled market claim.

The partial fill: the split into legs, and the pro-rated claim. A generator that only ever confirms the whole instructed quantity never walks the partial edge, so the split is unwitnessed. Branch 5 draws a partial-flag with rate r; when it fires it confirms q_s with $0 < q_s < Q$ before the record date, so the settlement-obligation unit splits into a settled leg (q_s, node `settled`) and a failed-residual leg ($q_r = Q - q_s$, node `instructed`), with a cum record date in the gap.

```
genSettlementHistory :: Gen History   -- (extended)
-- 5. draw a partial-flag with rate r; when it fires, confirm q_s with 0 < q_s < Q
     before the
--    record date, so the unit walks its partial edge: mint a settled leg (q_s,
    node settled)
--    and a failed-residual leg (q_r = Q - q_s, node instructed); a CUM record
    date sits in the
--    gap, so the residual leg raises the market claim on q_r and later walks
    instructed->failed.

prop_splitConservation :: History -> Property   -- (i) q_s + q_r = Q across the
    split's changes
prop_splitConservation h = isPartialFill h ==> conjoin
  [      settledLegQty h + residualLegQty h === instructedQty h   -- delivery: q_s
     + q_r = Q
    .&&. settledLegPay h + residualLegPay h === instructedPay h   -- payment: pro-
    rata, conserved
    .&&. nodeOf (settledLeg  h) === Settled                       -- both legs'
    UnitStatus nodes
    .&&. nodeOf (residualLeg h) === Instructed                    --   are the
    split's targets
    .&&. parentVector h === zeroVector ]                          -- parent
    retired from the zero vector

prop_proRatedClaim :: History -> Property   -- (ii) claim == entitlement on the
    UNSETTLED qty
prop_proRatedClaim h = isPartialFill h ==>
       claimMinted h === residualLegQty h * ratePerShare h        -- 4,000 * 1.50
    = 6,000
   .&&. claimMinted h =/= instructedQty  h * ratePerShare h       -- NOT the full
    10,000 * 1.50
  -- oracle: entitlement recomputed from the record-date holdings on the unsettled
     quantity
  -- alone -- independent of the minted claim it checks.

prop_split_fires =                      -- the firing witness, asserted, not hoped
  checkCoverage $ cover 1.0 isPartialFill
    "settlement-obligation unit splits: settled leg q_s + failed-residual leg q_r
    = Q"
    genSettlementHistory
```

When the partial-flag fires the generator confirms a strict part before the record date by

construction, so the unit splits and the residual raises its claim on q_r alone; `prop_split_fires` asserts the branch at $\geq 1\%$ (§2). On every partial-fill history the two legs' quantities sum to the instructed Q, both legs' UnitStatus nodes are the split's targets, and the parent's coordinate vector is zero — the retirement of §4. The pro-rated claim equals the entitlement on the unsettled quantity, computed by an oracle from the record-date holdings, and the property fails any design that raises the claim on the whole instructed quantity or carries the residual on one overloaded node. The four properties of Invariant 3.2, already on this surface, now run over the partial branch too: `prop_firedMatchesOneEdge` confirms the partial confirmation lands each successor on its declared node, and `prop_actionEqualsRecorded` confirms the split transaction equals the partial edge's declared action.

Close-out nets within a master, and refuses across masters. Every leg of a close-out transaction must name one master agreement unit, read from each unit's declared terms (Chapter 9); a leg naming a second master is refused at the door. A generator that only ever defaults a counterparty under a single master never exercises the refusal, so the cross-master branch is constructed by hand.

```
prop_crossMasterRefused :: History -> Property
prop_crossMasterRefused h = isCloseOut h ==>
      admitted h ==> Set.size (mastersNamed (closeOutTx h)) === 1
  -- every admitted close-out names exactly one master across all its legs; the
    net is a
  -- projection WITHIN one master, never across -- typing is not needed, the door
    decides.

genMultiMasterDefault :: Gen History  -- draw a cross-master flag with rate r;
    when it fires,
  -- the defaulted counterparty faces two masters M1, M2, each with its own member
     claims;
  -- propose ONE close-out transaction whose legs name BOTH M1 and M2 -> refused
    at the door.
  -- (the lawful path fires one close-out per master, each leg naming only its own
     master.)
prop_crossMaster_fires =                  -- the refusal witness, asserted, not
    hoped
  checkCoverage $ cover 1.0 isCrossMasterCloseOut
    "one close-out transaction naming two masters -> refused"
    genMultiMasterDefault
```

When the flag fires the generator builds a two-master close-out by construction, and the refusal is witnessed on at least 1% of runs (§2). The lawful history fires one close-out per master, each netting only the members whose declared terms name it; the cross-master transaction writes nothing, so no figure can net across masters — the per-agreement identity of the claim units, enforced at the door rather than carried in every type.

The close-out figure is recomputable from the record. The net and CLM-CO's par are a projection: recompute them from the recorded determination observations and the master's declared valuation convention, and the result equals the recorded figure to the minor unit. The oracle recomputes independently of the minted claim it checks.

```
prop_closeOutRecomputable :: History -> Property
prop_closeOutRecomputable h = isCloseOut h ==>
      recomputedNet h === recordedCLM_CO_par h    -- equal to the minor unit
  where recomputedNet h =
          sum [ takerValue (determination h m) (convention h master) | m <-
    members h master ]
```

```
          - collateralApplied h master
        master = masterOf (closeOutTx h)
  -- inputs: the recorded determination observations and the declared close-out-
    mechanics
  -- convention (TA-TERMS) -- both on the record; nothing improvised at default.

genCloseOutHistory :: Gen History  -- draw a default-flag with rate r; when it
    fires, default a
  -- counterparty under a master with >=1 member claim and posted collateral,
    record the
  -- determination prints, and mint CLM-CO at the net -> the recompute branch.
prop_closeOutRecompute_fires =            -- the firing witness, asserted, not
    hoped
  checkCoverage $ cover 1.0 isCloseOut
    "counterparty default: net + CLM-CO par recomputed from the record"
    genCloseOutHistory
```

When the default-flag fires the generator records the determination and mints CLM-CO by construction, and the branch is witnessed (§2). On every close-out history the recomputed net — taker-state member values less collateral applied, under the declared convention — equals the recorded CLM-CO par to the minor unit; a design that prices a member on the wrong side, or improvises the convention at default, is caught as a divergence. This is the regime-bit lesson of Chapter 9 carried onto the close-out figure: the convention is declared data, so two readers of one log compute one figure.

Members retire from the zero vector, markers first. No member claim retires on a nonzero vector, and no owned extinguishment is admitted while collateral marker mass still stands — the coverage ordering (Invariant 14.8) of the knock while pledged (§9.8), now over close-out histories. The property is metamorphic: reorder the close-out so a member retirement is proposed before the marker clears, and the door refuses it.

```
prop_closeOutOrdering :: History -> Property
prop_closeOutOrdering h = isCloseOut h ==> conjoin
  [ all (\c -> retires c h ==> vectorBefore c h === zeroVector) (members h)  --
    zero-vector only
  , markerMassStands h ==> not (ownedExtinguishmentAdmitted h) ]             --
    markers clear first

genCloseOutReorder :: Gen History  -- draw a reorder-flag with rate r; when it
    fires, take a
  -- close-out with posted collateral still on the marker plane and PROPOSE the
    member-claim
  -- retirement (an owned extinguishment) BEFORE the marker-clearing transaction
    -> refused.
  -- (the lawful order clears the marker first, then retires the member from the
    zero vector.)
prop_closeOutOrdering_fires =             -- the refusal witness, asserted, not
    hoped
  checkCoverage $ cover 1.0 isPrematureRetirement
    "member retirement proposed while collateral marker mass stands -> refused"
    genCloseOutReorder
```

When the reorder-flag fires the generator proposes the premature retirement by construction, and the refusal is witnessed (§2). Clear the marker first and the identical retirement admits, taking the member claim's vector to zero; the ordering is forced by coverage, not chosen, exactly as it is at step 6 of the knock while pledged. Together with the two properties above, these pin the close-out algebra: one figure per master, recomputable from the record, retired in the order

coverage compels.

The position-state collection: two entitlements in flight, each retiring alone. PositionState holds a collection keyed by the cause-derived identifier (Chapter 6): two entitlements can be live on one (wallet, unit) pair at once, each retiring exactly when its own obligation discharges. A generator that declares only one dividend at a time would satisfy every balance property without ever placing two facts in one slot.

```
prop_twoEntitlementsRetireIndependently :: History -> Property
prop_twoEntitlementsRetireIndependently h = twoDividendsInFlight h ==>
      -- after both record dates, the pair carries BOTH entitlements, keyed
    distinctly:
       Map.size (posFacts h alpha acme) === 2
      -- paying the first retires exactly its entry; the second stays live to its
    own date:
  .&&. posFacts (afterPay1 h) alpha acme === Map.singleton (txid2 h) (Owed d2)
  .&&. posFacts (afterPay2 h) alpha acme === Map.empty

genTwoDividends :: Gen History  -- draw a two-dividend flag with rate r; when it
    fires, declare
  -- a dividend (record r1, pay p1) and a SECOND (record r2, pay p2) with r1<r2<p1
    <p2, so both
  -- are owed at once and their payment dates are distinct -- the two-in-flight
    branch.
prop_twoInFlight_fires =                    -- the firing witness, asserted, not
    hoped
  checkCoverage $ cover 1.0 twoDividendsInFlight
    "two dividends owed to one holder at once, distinct payment dates"
    genTwoDividends
```

When the flag fires the generator overlaps the two dividends by construction ($r_1 < r_2 < p_1 < p_2$), so between p_1 and p_2 exactly one entitlement is live, and its firing witness asserts the branch (§2). On every history that clears the precondition, both entitlements are keyed distinctly and live together, paying the first retires its entry alone, and the second discharges on its own payment date. The property fails on any design that keeps one slot per pair: the second record-date firing would overwrite the first, and the collection could never hold the two the routine Tuesday puts in flight.

The coordinated cascade: delivered twice, each leg once, all n committed. One cause can move many units: a cascade of n legs sharing one *causeEventId*, each with its own *txid* (the derivation rule of Chapter 4). The property asserts both directions — no leg dropped, no leg duplicated — by delivering the whole cascade twice, reordered.

```
prop_cascadeIdempotence :: History -> Property
prop_cascadeIdempotence h = isCascade h ==>
      -- ALL n commit: the committed identifiers are exactly the cascade's n legs
      Set.fromList (committedTxids h) === Set.fromList (cascadeTxids h)
      -- EACH exactly once: the second delivery adds nothing and drops nothing
  .&&. length (committedTxids h) === n
  where n = length (cascadeTxids h)

genCascade :: Gen History  -- draw a cascade-flag with rate r; when it fires:
  -- 1. one cause on m>=2 units (a corporate action adjusting several referencing
    units)
  --    -> m legs sharing ONE causeEventId, distinct unitId (seq splits any same-
    unit legs);
  -- 2. deliver the m proposals once          -> n=m distinct txids, all commit;
```

```
  -- 3. deliver the SAME m proposals AGAIN, REORDERED (duplicate + reorder)
  --    -> each recomputes its identical txid; the door commits it zero further
    times.
prop_cascadeIdempotence_fires =            -- the firing witness, asserted, not
    hoped
  checkCoverage $ cover 1.0 isCascade
    "n>=2-leg cascade delivered twice, reordered" genCascade
```

When the cascade-flag fires the generator builds an $m \geq 2$-leg cascade and delivers it a second time in permuted order, and its firing witness asserts the branch (§2). On every history that clears the precondition, the committed identifiers are exactly the cascade's n distinct *txid*s — *all* n *commit*, because the legs differ in (*unitId*, *seq*), and *each exactly once*, because the reordered second delivery recomputes identifiers already on the log. Key idempotence on *causeEventId* alone and legs drop; drop the retry check and every leg double-commits — the property fails on both broken designs.

Independent transactions commute; dependent ones do not. Two adjacent admitted transactions with disjoint footprints swap to bit-identical state; two that share a footprint element may not (Proposition 14.2). The pair is tested both ways, and the second half keeps the first honest: without a witnessed non-commuting case, "swapping preserves state" could pass on a generator that only ever draws independent pairs. The first property swaps a disjoint pair and asserts the full state is identical; the second swaps a buy against a record-date event on the *same* stock and wallet and asserts the two orders differ, because the entitlement differs.

```
prop_independentSwapInvariant :: History -> Property  -- Prop. 14 (independent
    commutation)
prop_independentSwapInvariant h = hasDisjointAdjacentPair h ==>
      -- swap the two adjacent txs; full state (balances + three homes) is bit-
    identical.
      -- oracle is fullState . fold, independent of the swap under test (NOT x===x
    ):
      -- one side folds the given order, the other folds the swapped order.
  fullState (fold h) === fullState (fold (swapAdjacent h))

genDisjointPair :: Gen History  -- draw a disjoint-pair flag with rate r; when it
    fires,
  -- build two adjacent txs whose footprints share NOTHING, by construction: draw
    two
  -- DIFFERENT units and two DIFFERENT wallets, so the (w,u) coverage read-fibres
    are
  -- wholly disjoint (Prop. 14: the footprint includes the door's reads at fibre
    grain
  -- -- different agreements on one (w,u) would NOT suffice) and the three homes'
    targets
  -- differ.
prop_independentSwapInvariant_fires =    -- the firing witness, asserted, not
    hoped
  checkCoverage $ cover 1.0 hasDisjointAdjacentPair
    "adjacent pair, disjoint across all four change kinds" genDisjointPair

prop_dependentSwapDiffers :: History -> Property             -- the C-2.7 witness
prop_dependentSwapDiffers h = hasBuyThenRecordDate h ==>
      -- buy ACME then meet record date -> entitled; meet record date then buy ->
    not.
      -- the two orders share the (wallet, ACME) footprint, so they must NOT
    commute:
  entitlement (fold h) =/= entitlement (fold (swapAdjacent h))
```

```
genBuyRecordDate :: Gen History  -- draw a dependent-pair flag with rate r; when
    it fires,
  -- place a buy of ACME into W-ALPHA immediately adjacent to ACME's record-date
    event,
  -- SAME stock, SAME wallet -> shared footprint -> the entitlement depends on the
     order.
prop_dependentSwapDiffers_fires =        -- the non-commuting firing witness,
    asserted
  checkCoverage $ cover 1.0 hasBuyThenRecordDate
    "buy and record-date on the same stock and wallet, adjacent" genBuyRecordDate
```

Both preconditions are genuine, not tautologies, and the two guard against opposite failures. `genDisjointPair` constructs disjointness by drawing two different units, wallets, and agreements, so every generated pair shares no footprint element and the swap is provably harmless; a generator that never produced such a pair would let `prop_independentSwapInvariant` pass without witnessing a single lawful swap. `genBuyRecordDate` constructs the opposite — a buy and a record-date event on one stock in one wallet — so the shared footprint makes the entitlement order-dependent; its oracle, `entitlement . fold`, is computed independently on each order rather than compared to itself, so a green run means the two orders genuinely diverged. Each firing witness asserts its branch (§2). Together the pair pins C-2.7 executably: the harmless swap is bit-identical, the dependent swap is not, and neither branch is vacuous.

15.4 The three binding conditions as executable gates

Three conditions the specification relies on are stated here as gates a candidate implementation must pass, not as intentions it may assert.

(i) Moves are the sole balance mutator. The constitution makes the move the only operation that changes a balance (Chapter 3); the gate makes that mechanically decidable. Applying a transaction, then applying only its move list, must leave identical balances — so a unit-state, position-state, or terms change that touched a balance is caught as a divergence.

```
prop_movesOnly :: Transaction -> Property
prop_movesOnly tx = balances (apply tx l0) === balances (apply (onlyMoves tx) l0)
  where l0 = ledgerBefore tx
-- corollary, tested jointly:  apply (dropMoves tx) l0  leaves every balance fixed
    .
```

(ii) Determinism and replay. The clock is the single input not on the record, and the Event Monitor is the single place it is consulted (Chapter 4); every arrow downstream is a function of the record alone. Two gates follow. *Idempotence:* a duplicated proposal, carrying the same cause-derived identifier, commits once and produces the identical transaction — a duplicate is inert. A late or out-of-order proposal is not a no-op on the tail: the fold order for a unit's fixings is the recorded observation index k — the fixing's execution-time coordinate, distinct from its door-time position, the point at which its print was appended to the log (Chapter 2, Section 2.5) — not the order in which proposals arrive, so the proposal is inserted at its observation position k and the order-dependent accumulator tail $A_k, A_{k+1}, \ldots$ is re-sequenced and recomputed from k forward. *Replay determinism:* re-running the map on the recorded observations *in observation-index order* reproduces every recorded statistic bit-for-bit, because barrier and fixing observation are deterministic — the data crosses the boundary once, as a recorded observation, and is never re-read live (Chapter 8); a late insertion recomputes the tail, and the recomputation is bit-exact.

Episode — the future (T1): a duplicate is inert; a late firing is identical. FUT-IDX settles at recorded IDX closes 1,000, 990, 1,005 (multiplier 10); cumulative variation margin is $+50 - 100 + 150 = 100$ (Chapters 5 and 7). *The gate.* The day-2 settlement event is delivered twice, and delivered late, in a scheduler that reorders it against *unrelated* firings. Reordering against unrelated firings leaves FUT-IDX's own fixing sequence — its observation-index order — untouched, so the contract, a function of the record, emits the identical USD 100 move; the duplicate carries the same cause-derived identifier and commits once at the door. Were day 2 instead reordered *within* FUT-IDX's own sequence — arriving after day 3 — it would be inserted at its observation position and the variation-margin tail recomputed from there, not applied as a late no-op. *Design point. Timeliness depends on the untrusted machines; integrity does not — a duplicate is inert, a firing reordered against unrelated events is identical, and one reordered within its unit's own sequence is re-sequenced and its tail recomputed; all are properties the scheduler tries and fails to break.*

Episode — the variance swap (T5): regenerate firing 127. VS-1 accrues a sum-of-squared-returns statistic in UnitStatus, an integer at scale 10^{-8}, written by each firing for the next (Chapter 6). At the mid-life checkpoint the accrued statistic is $A_{126} = 1{,}805{,}000$, and the previous recorded close is fixing $126 = 1{,}005$. Firing 127 reads A_{126} from the record, forms the return against the recorded closes, and writes A_{127}.

```
accrue :: Close -> Close -> Accrued -> Accrued        -- named, versioned, pure
prop_replay127 :: Property
prop_replay127 = accrue close127 1005 1805000 === recorded A_127
-- the recorded closes are deterministic; A_126 is a recorded integer; so any
    party
-- re-running the same map on the same recorded fixings obtains the identical
    A_127.

-- replay folds the fixings in recorded OBSERVATION-INDEX order (the sequence k),
    never
-- arrival order; a late or out-of-order print is inserted at position k and the
    tail
-- A_k, A_{k+1}, ... is recomputed from k forward:
prop_lateInsertRecomputesTail :: Fixing -> [Fixing] -> Property
prop_lateInsertRecomputesTail late fs =
  foldAccrue (insertAtObsIndex late fs) === foldAccrue (sortByObsIndex (late : fs)
    )
-- the fold is over an ordered integer sequence, so the recomputed tail is bit-
    exact.
```

The recomputation is bit-exact because every input — the accrued integer, the recorded closes — is on the record: a late or out-of-order print recomputes the accrual tail from k forward (`prop_lateInsertRecomputesTail` above), while the downstream CCP-cash figures already posted are corrected by authorised compensation, not left standing. Firing 127 is not trusted to have been right because it ran; it is verified by recomputation. *Economic verification is re-running the map on recorded inputs and comparing; determinism is what makes the comparison bit-exact.*

The global refold, made executable. The per-unit fold above is the special case of the global total order (Chapter 4); these properties pin the global refold, each with a firing witness, since a precondition that never fires witnesses nothing (§2).

```
-- P1: (execTime, doorTime, hash) is a strict total order on the post-absorption
    event set.
prop_totalOrderTotal h = totalStrictOrder (lt keyOf) (admittedDistinct h)
```

```
prop_totalOrderTotal_fires =        -- inject door-time collisions, else the hash
    is unwitnessed
  checkCoverage $ cover 1.0 hasExecTie "distinct events share execTime"
    genPackageHistory
-- P3 (thm:refold): refold == the TIMELY fold of the same external arrivals, the
    Monitor
--   re-deriving firings; NOT a permutation of one fixed set (which is blind to a
    barrier the
--   corrected order newly satisfies). Oracle runs the full pipeline with late
    present throughout.
--   Sound only under the structural finiteness obligations (H-FIN finiteness, H-
    WF; ch:objects): a non-terminating closure hangs, not passes.
prop_refoldEqualsTimely late h = insertsBeforeHead late h ==>
  refoldState late h === timelyState late h
  where timelyState l h = runPipeline (injectAtExec l (worldOf h))    -- Monitor
    may fire anew
        refoldState l h = refold (admitLate l (runPipeline (worldOf h)))
-- Refold idempotence (memo P4-iii): reordering twice equals reordering once.
prop_refoldIdempotent h = foldLedger (reorder (reorder h)) === foldLedger (reorder
    h)
prop_refoldIdempotent_fires =        -- F3: force a FIRST-PASS synthesis, so the
    second pass must
  checkCoverage $ cover 1.0          -- ABSORB the synthesized firing under its
    cid, not duplicate it
    (\l h -> insertsBeforeHead l h && any isSynthesized (firings (refoldState l h)
    ))
    "second reorder absorbs a first-pass synthesized firing"
    genBarrierBreachingCorrection
-- C-12.6: a refold before the head flags the insertion and every changed state,
    and names the
--         PnL explain item; a duplicate cid (same OR differing execTime) changes
    nothing.
prop_reorderFlagged late h = insertsBeforeHead late h ==>
  isReordered late && restatedExactlyChangedStates late h && explainItemNamed late
    h
prop_duplicateAbsorbed dup h = sameCid dup h ==> foldLedger (admit dup h) ===
    foldLedger h
-- Coverage: witness a STATE-CHANGING refold (not just a non-empty tail), and a
    CROSS-UNIT one.
prop_refoldChangesState_fires =
  checkCoverage $ cover 1.0 (\l h -> insertsBeforeHead l h && refoldChangesState l
    h)
    "late arrival restates >= 1 tail state" genOverlappingLateArrival
  where refoldChangesState l h = foldLedger (insertAtOrder l h) /= addOnly l (
    foldLedger h)
prop_reorderCrossUnit_fires =        -- genCrossUnitLateArrival must reach a
    SYNTHESIZED firing on
  checkCoverage $ cover 1.0 (\l h -> distinctUnits (restatedSet l h) >= 2)   -- V,
    not a restated balance
    "refold on U restates a state on unit/agreement V /= U"
    genCrossUnitLateArrival
-- A3: the headline case thm:refold EXISTS for -- a corrected order that NEWLY
    satisfies a
-- data-predicate watch, forcing a firing absent from the as-arrived order (
    monitor time null,
-- cause = the correction). Without this witness the false->true synthesis is
    never generated (S3).
prop_firingSynthesized_fires =
```

```
  checkCoverage $ cover 1.0 newlySatisfies
    "late arrival newly satisfies a watch -> firing synthesized in refold"
    genBarrierBreachingCorrection
  where newlySatisfies l h =
          any (\f -> isSynthesized f && monitorTime f == Null
                     && execTime f == tippingObs l h
                     && cause f == cid l
                     && not (firedIn (asArrived h) f))
              (firings (refoldState l h))
-- H-INERT witness (F3): force a TRUE->FALSE voided firing (covered-call: a record
    -date firing the
-- exercise refold now finds zero shares) and assert the retained firing
    contributes ZERO to fold
-- state -- else H-INERT never fires (every other generator produces false->true,
    a zero-firing gap).
prop_supersededFiringInert_fires =
  checkCoverage $ cover 1.0
    (\l h -> any (\f -> supersededIn (refoldState l h) f
                        && foldContribution (refoldState l h) f == mempty)
                 (firings (asArrived h)))
    "voided firing retained is a fold-state no-op" genCoveredCallAssignment
-- H-WF discharge (RT-B): registration REFUSES a product whose declared cross-unit
     "arms a further
-- watch" relation is cyclic (a decidable static check) and ADMITS the acyclic;
    the coverage forces
-- the cyclic branch so the refusal is exercised, not assumed (no zero-firing gap
    on H-WF).
prop_armingWellFounded =
  forAll genMultiUnitProductGraph $ \g ->
    cover 1.0 (cyclic (armsRelation g)) "cyclic arming graph generated" $
      registers g === (if cyclic (armsRelation g) then Refused else Admitted)
```

The watch properties `prop_firedMatchesOneEdge` and `prop_actionEqualsRecorded` above are stated over the *refolded* state (a recorded firing stands; its superseded consequence takes the C-12.6 treatment, §4.5). Two facts stay assumptions, not theorems, and are labelled so — door-time strict monotonicity is a property-tested obligation (`prop_doorTimeMonotonic`); the enforceability of asserted execution times is TA-EXECUTION-TIME (Chapter 16).

Bitemporality: as-known-at and as-of, made executable. Determinism above folds a unit's fixings in execution-time order; the bitemporal regime pins the two axes of Chapter 2 (Section 2.5) and Theorem 14.5 to generated histories. One property per clause of that theorem.

```
-- BITEMP-1 (as-known-at monotonicity, Theorem 14.4(a)): a correction admitted
    after a
-- prefix never rewrites the view known at that prefix.
prop_bitemp1 :: History -> Prefix -> CorrectiveObs -> Property
prop_bitemp1 hist p c = isBackDated c ==>
  asKnownAt p hist === asKnownAt p (appendAfter p c hist)

-- BITEMP-2 (as-of determinism, Theorem 14.4(b)): the as-of view over the prefix
    that
-- carries the correction recomputes bit-for-bit from a fresh execution-time fold.
prop_bitemp2 :: History -> Property
prop_bitemp2 hist = hasBackDated hist ==>
  asOf p' h' (replayFromScratch p') === asOf p' h' hist
  where (p', h') = horizonIncluding (lastBackDated hist) hist
```

Both share one non-vacuous precondition: the history must contain a *back-dated* observation

— one recorded at a door-time position after its execution-time index k, with k strictly inside the existing sequence ($k <$ end). A generator that only ever appended fixings at the tail would satisfy both properties without ever witnessing a correction. The generator is therefore built to reach the branch and to prove it did:

```
genBackDated :: [Fixing] -> Gen History
-- draw a late-flag with rate r; when it fires, append one observation at
    execution-time index
--   k <- choose (1, maxIndex - 1)   -- strictly inside the sequence: a genuine
    back-dation
-- otherwise append at the tail (the timely path).
prop_bitemp_fires =                        -- the firing witness, asserted, not
    hoped
  checkCoverage $ cover 1.0 isBackDated "corrective insertion (k < end)" (
    genBackDated fs)
```

When the late-flag fires the generator constructs $k <$ end by construction, and its firing witness asserts the branch (§2). On every history that clears the precondition, BITEMP-1 fires and finds the as-known-at view unchanged by the later correction, and BITEMP-2 fires and finds the as-of view recomputed bit-for-bit — the two axes exercised, not merely asserted.

(iii) The temporal residual: a safety and liveness model. The move algebra owns what is true *within* a transaction — conservation, coverage, atomicity, all checked at the door. It does not own the ordering *across* transactions: which interleavings of independent firings are reachable, and whether an open obligation is eventually discharged, are temporal questions the algebra cannot answer. The one-touch knock while pledged is where the residual bites, and it carries an explicit model.

Episode — the one-touch (T2): the knock–revalue–call–discharge model. OT-1 (Cast, §2.1) is pledged to B under agreement G as in §9.8 (collateral value USD 200,000 against exposure USD 150,000). OMEGA's official close prints its barrier close, $79.00 \leq 80.00$. Four steps then interleave: the Monitor emits the knock; OT-1's contract triggers the unit and mints the conditional payment obligation; the pricing layer, reading the triggered state, marks OT-1 at 0, so G's valuation watch opens a margin call for USD 150,000; the payment obligation discharges USD 1,000,000 (USD 850,000 free, USD 150,000 posted to B), and the pledge closes only after the marker returns — an ordering the coverage invariant forces. These steps are separate admitted transactions, and their interleaving is the temporal residual. The model is a state machine over the reachable states, with two invariants held always and one obligation held eventually. What follows is TLA$^+$-style specification written as *illustration*, not a machine-checked proof: no model checker is run inside this specification, and the reference implementation together with its model-checking lies outside it (§5 of the project brief). The operators are read plainly — $\Box$ is "at every reachable committed state," $\rightsquigarrow$ is "eventually leads to," WF is "weak fairness on an action," and $\forall$ is "for all." Safety and Liveness below are therefore *asserted* properties, each with an executable check named in the regime, not theorems certified here:

```
-- TLA+-style ILLUSTRATION (not machine-checked). Operators: [] always, ~> leads-
    to, WF_ weak fairness.
Coverage(s)      == \A (w,u): postedMass(w,u) =< Max(owned(w,u), 0)  -- mass; the
    door's invariant
NoDoubleCount(s) == \A u : extinguished(u,s) => price(u,s) = 0       -- an
    extinguished exposure is not marked
Safety   == [] ( Coverage /\ NoDoubleCount )                        -- SAFETY:
    asserted at every committed state
Liveness == WF_vars(dischargeObligation)                            -- fairness
    hypothesis, INSIDE the formula
            => ( (call = "open") ~> (call = "discharged" \/ call = "closed_out")
```

```
    )  -- LIVENESS: open ~> resolved
-- The condition under which Liveness holds is the WF_ conjunct above, not a
    sentence beside
-- the formula: drop weak-fairness on dischargeObligation and the leads-to need
    not hold.
-- Sufficiency (collateral value >= exposure) is NOT a [] invariant: it is
    lawfully false
-- on the interval [trigger, discharge); only Liveness constrains it. Asserting it
     as an
-- invariant would be a false theorem, and would forbid a lawful state.
```

Safety says no reachable committed state double-counts value or violates coverage. The clause quantifies over units: for every unit whose exposure an admitted lifecycle event has extinguished, the mark is 0. A triggered OT-1 is one instance — no committed state shows both the option's positive mark and the USD 1,000,000 cash it resolves into. Coverage is excluded because it is checked at the door on every one of the interleaved transactions, and the marker-before-extinguishment ordering of the final step is coverage refusing to strip owned while posted mass remains. Safety is not asserted in prose alone: `NoDoubleCount` is exercised by `prop_noDoubleCount` below, witnessed firing on both extinguishing kinds, and `Coverage` is the door invariant every one of the interleaved transactions must clear. *Liveness* says the margin call opened at the revalue is eventually discharged or closed out — the obligation-liveness requirement of the Event Monitor (§14.1, Chapter 16), asserted as a leads-to property and exercised over generated histories as a bounded-horizon discharge check: every opened call reaches a resolved state before its history ends. The fairness under which it holds is the weak-fairness conjunct written *into* the Liveness formula above — the Events Executor eventually delivers each armed firing — not a hope stated beside it; drop that conjunct and the leads-to need not hold. The central discipline is what the model *refuses* to assert: between trigger and discharge the book is lawfully under-collateralised, so sufficiency appears only under $\rightsquigarrow$, never under $\Box$. Coverage — the mass guard — and sufficiency — the value guard — are the two guards of Chapter 9, and here the temporal model is exactly where their difference is drawn: one is a safety invariant asserted always, the other a liveness obligation asserted eventually, and a model that confused them would be unsound in the direction that either fabricates breaches or hides them.

NoDoubleCount fires over every extinguishing event. `NoDoubleCount` quantifies over units, so it must be witnessed on every extinguishing kind. Two kinds extinguish: OT-1's barrier touch (a triggered one-touch prices at 0) and the settled-to-market future's daily settlement (the exposure is owned cash; marking against the futures level would double-count, §7.3). The collateralised-to-market future is the honest contrast: its return obligation survives, it is *not* extinguished, and it prices at the last settled level — the generator must produce it and the property must not zero it. The signatures below are Haskell illustration — the executable form of `NoDoubleCount` and its firing regime, kept separate from the TLA$^+$-style model above so the two notations do not run together in one block:

```
extinguished :: Unit -> State -> Bool                -- an admitted lifecycle
    event has extinguished u
extinguished u s =  triggered u s                    -- OT-1: barrier touched,
    claim resolved
                 || (isFuture u && stm u && vmApplied u s)  -- STM future: day's
    VM settled, exposure gone
                 -- a CTM future is never extinguished: its return-obligation unit
     stays live

prop_noDoubleCount :: History -> Property            -- Safety 14.x, quantified
    over valued units
prop_noDoubleCount h = conjoin
```

```
  [ extinguished u (fold h) ==> price u (fold h) === 0 | u <- valuedUnits h ]

prop_noDoubleCount_fires = forAll genMixedHistory $ \h ->  -- witnesses each kind,
     asserted not hoped
  checkCoverage
  $ cover 1.0 (hasTriggeredOneTouch h) "OT-1 triggered"
  $ cover 1.0 (hasSettledSTMFuture  h) "STM future, VM applied"
  $ cover 1.0 (hasMarkedCTMFuture   h) "CTM future, obligation survives"
  $ prop_noDoubleCount h
```

`genMixedHistory` interleaves three constructions: a knock that triggers a one-touch, a full variation-margin day applied to a future whose declared regime is settled-to-market, and the same day applied to a future whose declared regime is collateralised-to-market. Each of the three labels carries its own firing witness (§2), so `prop_noDoubleCount` is exercised on *both* extinguishing kinds — the OT-1 trigger and the settled-to-market future's settlement — and the collateralised-to-market future is exercised as a non-extinguished unit that must price at its last settled level, never at 0. Because `extinguished` is an extensible predicate rather than a property that holds by construction, each new extinguishing lifecycle event-kind must extend *both* the pricing rule (Definition 7.1, §7.3) *and* the `extinguished` predicate together, or `prop_noDoubleCount` silently under-quantifies and stops witnessing the new kind.

Available free mass is non-negative for every wallet, shorts included. The report's available-mass figure (Chapter 12) carries the same $\max(\cdot, 0)$ as the coverage invariant; without it the figure goes negative for a negative-owned wallet — a plain short, or an issuer wallet conservation creates. The property asserts non-negativity on exactly those wallets.

```
prop_availabilityNonNegative :: History -> Wallet -> Unit -> Property
prop_availabilityNonNegative h w u =
  admitted h ==>
       availableMass h w u >= 0                                  -- the CH.12
    REPORT figure, non-negative
  .&&. availableMass h w u
         === max (ownedBal h w u) 0 - sum [ postedG h w u g | g <- agreements h ]
  -- availableMass is the ch12 availability PROJECTION under test, read from the
    report, NOT
  -- re-derived here: a report shipping (owned - Sigma posted) WITHOUT the max
    returns < 0 for a
  -- negative-owned wallet and fires this red. Distinct from
    prop_coverageSignConvention, which
  -- tests the DOOR invariant and would stay green under exactly that broken
    report.

genNegativeOwned :: Gen History  -- draw a short-flag with rate r; when it fires,
    build a wallet
  -- with owned < 0 (a plain short / issuer wallet) and read its available-mass
    report figure.
prop_availability_fires =        -- the firing witness, asserted, not hoped
  checkCoverage $ cover 1.0 hasNegativeOwnedWallet
    "a negative-owned (short/issuer) wallet in the availability report"
    genNegativeOwned
```

The negative-owned branch is a genuine precondition: a generator that only ever built long wallets would pass `prop_availabilityNonNegative` without ever testing the max; its firing witness asserts the branch (§2). The property reads the Chapter 12 availability figure itself, not a re-derivation of it: on every negative-owned wallet that figure is zero, never negative, and a report that dropped the max — shipping $\text{bal}_{\text{owned}} - \sum_G \text{bal}_{\text{posted},G}$ — returns a negative number and fails the property, exactly where `prop_coverageSignConvention` (the door invariant) would

still pass. That separation is the point: the door bound and the report figure are two statements, and this test pins the report.

The fail transition has exactly one writer, and no orphan. The walk `instructed→failed` is written by the settlement-obligation unit's own contract, on one of the two recorded triggers of Chapter 11 (Invariant 6.2). The property asserts the writer and forbids an orphan failed node.

```
prop_failWriterAttribution :: History -> Property
prop_failWriterAttribution h = reachesFailedNode h ==> conjoin
  [      writerOf t === settlementObligationContract h       -- one writer, named
    .&&. triggerOf t `elem` [FailNotice, DueDateNoConfirm]   -- one of two
    recorded triggers
  | t <- failTransitions h ]  -- EVERY failed node, so a second unit's orphan
    cannot escape
  -- no 'failed' node exists without one of the two triggers on the record.

-- genSettlementHistory (above) already walks instructed -> failed under a fail-
    flag; the writer
-- attribution and trigger presence are asserted on that same branch.
prop_failWriter_fires =          -- the firing witness, asserted, not hoped
  checkCoverage $ cover 1.0 reachesFailedNode
    "instructed -> failed written by the unit's contract on a recorded trigger"
    genSettlementHistory
```

On every history that reaches the failed node, the transition is authored by the settlement-obligation unit's contract and carries one of the two triggers on the record; a failed node written by any other machine, or with no trigger, fails the property — the writerless fail the one-writer rule forbids.

Venue is a custody wallet, not a coordinate: the mark-to-market fold. Sub-custody is a wallet partition (Chapter 7); the mark-to-market NAV is a fold over custody wallets, each priced at its venue's official close. The property asserts that fold and the absence of any venue axis, and it bites when two or more custody wallets hold the same unit at different closes.

```
prop_navMkFoldsOverCustodyWallets :: History -> Unit -> Property
prop_navMkFoldsOverCustodyWallets h u = twoCustodyVenues h u ==>
       navMk h u === sum [ closeAt (venueOf w) * ownedBal h w u | w <-
    custodyWallets h u ]
  .&&. balanceCoordinates h === [Owned, Lent, Posted]  -- NO venue-indexed
    coordinate exists
  -- venue is WHICH custody wallet, an existing dimension; the position cannot
    diverge because
  -- there is no venue axis to hold a divergent figure. balanceCoordinates reads
    the balance
  -- SCHEMA, independent of the fold, so this conjunct is not x===x: add a venue
    coordinate and
  -- it fires red.

genMultiVenue :: Gen History  -- draw a multi-venue flag with rate r; when it
    fires, hold u across
  -- >=2 custody wallets (W_A,W_B,...) whose venues publish DIFFERENT official
    closes.
prop_navMk_fires =            -- the firing witness, asserted, not hoped
  checkCoverage $ cover 1.0 twoCustodyVenues
    "one unit held across >=2 custody wallets at different venue closes"
    genMultiVenue
```

When the flag fires the holding is split across custody wallets at differing closes by construction,

so the cross-venue basis is non-zero and the fold is exercised; its firing witness asserts the branch (§2). The property fails on the design that adds a venue coordinate: the position could then diverge from the wallet-partition sum, and `prop_navMkFoldsOverCustodyWallets` reports the divergence the single position record forbids.

Variance fixings read one-plus-closes, and the off-by-one it refuses. A realised-variance fold over N daily log-returns reads $N+1$ official closes — C_0 at the effective date, then $C_1, \ldots, C_N$ (Chapter 13). The property asserts the count relation, that C_0 is on the record before fixing 1, and that the fold sums exactly N squared returns; every generated variance swap witnesses it.

```
prop_varianceFixingsReadOnePlusCloses :: History -> Property
prop_varianceFixingsReadOnePlusCloses h = isVarianceSwap h ==>
       length (closesRead h)  === nFixings h + 1     -- N returns read N+1 closes
  .&&. referenceClose h `recordedBefore` fixing 1 h  -- C_0 on the record before
    fixing 1
  .&&. length (returns h)     === nFixings h          -- exactly N returns, none
    missing a close
  .&&. length (foldSquares h) === nFixings h          -- fold sums exactly N
    squared returns
  -- nFixings h is the DECLARED schedule count from the product terms (the
    scheduled 252), read
  -- independently of the fold -- NOT length (returns h) -- so these are genuine
    count checks,
  -- not x===x: an implementation reading N closes for N fixings fails the first
    conjunct.

genVarianceSwap :: Gen History
-- draw N in [1..252] so N VARIES; fix a reference close C_0 at the effective date
    ,
-- then N daily closes C_1..C_N; fixing i is the return ln(C_i / C_{i-1}).

prop_varFixings_fires =                       -- the firing witness, asserted, not
    hoped
  checkCoverage $ cover 1.0 isVarianceSwap
    "a variance-swap history: N returns folded over N+1 closes" genVarianceSwap
```

Because C_0 is drawn at the effective date and $C_1, \ldots, C_N$ one per fixing date, every generated variance-swap history clears the precondition by construction, and its firing witness asserts the branch (§2). The property fails exactly on the off-by-one design: read N closes instead of $N + 1$ and C_0 goes unnamed, the fold produces $N - 1$ returns, and `prop_varianceFixingsReadOnePlusCloses` reports the missing close the reference exists to supply. For $N = 252$ the fold reads 253 closes and sums 252 squared returns.

The record is the log: replay reconstructs every observation. Every observation enters the log as a moveless observation-recording transaction (Chapter 8), so replaying a log prefix reconstructs every recorded observation — there is no second store. The property discards everything but the log, rebuilds the whole record, and compares; a generator recording observations outside the log — a second door — fails it at once.

```
prop_replayReconstructsRecord :: History -> Property
prop_replayReconstructsRecord h = hasObservations h ==>
       observations (replayFromLog (logOf h)) === observations h  -- every obs
    rebuilt from the log
  .&&. record       (replayFromLog (logOf h)) === record h        -- and the whole
     record with it
  -- observations h / record h are the generator's INDEPENDENT oracle -- what it
    intended to
```

```
  -- record -- NOT defined as replayFromLog; recording an observation outside the
    log makes
  -- replayFromLog drop it and fires this red (oracle-vs-replay, not x===x). This
    is the decisive
  -- check for D4: it is the record/log collapse made falsifiable.

genObservationHistory :: Gen History  -- draw histories that record >=1
    observation (a close, a
  -- fixing, a barrier print) as a moveless observation-recording transaction on
    the one log.
prop_replayRecord_fires =              -- the firing witness, asserted, not hoped
  checkCoverage $ cover 1.0 hasObservations
    "a history recording >=1 observation as a moveless transaction"
    genObservationHistory
```

Every generated history records at least one observation through the one door, so the precondition clears by construction and its firing witness asserts the branch (§2). The property fails exactly on the second-door design: record an observation anywhere but the log and `replayFromLog` cannot rebuild it, and `prop_replayReconstructsRecord` reports the store the one-door discipline exists to abolish.

The event door: one registry, mandatory routing, and the quarantine that never drops. Every arriving external event is recorded, or its refusal recorded; a kind registers only with routing defined for it, and an event of an unregistered kind is recorded as a *quarantined event* — never dropped, never a bare value (C-4.12). Four properties pin the door.

```
prop_registryTotality :: Registration -> Registry -> Property
prop_registryTotality r reg =
      lacksRouter r ==> refusedAtDoor r                                  -- router-
    less: refused
  .&&. conjoin [ isJust (routerFor reg k) | k <- registeredKinds reg ] -- admitted
    : routing total
  -- the refusal is the DOOR's, not the constructor's (not x===x).
genRegistration :: Gen Registration  -- draw a router-flag rate r: when it fires
    OMIT the router.
genRegistry     :: Gen Registry      -- >=1 registered, routed kind, so the
    totality conjunct
                                     -- never ranges over an empty registry (
    vacuity guard).
prop_registryTotality_fires =
  checkCoverage $ cover 1.0 lacksRouter "registration without a router -> refused"
     genRegistration
prop_registryRouted_fires =
  checkCoverage $ cover 1.0 hasRegisteredKind "a routed, non-empty registry"
    genRegistry
prop_routingDeterminism :: History -> Property
prop_routingDeterminism h = hasCapturedEvents h ==>
  routeAll (replayFromLog (logOf h)) === recordedRouting h
  -- replay-vs-recorded routing: an independent oracle (not x===x).
genCapturedHistory :: Gen History  -- capture >=1 event of a REGISTERED kind;
    replay the prefix twice.
prop_routingDeterminism_fires =
  checkCoverage $ cover 1.0 hasCapturedEvents "a captured event routed on replay"
    genCapturedHistory
prop_quarantineNeverDrop :: History -> Property
prop_quarantineNeverDrop h =
      length (arrivals h) === length (recordedOrQuarantined h)          -- nothing
    dropped
```

```
  .&&. all (\q -> hasProvenance q && hasOpenItem q) (quarantined h)    -- each
    visible, deadline-bearing
  -- arrivals h is the generator's INDEPENDENT delivered-count, never read back
    from the log.
genUnregisteredArrivals :: Gen History  -- unregistered-kind flag rate r: deliver
    an off-registry kind.
prop_quarantineNeverDrop_fires =
  checkCoverage $ cover 1.0 hasUnregisteredArrival
    "unregistered-kind event delivered -> recorded as a quarantined event"
    genUnregisteredArrivals
prop_lateRegistrationReplay :: History -> Property
prop_lateRegistrationReplay h = registersAfterQuarantine h ==>
      -- replayFromLog reads the registry IN FORCE at each log position (Ch. 8, W4
    ; M8):
      fullState (replayFromLog (logOf h)) === fullState (fold h)       -- replay
    bit-identical
  .&&. asKnownAt (beforeRegistration h) h === showsQuarantined h       -- pre-reg
    view still parks it
  -- replay-vs-fold (not x===x); parked processing keyed on the quarantined event'
    s cause.
genLateRegistration :: Gen History  -- late-reg flag rate r: quarantine an off-
    registry event,
  -- THEN register its kind+router, THEN process the held event (cause = the
    quarantined
  -- event), delivered TWICE (idempotence).
prop_lateRegistrationReplay_fires =
  checkCoverage $ cover 1.0 registersAfterQuarantine
    "kind registered after quarantine -> held event processed once, replay bit-
    identical" genLateRegistration
```

Each firing witness asserts its branch (§2): drop the routing-total conjunct and a router-less kind admits; drop the arrival count and a silent drop passes; key the later processing on arrival rather than the quarantined event's cause and replay diverges.

15.5 What an auditor checks

A candidate implementation satisfies this chapter when the checks above run green over the generator universe, each firing witness clearing the $\geq 1\%$ floor (§2): the structural invariants of Chapter 14 hold as invariants and no adversarial history reaches a forbidden state; the economic oracle reproduces every recorded transaction from its recorded event, and a seeded divergence is repaired only by an authorised compensating transaction, never by an edit; the four product-graph properties confirm that watches, state schema, and actions never disagree; across a generated split of a referenced underlying the derivative's payoff is invariant and no stale-barrier knock fires; moves are the sole balance mutator; a duplicate firing is inert and a late firing folds in at its observation index — identical when reordered against unrelated events, its tail recomputed when reordered within its unit's own sequence; firing 127 recomputes bit-for-bit; conservation holds per (unit, coordinate, agreement) and the cross-agreement netting branch — post under G_1, receive under G_2, aggregate posted axis zero — reads the wallet encumbered where a net read would show zero; two dividends in flight are both live in the position-state collection and each retires on its own payment date; a coordinated cascade delivered twice commits each of its n legs exactly once and all n; the bitemporal pair BITEMP-1 and BITEMP-2 hold with the back-dated-observation branch ($k <$ end) exercised; every arriving external event is recorded or refused, no event kind registers without routing defined for it, an event of an unregistered kind is recorded as a quarantined event with provenance and an open item rather than dropped, and a kind registered after quarantine processes its held event once with replay bit-identical; and the knock–revalue–call–discharge model holds its two safety invariants always and its liveness

obligation eventually, with sufficiency asserted only as the latter. Each is mechanical, each is replayable from a seed, and each fails closed. A property that cannot be checked this way is redesigned until it can be: property-based testing is a construction principle here, not a phase at the end.

16 Minimum Requirements

Governed by C-14.1–14.15, C-4.12 (§16.5).

It adds nothing. Each requirement below is a consequence already derived and proved in the body — restated once, imperatively, as the minimum any implementation must meet, and traceable by cross-reference to the chapter that established it. The requirements fall in three groups: the Event Monitor and the Events Executor (§14.1, from Chapter 4); the valuation semantics (§14.2, from Chapter 7); and the data boundary (§14.2, from Chapter 8). A candidate implementation is conformant when every requirement holds; §16.4 states how an auditor confirms it.

Principle 16.1 (What the minima secure)**.** *The ledger's integrity holds by construction and rests on the Transaction Executor alone (Chapter 4). The requirements of this chapter secure the properties integrity does* not *supply: the liveness and replayability of the untrusted machinery, and the reproducibility and boundary-closure of the pricing layers. Integrity owes these components nothing; timeliness, reproducibility, and the closed-system property are conditional on them.*

16.1 The Event Monitor and the Events Executor

The Event Monitor and the Events Executor sit outside the trust boundary (Chapter 4). No failure either can commit corrupts ledger state; every one must degrade to a visible, overdue item rather than to silence.

M1. Durable watches and timers. A watch registered for a future date or condition must fire when the condition is met, across restarts and failures; timer state must be persisted, never held only in volatile memory (Chapter 4). A watch that could be lost on restart is a silent gap, which §14.1 forbids.

M2. At-least-once emission. No emitted event may be lost; the Monitor must re-emit until the event is on the record. A duplicate emission must be harmless, made so by the cause-derived idempotence check at the door (Chapter 4). Losing an event is unrecoverable; emitting one twice costs nothing.

M3. Acknowledged watches. Every declared watch must be acknowledged by the Monitor, and the acknowledgement recorded in the unit's state, so a watch is visibly declared, then armed, then fired or expired. A watch unacknowledged beyond its arming window must stand on the record as an overdue item, never as absence (Chapter 4).

M4. Triggers carry timing, not data. A trigger must say only *when*; everything the event means must be read from the record by the contract it fires (Chapter 4). A trigger carrying data would place an unrecorded input before the door and break reproducibility.

M5. Deterministic, replayable orchestration. Each machine's own state must be reconstructible by replaying its append-only history, so a late firing produces the identical transaction and any party can recompute which events should have been emitted, and when (Chapter 4).

M6. No write privilege. Neither the Event Monitor nor the Events Executor may hold write access to the ledger. The Transaction Executor is the single writer (Chapter 4); an emission or a firing is a proposal, never a commit.

M7. Obligation liveness. Every contractual obligation must be registered with a deadline, a test for its discharge, and a fallback action; by its deadline it must be discharged, compensated, or declared defaulted. A duty that fails to fire must degrade to a visible, overdue item, never to silence. This is the liveness counterpart of the value sufficiency obligation of Chapter 9 and Chapter 14.

M8. No captured event is lost. Every arriving external event must be recorded, or its refusal recorded; an event of an unregistered kind must be recorded as a *quarantined event* with its provenance and a deadline-bearing open item, never returned as a bare value (C-4.12, adopted in v1.3); later processing is idempotent under the cause-derived identifier, over a versioned registry read in force at each position. A dropped arrival is unrecoverable; a quarantined one stands visible until its kind is registered or a person disposes.

16.2 Valuation semantics

Chapter 7 derived six requirements (P1–P6) on the pricing stack and the pricing data layer as consequences of NAV being a projection of the log; they are the minima any pricing layer must meet.

V1. The ledger records pricing outputs and never runs a model. A pricing model must run outside the ledger; its output must re-enter only as a recorded observation carrying sufficient provenance (Chapter 7, P1). Reproducing a valuation from recorded prices is then unconditional; reproducing the model's number requires the model itself, whose retention is a governance matter outside scope (Chapter 17).

V2. Valuation is state-parameterised, and the state comes from the ledger. The price function must read the unit's lifecycle state from UnitStatus and PositionState (Chapter 6), never from a store the pricing layer keeps of its own (Chapter 7, P2). Value that turns on state turns on state the ledger alone authors.

V3. Valuations are reproducible from recorded inputs. Every NAV must be recomputable to the minor unit from the composition — a fold of the log — and prices identified on the record (Chapter 7, P3). No valuation may depend on an input that is not on the record.

V4. Prices and quantities are never mixed across an event. A price and a quantity may be combined only when both refer to the same state of the instrument — consistency of reference (Chapter 7, P4; catalogued in Chapter 14). The mechanism that enforces this at ingestion is requirement B5.

V5. Two price vectors are both projections over one composition. Mark-to-market and mark-to-mid must be two price vectors applied to the single composition the fold produces; positions must never diverge — only the two money figures may (Chapter 7, P5). One position record forbids the settlement-versus-risk position drift that is a classical reconciliation break.

V6. Estimates are kept apart from entitlements. An estimated or model-derived value must never be recorded as, or substituted for, an owed quantity: a mark is a projection, an entitlement is a recorded fact (Chapter 7, P6). The complementary boundary duty — originals preserved, adjusted and derived values recomputed at read — is requirement B4.

16.3 The data boundary

Chapter 8 held the boundary — the one place the closed system is not closed (Chapter 8) — watertight with two devices, the recorded observation and the market data operator, and listed six checks a candidate implementation must pass. A seventh extends the same discipline to the

declared terms of a governing agreement, whose provenance is the executed agreement rather than a market feed (Chapter 9). They are the boundary minima.

B1. All data carries provenance. All data on the record must carry its provenance — source, observation time, and basis frame — and no valuation may read data that does not (Chapter 8, §8.2). A bare read against a live feed leaves no record and is unreproducible; it must be refused.

B2. Kinds are registered before they cross. Every data kind must be registered, with the dimension of each component — price, proportional, or quantity — before any data of that kind crosses; data of an unregistered kind must be refused at the door (Chapter 8, §8.3). Registration is immutable: a correction is a new kind plus a retirement of the old, never an edit.

B3. An operator is declared for every pairing. For every (registered data kind, event kind) pair the boundary admits, a market data operator must be declared — the identity operator where a kind is genuinely unaffected, but declared, not assumed. A pairing with no declared operator must be refused, never defaulted to identity (Principle 8.1, Chapter 8). This is adjustment-schedule totality, and it fails closed.

B4. Originals preserved, adjusted computed at read. All observed data must be preserved on the record exactly as observed; every adjusted value must be recomputed at each read from the original under the declared operators, and every derived quantity recomputed from operator-adjusted inputs (Chapter 8, §8.5–§8.8). An original must never be overwritten by its adjustment.

B5. The invariance witness holds. Across every corporate action the operator on the value frame and the move on the quantity must be bound so that total value is invariant across the event; a stale-frame read — a new count against an old price — or a wrong-side-of-ex read must be refused (Chapter 8, §8.7). This is the ingestion-side enforcement of requirement V4.

B6. Each failure mode has its determined disposition. Each boundary failure — missing data (W1), late data (W2), contradictory sources (W3), unmappable event (W4) — must produce its determined, replayable disposition, a function of log state alone, so replay reproduces it bit-identically; none may improvise or pick silently (Chapter 8, §8.9). In particular a single source for data that materially affects valuation is insufficient: such data must be drawn from more than one attested source, aggregated by a declared rule, with divergence beyond a declared threshold producing a flagged output rather than a silent pick (W3).

B7. Declared agreement terms are reconciled at the boundary. Every declared term of a governing agreement — eligibility, valuation percentage, thresholds, the trapping predicate, and the legal-regime bit — is boundary data whose provenance is the executed agreement; it must be reconciled against the counterparty's own record, and a transcription error repaired by terms amendment plus an authorised compensating rebooking, never a silent edit (Chapter 9; Chapter 8). The regime bit is the worked instance; the duty is general over every declared term.

The boundary rests on five named trust assumptions and no other: TA-KIND, that a kind declaration faithfully renders its publisher's quotation convention, owned by data governance and detected by the invariance witness, source divergence, and recomputation (Chapter 8); TA-TERMS, that a declared agreement term faithfully transcribes the executed agreement (B7), owned by the same governance and detected by boundary reconciliation against the counterparty's own record; and TA-EXDATE, that the recorded ex-date coheres with the record date and the settlement cycle so that a trade's registration state at the record date agrees with

its cum/ex entitlement. *Owner:* the party that records the corporate-action calendar (data governance). *Violation:* an ex trade settles before the record date — lawfully, under a due-bills special dividend whose ex-date falls after the payment date — so the registered holder, the buyer, is not the entitled party, the seller. *Detection:* the entitlement routing itself, which in that fourth quadrant raises the mirror market-claim leg from buyer to seller (Chapter 11) rather than misrouting silently, together with boundary reconciliation of the recorded ex-date against the external corporate-action calendar. *Residual:* the coherence is an external convention nowhere on the record, so a recorded ex-date that misstates it misroutes faithfully to the record — a perimeter reconciliation matter, the sibling of TA-KIND and TA-TERMS. The fourth quadrant is handled on the record either way: the mirror market-claim leg routes the distribution to the entitled seller whether or not the convention held, so TA-EXDATE names a residual the ledger reconciles against, not a gap in the routing. And **TA-ARRIVAL**: the world's events reach the boundary — the set of sources the perimeter listens to covers the events the book's units require. The adopted C-4.12 fixes this bound itself: the quarantine guarantee runs from arrival, and whether the world's events reach the boundary at all is a perimeter matter, named here and reconciled against external authorities, never assumed. *Owner:* perimeter governance. *Violation:* an event the world presented never arrives — a lapsed subscription, a silent source, an unconnected venue — so a fixing, notice, or fill is missed with nothing recorded. *Detection:* for scheduled events, the declared watches themselves — an armed watch whose observation never arrives stands overdue (M1, M3, M7; W1 staleness); for unscheduled events, perimeter reconciliation against external authorities (custodian, venue, corporate-action calendar). *Residual:* the non-arrival of an unscheduled event is invisible from inside the boundary — the exact sibling of TA-KIND's residual. Everything that does cross is recorded, framed, and replayable.

And **TA-EXECUTION-TIME**: the fold trusts each asserted execution time as a legally enforceable, contestable fact of the world — the time a court would enforce (C-2.7). It orders the fold (Chapter 4) and is trusted as asserted, resolved only by a later correction event, never edited at the door. *Owner:* the source, under perimeter governance. *Violation:* a source asserts a wrong execution time — a mis-stamped exercise notice, a print timed to the wrong instant — so the fold sequences an event where the world would not. *Detection:* reconciliation against the counterparty's or clearing house's own record; the wrong time is corrected by a later event that names it (C-12.4), which re-inserts the event at its true execution position and refolds the tail (Chapter 4), never a door edit. *Residual:* an execution time nobody contests is folded as asserted — the sibling of TA-KIND's and TA-ARRIVAL's residuals, not a gap in the fold: everything folded is on the record, framed, and replayable.

16.4 Verification

An auditor confirms conformance by exercising each group — the machine minima (M1–M8), the valuation minima (V1–V6), the boundary minima (B1–B7) — against generated products, events, and histories on the executable-check surface of Chapter 15, each requirement holding and replaying bit-identically (Chapter 14).

16.5 Constitutional traceability

This table is the normative clause-discharge map, replacing the seventeen per-chapter opening sentences that stated it piecemeal and could not be checked for completeness in one place. It is complete and two-way: the constitution declares 116 clauses, C-1.1 through C-Scope.11, each listed once in the left column against its primary discharging chapter (clause-to-chapter total, no gap); an italic *also* note adds a chapter that co-discharges that clause. Every one of the seventeen chapters appears in the right column (chapter-to-clause total). One clause the openers omitted — C-12.6, added in Constitution v1.4 — is discharged in Chapter 4 (§4.5); the table records it, closing the gap the scattered openers hid.

Constitution clause	Discharged in (chapter)
C-1.1–1.5	1
C-2.1–2.8	1; 2.1–2.2 also 12, 2.5 also 15, 2.6 also 13, 2.7 also 4
C-3.1–3.11	2
C-4.1–4.11	3; 4.10–4.11 also 9
C-4.12	8; also 4, 15, 16
C-5.1–5.8	4
C-6.1–6.5	5; 6.5 also 9, 13
C-6.6	17
C-7.1–7.5	6
C-8.1–8.9	7; 8.6–8.9 also 9
C-9.1–9.3	8
C-10.1–10.5	10
C-11.1–11.5, C-12.1–12.5	14
C-12.6	4
C-13.1–13.3	15
C-14.1–14.15	16
C-Auth.1–4	17; Auth.1, Auth.4 also 1
C-Scope.1–11	17; Scope.9 also 11

17 Scope

Governed by C-Scope.1–11, C-Auth.1–4, C-6.6 (§16.5).

Scope fixes the boundary: what the specification covers, and what it reconciles against at its perimeter but never performs. The constitution states the in-scope object in one sentence — the recording, valuation, and lifecycle of trading-book positions held at fair value, and the interface that projects committed activity into settlement; everything below is that sentence made exact, with the external authorities that lie outside it.

17.1 In scope

That sentence is made exact, component by component, across Chapters 2–16: nothing in scope lives outside a chapter, and nothing is left to an implementation to invent.

17.2 Out of scope

The following are external authorities. The ledger reconciles against each at its boundary; it performs none of them.

- **Price formation** — the ledger records the outputs of pricing models and never runs one (Chapter 7; Chapter 16, V1); price discovery is external.
- **Legal agreements** — every right and obligation a contract creates is a unit (Chapter 5); the instrument's legal standing is decided outside the boundary.
- **The settlement infrastructure itself** — finality, payment-system access, custodian and CSD connectivity: the ledger says *what* settles; the infrastructure decides how, and whether (Chapter 11).
- **Regulatory submission** — every report is a projection of the record (Chapter 12); its transmission and per-regime format are external.
- **Reference-data authority** — instrument, counterparty, and calendar mastering: reconciled against at the boundary, never performed.

- **Model governance and retention** — validation, approval, retention of models, and accounting policy. Reproducing a valuation from recorded inputs is unconditional; reproducing a model's number requires the model, whose retention is governance outside scope (Chapter 16, V1).

Five companion documents are named in the Exclusions Register as out-of-scope deliverables — expansions the specification points to but does not contain: the *Worked-Examples Volume* (E71, E72), the *SBL Operations Companion* (E73), the *Track B graphs-interpreter companion* (E74), the *Managed-Account Companion* (E75, the C-6.6 delegation), and the *Simulation Companion* (E76). Each is named so its absence is deliberate and auditable, never silent.

17.3 The open-problems index and the constitutional-parking rule

Two registers live here: open design questions, and parked constitutional conflicts.

The open-problems index. The constitution requires that implementation questions be named and left, not silently dropped; each is carried in full in the *Open-Problems and Escalation Register* companion, and naming is the whole obligation. Three long-standing items are now discharged and kept named: the **close-out and netting algebra** (Chapter 9, Section 9.11), the **right-of-use gate on collateral re-use** (Chapter 14, C-8.8), and the **quantity-aware settlement-obligation unit** (Chapter 11). Named and open: the correction algebra beyond the single compensating transaction; inter-ledger federation and eliminations; **behavioural-event generation**; **corporate-action generation**; **nested simulation**; and the remaining companion items.

The constitutional-parking rule. The second register governs conflict with Constitution v1.4 itself.

Principle 17.1 (Constitutional parking)**.** *A genuine conflict between this specification and the constitution is parked in this index with the exact proposed amendment text — never silently amended, never fudged. Amendment is the owner's alone (the constitution's* Authority and Amendment *section); parking makes the conflict visible and hands the owner a precise, decidable choice.*

Current status: the parking index is fully closed. Five conflicts parked against v1.1 were resolved by the owner's v1.2; the sixth, the event universe, was ratified 2026-07-14 and adopted in v1.3. Each closure keeps its resolution — an index that empties without a trace would be indistinguishable from one never used:

- **PARK-1 (structural) — CLOSED in v1.2.** Clause addressing: v1.2 embeds the margin identifiers, append-only; the separate rendering retired; every chapter opens with the clauses it discharges.
- **PARK-2 (§12) — CLOSED in v1.2.** One timestamp where two axes were needed: C-12.1 restated — both honest answers, indexing delegated here; the two-axis machinery retained (Theorem 14.5).
- **PARK-3 (§4) — CLOSED in v1.2.** Fixed schema against the netting collapse: C-4.10 restated — ownership first, three admission tests, schema delegated here; Section 3.7 demonstrates compliance (Theorem 14.1).
- **PARK-4 (§8) — CLOSED in v1.2.** Unconditional deposit-neutrality against the day-one basis: C-8.7 adopted, conditional on fair value (Chapter 7).

- **C-4.8 clarification — CLOSED in v1.2.** The second door: every guaranteed observation a moveless transaction through the one Executor; the record is the log (Chapters 3, 8, 14).
- **C-4.12 (the event universe) — CLOSED in v1.3.** Event kinds now finite, closed, declared; routing total at registration. The owner *generalised* the ratified text — the quarantine guarantee covers every unprocessable arrival (undeclared kind, unresolvable reference, failed validation) and runs from arrival, the perimeter named as TA-ARRIVAL. Discharged at Chapters 8, 4, 15, 16 (M8).

The parking index is fully closed and live in its history — six conflicts parked with exact text, six decided and adopted by the owner. A future conflict parks here the same way.

17.4 Version 16.1: adopted changes

This specification is rated against Constitution v1.4, declared in force by the owner on 2026-07-16; the document's own ratification date awaits the owner and is not asserted here. Version 16.1 adopts three constitutional changes and rethreads the sections they touch; no other guarantee is altered, and the cast's frozen numbers are untouched.

Adopted. **C-2.7 amended** (Order): every event bears three times (execution, monitor, door); the fold obeys execution order, a late arrival refolds the tail, and the total order is execution time, then door time, then hash; monitor time orders nothing; a duplicate under the cause-derived identifier is absorbed, never ordered. **C-12.6 added**: a reordering refold is never silent — insertion and every changed state flagged, the difference a named profit-and-loss explain item attributed to the reordering, its cause, and the segment of its lateness; settled money moves only as authorised compensation under C-12.4. **TA-EXECUTION-TIME added** beside TA-ARRIVAL: the fold trusts asserted execution times as legally enforceable, contestable facts, resolved by correction events, never edits.

Rethreaded. Chapter 4 carries the three-time envelope, the total order, the reordering step, the end-to-end workflow, and the orchestration substrate (C-2.7, C-12.6, C-2.8); Section 2.5's two axes are re-anchored to door time (what was known) and execution time (what is in force) under C-12.1; the per-unit fixing folds become the special case of the global total order, over which time travel, replay, and the new reordering properties (Chapters 14, 15) are stated. Terminology: *valid time* becomes *execution time*, the former single transaction time splits into *monitor time* and *door time*, and "(v1.3)" clause tags are historical.

Ruled (DL-01). The door's fail-closed refusal of two simultaneous non-commuting events with no declared precedence survives; the door-time tiebreak orders only pairs harmless to reorder or grounded in a declared precedence (Chapter 4). No amendment, no parking.

17.5 Authority

The direction of authority is one-way. This specification is rated against the constitution, and against nothing else; the constitution is rated against nothing. Where a future improvement requires departing from the constitution, the constitution is amended first — through the parking rule above and the owner's decision — never either bent in silence to fit the other.